Πανεπιστήμιο Δυτικής Μακεδονίας

Τμήμα Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Αλγόριθμοι Συνδυαστικής Βελτιστοποίησης για Προβλήματα Χωροθέτησης Θυρίδων Παράδοσης Δεμάτων

Ειρήνη Μαρκαντώνη (ΑΜ: 1857)

Επιβλέπων Καθηγητής: Νικόλαος Πλόσκας

Εργαστήριο Ευφυών Συστημάτων & Βελτιστοποίησης

31 Ιανουαρίου 2026

Περίληψη

Η παρούσα διπλωματική εξετάζει ένα ετερογενές ζήτημα χωροθέτησης θυρίδων παράδοσης δεμάτων (parcel lockers), που ανήκει στην κατηγορία των προβλημάτων χωροθέτησης και πιο συγκεκριμένα αποτελεί μια ειδική περίπτωση του προβλήματος p-median. Επιδιώκεται ο καθορισμός βέλτιστων θέσεων, ώστε να ελαχιστοποιηθεί η συνολική απόσταση που πρέπει να διανύσουν οι πελάτες για εξυπηρέτηση, υπό λειτουργικούς και ταυτόχρονα οικονομικούς περιορισμούς. Αρχικά, το πρόβλημα διατυπώνεται μέσω γραμμικού ακέραιου μοντέλου και επιλύεται με τον εμπορικό λύτη Gurobi. Ακολούθως, αναπτύχθηκαν δύο παραλλαγές αλγορίθμου διακλάδωσης και ορίων. Η πρώτη έκδοση βασίζεται σε εξερεύνηση κατά βάθος και σειριακή επιλογή κόμβου διακλάδωσης, ενώ η δεύτερη έκδοση ενσωματώνει υβριδική στρατηγική εξερεύνησης, ευρετικούς και μεθερευτικούς αλγορίθμους και μεθόδους βελτίωσης προκειμένου να μειώσουμε τον χρόνο εκτέλεσης και να πετύχουμε υψηλή ποιότητα λύσεων. Τέλος, ακολουθεί υπολογιστική μελέτη, η οποία περιλαμβάνει συγκριτική αξιολόγηση των δύο εκδόσεων με βασικά κριτήρια τον απαιτούμενο χρόνο εκτέλεσης, το ποσοστό των προβλημάτων που λύνονται, το μέσο σχετικό σφάλμα και τέλος, τον αριθμό των κόμβων που εξερευνώνται. Τα αποτελέσματα στην πράξη αποδεικνύουν ότι η δεύτερη έκδοση είναι πιο αποδοτική της πρώτης, μειώνοντας τον χρόνο εκτέλεσης, προσφέροντας καλύτερη ποιότητα λύσεων και εξερευνώντας λιγότερους κόμβους στο δέντρο της αναζήτησης του αλγορίθμου.

Λέξεις κλειδιά: ετερογενές πρόβλημα χωροθέτησης θυρίδων παράδοσης δεμάτων, πρόβλημα p-median, ελαχιστοποίηση απόστασης, γραμμικό ακέραιο μοντέλο, αλγόριθμος διακλάδωσης και ορίων, ευρετικοί αλγόριθμοι, μεθευρετικοί αλγόριθμοι

Abstract

This master's thesis investigates a heterogeneous facility location problem for parcel lockers (Parcel Lockers), which belongs to the class of location problems and more specifically, this problem is a specific case of the p-median problem. The objective is to determine the optimal locations in order to minimize the total distance that customers must travel to be served, while simultaneously considering operational and financial constraints. Initially, the problem is formulated as a mixed-integer linear program (MILP) and solved using the commercial solver Gurobi. Subsequently, two variants of a branch-and-bound algorithm were developed. The first model was developed by means of a sequential approach with respect to branching variables and a strategy for exploring the search space through the use of an initial depth-first search (DFS) algorithm; this was followed by a second variant, which included a hybrid search strategy that employed both heuristics and metaheuristics, in addition to procedures for improving the quality of the obtained solutions and reducing the overall amount of computational effort required to find them. A comparative evaluation of the two models was then performed, with the objective of evaluating their behavior in terms of the computational time required to solve each model, the percentage of cases that were solved, the average gap between the upper bound found by the solver and the value of the optimal solution found by the solver, and the number of nodes that had been visited in the search tree. The experimental results demonstrated that, in comparison to the first model, the second model was able to obtain solutions of greater quality and perform faster in finding them and therefore, it was possible to conclude that the second model was superior to the first one.

Keywords: heterogeneous parcel locker location problem, p-median problem, distance minimization, linear integer programming model, Branch and Bound algorithm, heuristic algorithms, metaheuristic algorithms

Δήλωση Πνευματικών Δικαιωμάτων

Δήλωση Πνευματικών Δικαιωμάτων Δηλώνω ρητά ότι, σύμφωνα με το άρθρο 8 του Ν. 1599/1986 και τα άρθρα 2,4,6 παρ. 3 του Ν. 1256/1982, η παρούσα Διπλωματική Εργασία με τίτλο ''Αλγόριθμοι Συνδυαστικής Βελτιστοποίησης για Προβλήματα Χωροθέτησης Θυρίδων Παράδοσης Δεμάτων'' καθώς και τα ηλεκτρονικά αρχεία και πηγαίοι κώδικες που αναπτύχθηκαν ή τροποποιήθηκαν στα πλαίσια αυτής της εργασίας και αναφέρονται ρητώς μέσα στο κείμενο που συνοδεύουν, και η οποία έχει εκπονηθεί στο Τμήμα Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών του Πανεπιστημίου Δυτικής Μακεδονίας, υπό την επίβλεψη του μέλους του Τμήματος κ. Νικόλαο Πλόσκα αποτελεί αποκλειστικά προϊόν προσωπικής εργασίας και δεν προσβάλλει κάθε μορφής πνευματικά δικαιώματα τρίτων και δεν είναι προϊόν μερικής ή ολικής αντιγραφής, οι πηγές δε που χρησιμοποιήθηκαν περιορίζονται στις βιβλιογραφικές αναφορές και μόνον. Τα σημεία όπου έχω χρησιμοποιήσει ιδέες, κείμενο, αρχεία ή / και πηγές άλλων συγγραφέων, αναφέρονται ευδιάκριτα στο κείμενο με την κατάλληλη παραπομπή και η σχετική αναφορά περιλαμβάνεται στο τμήμα των βιβλιογραφικών αναφορών με πλήρη περιγραφή.

Απαγορεύεται η αντιγραφή, αποθήκευση και διανομή της παρούσας εργασίας, εξ ολοκλήρου ή τμήματος αυτής, για εμπορικό σκοπό. Επιτρέπεται η ανατύπωση, αποθήκευση και διανομή για σκοπό μη κερδοσκοπικό, εκπαιδευτικής ή ερευνητικής φύσης, υπό την προϋπόθεση να αναφέρεται η πηγή προέλευσης και να διατηρείται το παρόν μήνυμα. Ερωτήματα που αφορούν τη χρήση της εργασίας για κερδοσκοπικό σκοπό πρέπει να απευθύνονται προς τον συγγραφέα. Οι απόψεις και τα συμπεράσματα που περιέχονται σε αυτό το έγγραφο εκφράζουν τον συγγραφέα και μόνο.

Copyright (C) Ειρήνη Μαρκαντώνη & Νικόλαος Πλόσκας, 2026, Κοζάνη

Υπογραφή Φοιτητή

Περιεχόμενα

1 Εισαγωγή **10**

1.1 Ορισμός του προβλήματος 10

1.2 Κίνητρα και Στόχοι Υλοποίησης 11

1.3 Διάρθρωση κειμένου 12

2 Βιβλιογραφική ανασκόπηση **13**

2.1 Θυρίδες Παράδοσης στο Last-Mile Delivery 14

2.1.1 Θυρίδες παράδοσης και ανασχεδιασμός ροών 14

2.1.2 Εμπειρικά αποτελέσματα και επεκτάσεις δικτύου 14

2.1.3 Αποδοχή χρηστών και παράγοντες ευχρηστίας, εμπιστοσύνης και ευκολίας 15

2.1.4 Λειτουργικοί περιορισμοί και αξιοπιστία λειτουργίας 16

2.2 Μοντέλα Χωροθέτησης Εγκαταστάσεων για Θυρίδες Παράδοσης . . . 17

2.2.1 Βασικά μοντέλα χωροθέτησης για δίκτυα θυρίδων 17

2.2.2 Ενσωμάτωση συμπεριφοράς χρηστών στον σχεδιασμό δικτύου . 18

2.2.3 Χωρητικότητα και ετερογένεια εγκαταστάσεων 19

2.2.4 Αβεβαιότητα και ανθεκτικός σχεδιασμός δικτύου 20

2.2.5 Πολυκριτηριακός σχεδιασμός δικτύων θυρίδων 21

2.3 Ευρετικές και Μεθευρετικές Μέθοδοι 22

2.3.1 Υπολογιστική δυσκολία και περιορισμοί των exact MILP 22

2.3.2 Μεθευρετικές για προβλήματα χωροθέτησης με χωρητικότητα . 23

2.3.3 Μεθευρετικές σε δίκτυα Θυρίδων Παράδοσης με περιορισμούς και ετερογένεια 24

2.3.4 Δυναμική διαχείριση θυρίδων με Reinforcement Learning 25

2.4 Προσβασιμότητα, Συμπεριφορά Χρηστών & Χωρική Ισότητα 25

2.4.1 Προσβασιμότητα ως σχεδιαστική απαίτηση 25

2.4.2 Συμπεριφορά επιλογής και πιθανότητα χρήσης 26
2.4.3 Ετερογένεια χρηστών και παραλαβή "on the way" 27
2.4.4 Χωρική ανισότητα και ασυμφωνία προσφοράς–ζήτησης 28
2.5 Βιωσιμότητα, Εκπομπές & Αστική Κινητικότητα 29
2.5.1 Περιβαλλοντικό ισοζύγιο και όρια συστήματος 29
2.5.2 Αποτελέσματα εφαρμογών σε πόλεις και σενάρια 30
2.5.3 Βιωσιμότητα ως πολυκριτηριακός στόχος 31
2.5.4 Έμμεση βιωσιμότητα μέσω λειτουργικής αξιοπιστίας 32
2.6 Προσομοίωση, Δυναμικά Μοντέλα και Στοχαστική Συμπεριφορά Συστήματος 32
2.6.1 Πολυπεριοδικός σχεδιασμός με προσομοίωση 32
2.6.2 Στοχαστική και ανθεκτική μοντελοποίηση ζήτησης και χωρητικότητας 33
2.6.3 Μικροδυναμική λειτουργία θυρίδων και στενά σημεία ροών . . 34
2.6.4 Δυναμική διαχείριση ζήτησης και αποφάσεις σε πραγματικό χρόνο 35
2.7 Σύνθεση Βιβλιογραφίας & Τοποθέτηση της Διπλωματικής 35
2.7.1 Θυρίδες παράδοσης και ετερογένεια 35
2.7.2 Θυρίδες παράδοσης, συμπεριφορά μετακίνησης και βιωσιμότητα 37
2.7.3 Θυρίδες παράδοσης και Αλγοριθμικός Σχεδιασμός 38

3 Υλοποίηση 40
3.1 Κατασκευή μαθηματικού μοντέλου 40
3.1.1 Μεταβλητές Απόφασης και Παράμετροι 40
3.1.2 Αντικειμενική συνάρτηση και περιορισμοί 42
3.2 Επίλυση μέσω Gurobi 44
3.2.1 Ψευδοκώδικας κατασκευής και επίλυσης μοντέλου με τον λύτη Gurobi 47
3.3 Αλγόριθμος Branch and Bound 47
3.3.1 Υλοποίηση 49
3.4 Απλή Έκδοση Branch and Bound 51
3.4.1 Υλοποίηση 51
3.4.2 Ψευδοκώδικας Απλής Έκδοσης Branch and Bound 52

3.5 Βελτιωμένη Έκδοση Branch and Bound 55
3.5.1 Υλοποίηση 55
3.5.2 Στρατηγική Εξερεύνησης 55
3.5.3 Δομή Κόμβου και Κριτήριο Εξερεύνησης 56
3.5.4 Προεπεξεργασία και Υπολογισμός Μετρικών για Διακλάδωση . 57
3.5.5 Επιλογή Μεταβλητής Διακλάδωσης 58
3.5.6 Τεχνικές Διάδοσης Περιορισμών 62
3.5.7 Βοηθητικές Συναρτήσεις Ευρετικών και Μεθευρετικών Αλγορίθμων 62
3.5.8 Ευρετικοί Αλγόριθμοι 64
3.5.9 Μεθευρετικοί Αλγόριθμοι 71

4 Υπολογιστική μελέτη 83
4.1 Παραγωγή και Οργάνωση Σεναρίων 83
4.2 Σύγκριση Απλής και Βελτιωμένης Έκδοσης 86

5 Συμπεράσματα 96

Κατάλογος σχημάτων

2.1 Ενδεικτική απεικόνιση σταθμού παραλαβής δεμάτων (Parcel Lockers) 13

3.1 Στιγμιότυπο 1 επίλυσης με Gurobi 46
3.2 Στιγμιότυπο 2 επίλυσης με Gurobi 46
3.3 Στιγμιότυπο 3 επίλυσης με Gurobi 47
3.4 Στιγμιότυπο 4 επίλυσης με Gurobi 47
3.5 Παράδειγμα δέντρου Branch and Bound με Depth First Search και Best First Search 51
3.6 Κοινή δομή των ευρετικών αλγορίθμων. 65

4.1 Ποσοστό Λυμένων Προβλημάτων Απλής και Βελτιωμένης Έκδοσης . . 88
4.2 Μέσο GAP(%) ανά Κλάση Προβλήματος και Μέθοδο 90
4.3 Μέσος Αριθμός Εξερευνημένων Κόμβων ανά Κλάση Προβλήματος και Μέθοδο 93

Κατάλογος αλγορίθμων

1 Μοντελοποίηση προβλήματος χωροθέτησης με λύτη Gurobi 48
2 Απλή Έκδοση Branch and Bound (Μέρος 1ο) 53
3 Απλή Έκδοση Branch and Bound (Μέρος 2ο) 54
4 Επιλογή Μεταβλητής με Υβριδική Στρατηγική Μέρος 1ο 60
5 Επιλογή Μεταβλητής με Υβριδική Στρατηγική Μέρος 2ο 61
6 Ευρετικός Αλγόριθμος: Distance Greedy Cover Μέρος 1ο 66
7 Ευρετικός Αλγόριθμος: Distance Greedy Cover Μέρος 2ο 67
8 Ευρετικός Αλγόριθμος: Cost-Aware Greedy 69
9 Ευρετικός Αλγόριθμος: Feasibility-First 70
10 Ευρετικός Αλγόριθμος: GRASP Multistart Μέρος 1ο 72
11 Ευρετικός Αλγόριθμος: GRASP Multistart Μέρος 2ο 73
12 Μεθευρετικός Αλγόριθμος: Variable Neighborhood Descent (VND) Μέρος 1ο 76
13 Μεθευρετικός Αλγόριθμος: VND (Μέρος 2ο) 77
14 Μεθευρετικός Αλγόριθμος: VND (Μέρος 3ο) — Διαδικασία ενημέρωσης 78
15 Μεθευρετικός Αλγόριθμος: Simulated Annealing 79
16 Μεθευρετικός Αλγόριθμος: Iterated Local Search (ILS) 81

Κατάλογος πινάκων

3.1 Βασικές παράμετροι προβλήματος 41
3.2 Παράμετροι μεταφοράς και εκπομπών 42
3.3 Διαφορές Απλής και Βελτιωμένης Έκδοσης Branch and Bound 82

4.1 Σύνοψη παραμέτρων instance generator 85
4.2 Χρόνοι εκτέλεσης (σε δευτερόλεπτα) Απλής και Βελτιωμένης Έκδοσης 87
4.3 Σύγκριση Τιμών Αντικειμενικής Συνάρτησης 89
4.4 Σύγκριση Εξερευνημένων Κόμβων Gurobi, Απλής και Βελτιωμένης . . 92
4.5 Αποτελέσματα μετρήσεων λύτη Gurobi και Βελτιωμένης Έκδοσης . . . 94
4.6 Σύγκριση Μέσου GAP (%) μεταξύ Gurobi και Βελτιωμένης Έκδοσης . 95

Κεφάλαιο 1

Εισαγωγή

1.1 Ορισμός του προβλήματος

Η αδιάλειπτη εξέλιξη του ηλεκτρονικού εμπορίου και των υπηρεσιών ταχυμεταφορών έχει καταστήσει επιβεβλημένη τη σωστή διαχείριση της αστικής διανομής πακέτων στις σύγχρονες πόλεις. Κεντρική θέση σε αυτή τη διαδικασία έχει η ανάγκη για αποτελεσματικά και φιλικά προς το περιβάλλον συστήματα διανομής, τα οποία προσφέρουν εύκολη πρόσβαση στους χρήστες. Οι θυρίδες παραλαβής γνωστές ως Parcel Lockers ή Smart Lockers προσφέρουν μια σύγχρονη και άνετη λύση για την παραλαβή προϊόντων. Προσφέρουν στους πελάτες τη δυνατότητα να έχουν πρόσβαση όλο το 24ωρο, μειώνοντας παράλληλα την ανάγκη για φυσική παρουσία. Η καλύτερη τοποθέτηση αυτών των σημείων παροχής υπηρεσιών ανήκει στην ευρύτερη κατηγορία των προβλημάτων που αφορούν την χωροθέτηση εγκαταστάσεων. Αναλυτικότερα, το ζήτημα μπορεί να διατυπωθεί ως πρόβλημα χωροθέτησης εγκαταστάσεων με περιορισμένη ικανότητα εξυπηρέτησης. Δηλαδή κάθε εγκατάσταση έχει μια συγκεκριμένη χωρητικότητα για την υποστήριξη των πελατών της. Μία από τις πιο διαδεδομένες παραλλαγές είναι το πρόβλημα p-median. Σε αυτή την περίπτωση, οι στόχος είναι να επιλεγούν p σημεία για εγκατάσταση με σκοπό τη μείωση του συνολικού κόστους ή της απόστασης που διανύουν οι πελάτες προς αυτές τις εγκαταστάσεις. Η μέθοδος p-median εφαρμόζεται σε πολλές περιπτώσεις, όπως για την επιλογή τοποθεσιών για κέντρα διανομής, νοσοκομεία, σχολεία, σταθμούς φόρτισης ηλεκτρικών οχημάτων και θυρίδες παράδοσης πακέτων.

Παρόλα αυτά, το κλασικό p-median δε λαμβάνει υπόψη την ετερογένεια των εγκαταστάσεων. Δηλαδή, υποθέτει πως όλες οι εγκαταστάσεις είναι όμοιες, με ίδια

χωρητικότητα και ίδια λειτουργικά χαρακτηριστικά. Αυτό όμως, δεν ανταποκρίνεται στην πραγματικότητα. Συγκεκριμένα, οι θυρίδες εμφανίζουν διαφοροποιήσεις στην χωρητικότητα (πόσα δέματα χωράνε), στο κόστος εγκατάστασης και λειτουργίας αλλά και στις απαιτήσεις χώρου. Ως παράδειγμα, μια θυρίδα μεγαλύτερου μεγέθους είναι κατάλληλη για περιοχές με υψηλή ζήτηση, σε αντίθεση με μια μικρή θυρίδα που ενδείκνυται για περιοχές με μικρότερη ζήτηση ή περιορισμένο χώρο. Ακόμη, η ετερογένεια επεκτείνεται περαιτέρω και στις επιλογές μεταφοράς των πελατών, γεγονός που επίσης αντικατοπτρίζει την πραγματικότητα. Έτσι συνολικά, το ετερογενές p-median απεικονίζει με σαφώς μεγαλύτερη ακρίβεια την πολυπλοκότητα, επιτρέποντας με αυτόν τον τρόπο τον σχεδιασμό δικτύων θυρίδων που ανταποκρίνονται καλύτερα στην πραγματικότητα και στις ανάγκες των πελατών.

Η υπολογιστική πολυπλοκότητα του προβλήματος αυξάνεται καθώς εξετάζουμε τις διαφορετικές κατηγορίες θυρίδων, οι οποίες περιλαμβάνουν τις ανάγκες για διάφορα μεγέθη και τύπους εγκαταστάσεων. Αυτό οδηγεί σε περιορισμούς τόσο στην ικανότητα όσο και στη λειτουργία κάθε διαφορετικής κατηγορίας. Για να διαχειριστούμε αυτήν την πολυπλοκότητα, διατυπώνουμε το ζήτημα ως Μικτό Ακέραιο Γραμμικό Πρόγραμμα (MILP). Η επίλυση μπορεί να πραγματοποιηθεί είτε με τη βοήθεια εμπορικών εργαλείων, όπως ο Gurobi, είτε μέσω εξειδικευμένων αλγορίθμων, όπως ο αλγόριθμος διακλάδωσης και ορίων (Branch and Bound), που εξετάζουν συστηματικά τον χώρο λύσεων.

1.2 Κίνητρα και Στόχοι Υλοποίησης

Η συνεχώς αυξανόμενη ανάγκη για υπηρεσίες ταχυμεταφοράς, σε συνδυασμό με την περιορισμένη ικανότητα των ήδη υπαρχόντων σημείων, καθιστά απαραίτητη τη συστηματική τοποθέτηση θυρίδων παράδοσης. Παράλληλα, οι μεγάλες αποστάσεις που πρέπει να διανύσουν οι παραλήπτες και η αναζήτηση βιώσιμων λύσεων για την τελική φάση καθιστούν τον σχεδιασμό ακόμα πιο περίπλοκο. Η σωστή επιλογή τοποθεσίας είναι κρίσιμη, καθώς επηρεάζει άμεσα τον χρόνο εξυπηρέτησης, την προσβασιμότητα, τα κόστη μεταφοράς και τη γενική περιβαλλοντική απόδοση. Η πολυπλοκότητα αυτή, μαζί με την ποικιλία των εγκαταστάσεων και την αυξανόμενη ζήτηση των πελατών στις αστικές περιοχές, δημιουργεί μια επείγουσα ανάγκη για έναν οργανωμένο και ρεαλιστικό σχεδιασμό του έξυπνου δικτύου θυρίδων. Επομέ-

νως, είναι αναγκαίο να αναλυθεί σε βάθος το πρόβλημα χωροθέτησης των θυρίδων, ώστε τα αποτελέσματα της συγκριτικής αξιολόγησης να συμβάλλουν σε αποδοτικές αποφάσεις για την εγκατάσταση αυτών στον πραγματικό κόσμο.

Η παρούσα διπλωματική εργασία, ωστόσο δεν στοχεύει μόνο στην εκτενή μελέτη του ζητήματος. Στην πράξη, διατυπώνεται το μαθηματικό μοντέλο μέσω μικτού ακέραιου γραμμικού προγραμματισμού (MILP). Σε αυτό, ενσωματώνεται η ετερογένεια στις θυρίδες προκειμένου να απεικονίσουμε πιο ρεαλιστικά το δίκτυο θυρίδων. Ακόμη, αναπτύσσονται και αξιολογούνται αποδοτικοί μέθοδοι επίλυσης του προβλήματος. Πιο συγκεκριμένα υλοποιούνται δύο εκδόσεις του αλγορίθμου διακλάδωσης και ορίων (Branch and Bound). Σχεδιάζεται μια απλή καθώς και μια βελτιωμένη έκδοση του με στόχο τη συγκριτική ανάλυση τους. Τέλος, αυτά τα αποτελέσματα συγκρίνονται με τις βέλτιστες λύσεις που προκύπτουν από τον λύτη Gurobi, προκειμένου να αναδειχθεί η αποτελεσματικότητα των μεθόδων.

1.3 Διάρθρωση κειμένου

Η παρούσα εργασία οργανώνεται σε πέντε κεφάλαια. Το δεύτερο κεφάλαιο ανασκοπεί τη σχετική βιβλιογραφία. Το τρίτο κεφάλαιο, περιγράφει το μαθηματικό μοντέλο που αναπτύχθηκε καθώς και την επίλυσή του με τη χρήση διαφόρων μεθόδων, τόσο ακριβών όσο και προσεγγιστικών. Αυτό ακολουθείται από μια υπολογιστική μελέτη στο τέταρτο κεφάλαιο που στοχεύει στην σύγκριση των μεθόδων. Τέλος, το πέμπτο κεφάλαιο συνοψίζονται τα κύρια συμπεράσματα της εργασίας.

Κεφάλαιο 2

Βιβλιογραφική ανασκόπηση

Οι θυρίδες παραλαβής δεμάτων έχουν γίνει αναπόσπαστο κομμάτι των δικτύων τελευταίου χιλιομέτρου, εφόσον η διαδικασία παράδοσης μεταφέρεται από την οικία του παραλήπτη σε μια σειρά σημείων όπου μπορεί να γίνει η συλλογή. Ωστόσο, ο σχεδιασμός τους πρέπει να εξετάζει θέματα προσβασιμότητας, χωρητικότητας, ζήτησης και κόστους.

Στο Σχήμα 2.1 απεικονίζεται μία από τις πολλές ποικιλίες θυρίδων παράδοσης που εφαρμόζονται σε αστικά πλαίσια. Αυτό το κεφάλαιο επικεντρώνεται σε έρευνες που αναλύουν τις λειτουργικές και περιβαλλοντικές επιπτώσεις των θυρίδων, τα μοντέλα χωροθέτησης εγκαταστάσεων καθώς και το γραμμικό προγραμματισμό με ακέραιες μεταβλητές (MILP). Επιπλέον, ενσωματώνει ακριβείς, προσεγγιστικές, ευρετικές και μεθευρετικές τεχνικές, ανάλογα με την έκταση και τη δυσκολία του κάθε προβλήματος.

Σχήμα 2.1: Ενδεικτική απεικόνιση σταθμού παραλαβής δεμάτων (Parcel Lockers)

2.1 Θυρίδες Παράδοσης στο Last-Mile Delivery

2.1.1 Θυρίδες παράδοσης και ανασχεδιασμός ροών

Οι θυρίδες παράδοσης λειτουργούν ως μηχανισμός αναδιοργάνωσης των διανομών. Αντί να υπάρχουν διάσπαρτες στάσεις στη διαδικασία παράδοσης, η ζήτηση συγκεντρώνεται σε συγκεκριμένα σημεία παραλαβής. Αυτή η προσέγγιση διευκολύνει την ενοποίηση των στάσεων, μειώνει τις επαναλαμβανόμενες διαδρομές και περιορίζει τα κρούσματα μη έγκυρης παράδοσης. Επιπλέον, η αποτελεσματικότητα της ενοποίησης επιβεβαιώνεται και μέσω εμπειρικών στοιχείων. Στην περίπτωση του Άμστερνταμ, οι στάσεις μειώθηκαν περίπου κατά 68%, από 1475 σε 477, εξαιτίας της εισαγωγής θυρίδων για την παραλαβή πακέτων (van Duin et al.[1]). Ωστόσο, η λειτουργία ενός τέτοιου συστήματος δεν είναι εύκολη, καθώς η μείωση των διαδρομών για τον προμηθευτή μπορεί να οδηγήσει σε περισσότερες μετακινήσεις για τους πελάτες. Μοντέλα που αξιολογούν την απόδοση με τη χρήση διαφόρων δεικτών, αντί να εστιάζουν μόνο στην ανάθεση στον πιο κοντινό κόμβο, υποστηρίζουν αυτήν την προσέγγιση. Αυτές οι μέθοδοι καθορίζουν ξεκάθαρα τις αντισταθμίσεις, όπως είναι η απόσταση του προμηθευτή σε σχέση με την απόσταση που διανύει ο πελάτης ή το επίπεδο ικανοποίησης των χρηστών (Eliyan et al.[2]). Επιπλέον, οι συνθετικές αξιολογήσεις των διαθέσιμων πληροφοριών αποκαλύπτουν τις ισορροπίες μεταξύ του κόστους, της κάλυψης και της ποιότητας των υπηρεσιών ως μια κύρια τάση στον τομέα. Επισημαίνουν επίσης ότι η απόδοση των θυρίδων παραλαβής πακέτων εξαρτάται σε σημαντικό βαθμό αποκλειστικά από τις συνήθειες πρόσβασης των χρηστών (Zhang & Demir[3], Campisi & Di Ruocco[4]).

2.1.2 Εμπειρικά αποτελέσματα και επεκτάσεις δικτύου

Οι πιο πειστικές αποδείξεις για τις θυρίδες βασίζονται σε πραγματικά δεδομένα. Στο Άμστερνταμ, η μερική μετάβαση από κατ' οίκον παράδοση σε θυρίδες οδηγεί σε μικρότερες αποστάσεις και καλύτερη αξιοποίηση δρομολογίων, καθώς οι παραδόσεις ενοποιούνται (van Duin et al.[1]). Στο Όσλο όμως παρατηρείται διαφορετική δυναμική. Αν και η επέκταση του δικτύου μειώνει κόστος, κυκλοφορία και ρύπους, υπάρχει όριο: πάρα πολλές θυρίδες διασπούν τις ροές και ακυρώνουν το πλεονέκτημα της συγκέντρωσης (Pinchasik et al.[5]). Εντυπωσιακό είναι ότι αυτή

η μελέτη βασίζεται σε δεδομένα ταξιδιωτικών συμπεριφορών που προήλθαν από εθνική έρευνα. Καταγράφει τα ποσοστά χρήσης πέντε διαφορετικών τρόπων μεταφοράς για την παραλαβή δεμάτων από θυρίδες. Αυτοί περιλαμβάνουν το περπάτημα, τη χρήση ποδηλάτου, δημόσιες συγκοινωνίες, αυτοκίνητα με εσωτερική καύση και ηλεκτρικά οχήματα. Επιπλέον, η τελική περιβαλλοντική και κοινωνική επίδραση εξαρτάται από τον τρόπο που οι παραλήπτες μετακινούνται για να παραλάβουν τα αγαθά (όπως με τα πόδια ή με ΙΧ). Έτσι, δεν είναι αρκετή μόνο η εξοικονόμηση από την πλευρά του παρόχου (Pinchasik et al.[5], Hovi & Bø[6]). Αυτά τα αποτελέσματα σχετίζονται άμεσα με την προσέγγιση της πολυκριτηριακής αξιολόγησης: η καλύτερη λύση προκύπτει από μια ισορροπία μεταξύ κόστους, ποιότητας εξυπηρέτησης και λειτουργικής απόδοσης, αντί για μια απλή βελτιστοποίηση σε μία διάσταση (Sawik et al.[7]).

2.1.3 Αποδοχή χρηστών και παράγοντες ευχρηστίας, εμπιστοσύνης και ευκολίας

Οι θυρίδες μπορεί να βελτιώνουν τους επιχειρησιακούς δείκτες του παρόχου, όμως το πραγματικό τους όφελος κρίνεται τελικά από το πώς τις αντιλαμβάνονται οι ίδιοι οι χρήστες. Η εμπειρία τους δεν εξαρτάται μόνο από την τεχνολογία, αλλά από πολύ καθημερινές λεπτομέρειες: αν η θυρίδα βρίσκεται σε σημείο που τους βολεύει, αν νιώθουν ασφαλείς στον χώρο, αν η παραλαβή ή η επιστροφή δημιουργεί μικρά αλλά ενοχλητικά εμπόδια. Έτσι, η αποδοχή του συστήματος δεν είναι αυτονόητη ούτε προκύπτει απλώς επειδή «είναι εύκολο» (Lemke et al.[8], Campisi & Di Ruocco[4]). Το σημείο είναι μεθοδολογικά κρίσιμο. Μοντέλα που υποθέτουν αυτοματοποιημένη συμπεριφορά, όπως για παράδειγμα ότι οι χρήστες επιλέγουν πάντα το πιο κοντινό σημείο για εξυπηρέτηση, μπορεί στη θεωρία να παράγουν βέλτιστες λύσεις όμως αβέβαιες στην πράξη λόγω πολύπλευρων περιορισμών. Η βιβλιογραφία που βασίζεται στην ανάλυση επιλογών, επιχειρεί να καλύψει αυτό το κενό. Μετατρέπει την απόσταση και τα χαρακτηριστικά υπηρεσίας σε πιθανότητες επιλογής. Το Πολυκατηγορικό Μοντέλο Πιθανοτήτων Επιλογής (Multinomial Logit, MLN), λαμβάνει υπόψη το αν ο πελάτης βρίσκεται στο σπίτι του κατά την στιγμή της παράδοσης. Όταν συνδυάζεται με τεχνικές βελτιστοποίησης αποδεικνύει πως η βέλτιστη χωροθέτηση δεν είναι σταθερό κριτήριο. Στην πράξη, μεταβάλλεται καθώς

η ζήτηση προσαρμόζεται στα σημεία που είναι πραγματικά διαθέσιμα, αντί να αντιμετωπίζεται ως κάτι εκ των προτέρων καθορισμένο και ντετερμινιστικό (Peppel & Spinler[9], Lin et al.[10]). Με παρόμοιο τρόπο, το Μοντέλο Luce με Όρια Αποδοχής Επιλογών (Threshold Luce Model, TLM) δείχνει ότι οι χρήστες δεν αξιολογούν απεριόριστες εναλλακτικές επιλογών. Λαμβάνουν υπόψη πιο στενό σύνολο επιλογών, περιορίζοντας την προσοχή τους μόνο σε όσες βρίσκονται εντός ορίων. Η λογική αυτή συνδέεται άμεσα με τις ακτίνες προσβασιμότητας και τα όρια αποδοχής που διαμορφώνουν οι ίδιοι οι χρήστες (Lin et al.[11], Rodríguez-Esparza et al.[12]). Τέλος, ζητήματα που αφορούν τη χωρική ισότητα καθώς και αναντιστοιχία ανάμεσα σε προσφορά και ζήτηση δείχνουν ότι η αποδοχή μιας υπηρεσίας δεν είναι απλώς προσωπικό χαρακτηριστικό. Επηρεάζεται δηλαδή από τον τρόπο που κατανέμονται οι υποδομές στον χώρο. Περιοχές με περιορισμένη πρόσβαση ή συχνή συμφόρηση τείνουν να εμφανίζουν χαμηλότερη χρήση και μεγαλύτερες ανισότητες στο κόστος πρόσβασης (Ding et al.[13], Campisi & Di Ruocco[4]).

2.1.4 Λειτουργικοί περιορισμοί και αξιοπιστία λειτουργίας

Στην πράξη, η επιλογή των σημείων εγκατάστασης δεν αρκεί από μόνη της αν δεν εξασφαλίζεται ότι η υπηρεσία θα λειτουργεί με συνέπεια. Παράγοντες όπως είναι η χωρητικότητα, η διάταξη των θυρίδων καθώς και ο χρόνος που απαιτείται για κάθε παραλαβή δημιουργούν ουσιαστικές επιχειρησιακές προκλήσεις. Η εμφάνιση καταστάσεων όπως συμφόρηση ή ακόμη και υπερχείλιση μπορούν πολύ εύκολα να μειώσουν ένα μεγάλο μέρος των πλεονεκτημάτων που υπόσχεται το σύστημα. Μελέτες έχουν προτείνει συγκεκριμένες ποσότητες καθώς και διαστάσεις για διαφορετικούς τύπους θυρίδων, προσφέροντας χρήσιμες κατευθύνσεις για τον σχεδιασμό δικτύων (van Duin et al.[1]). Ταυτόχρονα, έρευνες που μελετούν τη σχέση ανάμεσα στη θέση και στη διάταξη των θυρίδων δείχνουν ότι οι διαστάσεις τους επηρεάζουν άμεσα την αξιοπιστία. Ιδιαίτερα, όταν αλλάζουν τα μεγέθη των δεμάτων ή ο χρόνος παραμονής τους στο σύστημα (Kahr[14], Eliyan et al.[2]). Στην μοντελοποίηση, παράμετροι όπως η διαθέσιμη χωρητικότητα και οι χρονικοί περιορισμοί μπορούν να μεταβάλουν σε μεγάλο βαθμό τις ιδανικές λύσεις. Στην πράξη, ένας κόμβος που μπορεί να φαίνεται γεωγραφικά ιδανικός μπορεί μετά να εξελιχθεί σε σημείο συμφόρησης. Αυτό συμβαίνει αν δεν έχει σχεδιαστεί σωστά ως προς το φορτίο που μπορεί να

εξυπηρετήσει. Έτσι, κατά τον σχεδιασμό χρειάζεται να πετύχουμε ισορροπία ανάμεσα στην εύκολη πρόσβαση και στην αποφυγή υπερφόρτωσης (Rodríguez-Esparza et al.[12], Pinchasik et al.[5]). Προσομοιώσεις που χρησιμοποιούν μοντέλα βασισμένα σε πράκτορες δείχνουν ότι οι ουρές και οι συσσωρεύσεις δημιουργούνται λόγω χρονικού συγχρονισμού και τυχαίας συμπεριφοράς κατά τη διαδικασία παραλαβής, και όχι μόνο εξαιτίας υψηλής μέσης ζήτησης (Roșca et al.[15]). Έτσι, ο στρατηγικός σχεδιασμός απαιτεί έγκριση σύμφωνα με τη λειτουργική δυναμική (Sawik et al.[7]). Η σχετική βιβλιογραφία για τη δυναμική διαχείριση της ζήτησης εστιάζει περισσότερο στην καθημερινή λειτουργία παρά στη θέση. Πολιτικές που λαμβάνουν υπόψη την μελλοντική διαθεσιμότητα ενισχύουν την αξιοπιστία και περιορίζουν τις αναδρομολογήσεις (Sailer et al.[16], Roșca et al.[15]). Αυτό συνδέεται άμεσα με την παρούσα μελέτη, καθώς η εστίαση σε διαφορετικές κατηγορίες θυρίδων (τρεις τύποι με ποικιλία χωρητικοτήτων) και η υπερχείλιση αντικατοπτρίζουν τη λειτουργική πραγματικότητα. Συγκεκριμένα, οι χωρητικότητες, οι τύποι θυρίδων και οι πολιτικές δυναμικής κατανομής συνιστούν στρατηγικές αποφάσεις σχεδιασμού και δεν είναι δευτερεύοντα στοιχεία (Sailer et al.[16], Kahr[14]).

2.2 Μοντέλα Χωροθέτησης Εγκαταστάσεων για Θυρίδες Παράδοσης

2.2.1 Βασικά μοντέλα χωροθέτησης για δίκτυα θυρίδων

Ο σχεδιασμός ενός δικτύου θυρίδων για την παραλαβή δεμάτων βασίζεται στις παραδοσιακές μεθόδους χωροθέτησης εγκαταστάσεων. Σε αυτό το σενάριο, καθορίζονται οι θέσεις των θυρίδων καθώς και ο τρόπος κατανομής των πελατών. Οι Deutsch & Golany[17] απέδειξαν ότι όταν πληρούνται ορισμένες προϋποθέσεις, το θέμα μπορεί να αναπαρασταθεί ως ένα δυαδικό ακέραιο γραμμικό μοντέλο που σχετίζεται με τη χωροθέτηση εγκαταστάσεων χωρίς περιορισμούς στην ικανότητα (Uncapacitated Facility Location Problem, UFLP). Στην πραγματικότητα, αυτή η αντιστοίχιση παρέχει μια σαφή μαθηματική βάση, ενώ παράλληλα αναδεικνύει και τις ελλείψεις που υπάρχουν στα πραγματικά δίκτυα. Αυτό σημαίνει ότι υπάρχουν θέματα σχετικά με την χωρητικότητα, την ποικιλία των διαστάσεων των θυρίδων, και φυσικά, τους μηχανισμούς μέσω των οποίων οι χρήστες αποφασίζουν πού θα

στραφούν. Ακολούθως, ερευνητικές εργασίες όπως αυτές των Eliyan et al.[2] επεκτείνουν την έννοια του UFLP, χρησιμοποιώντας διατυπώσεις που περιλαμβάνουν τους αναγκαίους συμβιβασμούς μεταξύ της αποδοτικότητας, των αποστάσεων και της βιωσιμότητας. Η χωροθέτηση, άλλωστε, δεν είναι αυτοσκοπός. Η επιτυχία της κρίνεται από το κατά πόσο καλά λειτουργεί σε πραγματικές υπηρεσίες. Αν δηλαδή οι θυρίδες είναι εύκολα προσβάσιμες, αν το επίπεδο εξυπηρέτησης παραμένει υψηλό και αν οι λειτουργικές συνέπειες είναι διαχειρίσιμες. Αυτή η έρευνα εξετάζει τη μεταβολή από το παραδοσιακό πρόβλημα χωροθέτησης εγκαταστάσεων σε ένα ζήτημα χωροθέτησης που περιλαμβάνει περιορισμούς σχετικά με την ικανότητα και τις ετερογενείς εγκαταστάσεις. Το μοντέλο στηρίζεται στην κύρια μορφή για το άνοιγμα εγκαταστάσεων και την ανάθεση πελατών, όμως το επεκτείνει ώστε να συμπεριλάβει επιλογές για τον τύπο των εγκαταστάσεων. Οι καταστάσεις και οι περιορισμοί πρόσβασης (Krarup & Pruzan[18]). Η μαθηματική δομή έχει τη σημασία της, αλλά η επιλογή μιας εγκατάστασης εξαρτάται σε μεγάλο βαθμό από τις προτιμήσεις των χρηστών όσον αφορά την πρόσβαση και τη συμπεριφορά τους.

2.2.2 Ενσωμάτωση συμπεριφοράς χρηστών στον σχεδιασμό δικτύου

Η ανάθεση των χρηστών σε σημεία εξυπηρέτησης δεν είναι μια αυτόματη ή απολύτως μηχανική διαδικασία. Δεν είναι θέμα επιλογής του πιο κοντινού σημείου. Στην πραγματικότητα, η επιλογή αυτή επηρεάζεται από το πόσο εύκολη θεωρεί ο χρήστης την πρόσβαση, το κόστος της μετακίνησης καθώς και ποιες εναλλακτικές θεωρεί ο ίδιος αποδεκτές. Οι Lin et al.[11] παρουσιάζουν ένα μοντέλο που ενσωματώνει αυτές τις συμπεριφορικές πτυχές μέσω του Threshold Luce Model (TLM). Συγκεκριμένα, το εφαρμόζουν σε ένα πρόβλημα μεγιστοποίησης κέρδους. Το βασικό σημείο είναι πως ο χρήστης δεν εξετάζει όλες τις πιθανές επιλογές. Αντίθετα, λειτουργεί μέσα σε ένα περιορισμένο σύνολο από εναλλακτικές που «περνούν το φίλτρο» της αποδοχής του—για παράδειγμα, σημεία εντός 2 χιλιομέτρων. Έτσι, ένας αυστηρός κανόνας όπως για παράδειγμα η μη αποδοχή αποστάσεων πέρα από ένα όριο) ενσωματώνεται σε μια πιο ρεαλιστική περιγραφή της ανθρώπινης επιλογής. Στο ίδιο πνεύμα, οι Rodríguez-Esparza et al.[12] συνδυάζουν την προσέγγιση TLM με επιπλέον πρακτικές παραμέτρους. Δηλαδή, χωρητικότητα και χρονικά παράθυρα λειτουργίας. Με αυτή την πιο πλούσια περιγραφή, δείχνουν ότι η βέλτιστη χωροθέτηση δεν εξαρ-

τάται μόνο από τη γεωγραφική διάταξη, αλλά ταυτόχρονα και από τους κανόνες λειτουργίας του συστήματος. Τα μοντέλα που ενσωματώνουν τις πραγματικές επιλογές των χρηστών μετατοπίζουν τελικά το επίκεντρο του προβλήματος. Από τη στενή ελαχιστοποίηση της απόστασης στην προσπάθεια βελτιστοποίησης της ανταπόκρισης της ζήτησης μέσα σε πλαίσιο περιορισμένης προσβασιμότητας. Με άλλα λόγια, δεν είναι μόνο «πού είναι πιο κοντά», αλλά «πού είναι ρεαλιστικά πιθανό να πάει ο χρήστης»

2.2.3 Χωρητικότητα και ετερογένεια εγκαταστάσεων

Η ρεαλιστική αναπαράσταση των θυρίδων προϋποθέτει την κατανόηση της ετερογένειας: περιορισμοί στη χωρητικότητα, επιλογή του είδους εγκατάστασης και η εσωτερική διάταξη των θυρίδων. Η σχετική βιβλιογραφία εξετάζει αυτή τη σύνθετη κατάσταση σε τρία διαφορετικά επίπεδα. Καταρχάς, υπάρχει το πρόβλημα χωροθέτησης εγκαταστάσεων με περιορισμούς στη χωρητικότητα (CFLP), στο οποίο κάθε εγκατάσταση διαθέτει ένα όριο εξυπηρέτησης (για παράδειγμα, μέγιστο 50 δέματα την ημέρα). Εάν αυτό το όριο ξεπεραστεί, τότε προκύπτει υπερχείλιση. Δεύτερον, υπάρχουν ζητήματα που σχετίζονται με διάφορες χωρητικότητες και διαθέσιμα μεγέθη θυρίδων. Εκτός από την απόφαση για το άνοιγμα της εγκατάστασης, πρέπει να επιλέξεις και τον τύπο που προτιμάς (όπως μικρό, μεσαίο ή μεγάλο), καθώς αυτό επηρεάζει τόσο τη χωρητικότητα όσο και το κόστος. Αφετέρου, υπάρχουν ζητήματα που συνδυάζουν τον εντοπισμό, την ικανότητα και τη διάταξη. Αυτό υποδηλώνει τον καθορισμό της τοποθεσίας όπου θα λειτουργήσει η εγκατάσταση, ποια ζήτηση θα καλύψει και πώς είναι οργανωμένη στον διαθέσιμο χώρο.

Οι Schnieder et al.[19], βασίζονται σε πραγματικά δεδομένα από τη βιομηχανία θυρίδων του Λονδίνου. Συγκεκριμένα, υπολογίζουν το συνολικό κόστος των θυρίδων μέσω της Καθαρής Παρούσας Αξίας (NPV) για ένα διάστημα 15 ετών. Σε αυτόν τον υπολογισμό εφαρμόζουν προεξοφλητικό συντελεστή 1%. Με αυτόν τον τρόπο, λαμβάνονται υπόψη η αρχική δαπάνη για την εγκατάσταση καθώς και τα ετήσια λειτουργικά έξοδα. Ο Kahr[14] προσεγγίζει το ζήτημα ως μια ενιαία απόφαση που αφορά τόσο τη θέση όσο και τη διάταξη. Εστιάζει δηλαδή, στην προσαρμογή των θυρίδων στις απαιτήσεις της αγοράς. Από την πλευρά τους, οι Raviv & Gansterer[20] συνδυάζουν τα στοιχεία χωροθέτησης, χωρητικότητας και διάταξης σε ένα τριπλό

μοντέλο, δίνοντας προσοχή στο ότι η χωρητικότητα δεν είναι μόνο ένας απλός περιορισμός αλλά λειτουργεί ταυτόχρονα ως μηχανισμός αξιοπιστίας. Όταν οι διαστάσεις δεν έχουν ρυθμιστεί σωστά, το σύστημα αρχίζει να «μπουκώνει»: εμφανίζονται συσσωρεύσεις, υπερχειλίσεις, πτώση της ποιότητας και τελικά αλλοίωση της πραγματικής εικόνας της ζήτησης. Οι Zhou et al.[21] δείχνουν τον τρόπο που η επιλογή μεγέθους και η ποικιλία χρηστών επηρεάζουν την ιδανική λύση. Η χωροθέτηση συνιστά μια δυναμική επιλογή, συνδέεται με τα πρότυπα κίνησης και παραλαβής. Επιπλέον, η ποικιλία των χρηστών δεν αποτελεί πλέον δευτερεύον στοιχείο αλλά γίνεται κύριο χαρακτηριστικό του σχεδιασμού. Σε πιο μαθηματικό επίπεδο, οι El Amrani et al.[22] προτείνουν λύσεις για ζητήματα p–median με περιορισμούς χωρητικότητας, χρησιμοποιώντας επίπεδα πολλαπλών χωρητικοτήτων και συνδυάζοντας τόσο ακριβείς όσο και προσεγγιστικές μεθόδους. Στα δίκτυα θυρίδων παράδοσης, η διαδικασία διευκολύνει την αντιστοίχιση των τύπων θυρίδων σύμφωνα με τις κατηγορίες χωρητικότητας. Την ίδια στιγμή, καθορίζονται σταθερές τιμές, περιορισμοί και ανταλλαγές μεταξύ διαφορετικών προορισμών. Αυτή η μελέτη εφαρμόζει αυτή την αρχή επιλέγοντας τον τύπο εγκατάστασης για κάθε θέση και θέτοντας όρια στη χωρητικότητα, προκειμένου να διασφαλίσει ότι η ζήτηση παραμένει εντός των ορίων της διαθέσιμης χωρητικότητας.

2.2.4 Αβεβαιότητα και ανθεκτικός σχεδιασμός δικτύου

Τα δίκτυα παράδοσης μέσω θυρίδων συχνά λειτουργούν σε ένα περιβάλλον που δεν είναι ποτέ πραγματικά σταθερό. Η διαθέσιμη χωρητικότητα μπορεί να πιεστεί από τη μια στιγμή στην άλλη, η ζήτηση μεταβάλλεται και οι χρόνοι εξυπηρέτησης αλλάζουν Η βιβλιογραφία προσεγγίζει αυτές τις προκλήσεις μέσα από τρεις βασικές στρατηγικές. Η πρώτη κατηγορία είναι η ανθεκτική βελτιστοποίηση (Robust optimization). Οι Wang et al.[23] αξιοποιούν πιο ρεαλιστικά μοντέλα τύπου Bertsimas–Sim τα οποία περιλαμβάνουν διάφορα σύνολα αβεβαιότητας, ώστε να διασφαλίζεται ότι ο σχεδιασμός παραμένει βιώσιμος ακόμη και όταν οι πραγματικές τιμές της ζήτησης ή της διαθεσιμότητας ξεφεύγουν από τα συνηθισμένα. Η προσέγγιση αυτή, όμως, δεν είναι χωρίς κόστος. Περισσότερη προστασία οδηγεί σε πιο συντηρητικές αποφάσεις — και αυτό συνήθως μεταφράζεται σε αυξημένες δαπάνες. Επίσης, υπάρχει η δυνατότητα να επεκταθεί σε διάφορες χρονικές πε-

ριόδους. Οι Rabe et al.[24] εξετάζουν τη δημιουργία ενός δικτύου που λαμβάνει υπόψη τη στοχαστικότητα και υποδεικνύουν ότι ο στατικός σχεδιασμός μπορεί να παραβλέψει τις δυναμικές διαδικασίες που εκτυλίσσονται στην πραγματικότητα. Οι Ottaviani et al.[25] προτείνουν μια υβριδική προσέγγιση: ένα μαθηματικό πρόβλημα ακέραιου προγραμματισμού (MILP) που συνδυάζει την άπληστη μέθοδο και την προσομοίωση Monte Carlo. Συγκεκριμένα, η προσέγγιση τους χωρίζει την πόλη σε γεωγραφικές ζώνες με διαφορετικά ποσοστά ζήτησης και χρησιμοποιεί εκθετική κατανομή για να υπολογίσει έναν τυχαίο αριθμό παραγγελιών ανά πελάτη. Έτσι, παρέχει έναν αποτελεσματικό τρόπο απεικόνισης της χωρικής ασυμμετρίας στη ζήτηση. Τρίτον, μέσω στοχαστικού προγραμματισμού που βασίζεται σε σενάρια (Scenario-based stohastic programming). Στις θυρίδες παράδοσης, η αβεβαιότητα σχετίζεται τόσο με την ποσότητα ζήτησης όσο και με την διαθέσιμη χωρητικότητα. Οι Mancini et al.[26] αναδεικνύουν αυτό το διπλό ρίσκο μέσω σεναρίων και μιας ιεραρχικής αντικειμενικής συνάρτησης, η οποία προτάσσει την μέγιστη εξυπηρέτηση πριν από τη ελάχιστη μέση απόσταση. Προτείνουν συνδυασμένες τεχνικές που συνδυάζουν ακριβείς και ευρετικές προσεγγίσεις, μαζί με τους δείκτες EVPI/VSS, προκειμένου να εκτιμήσουν την αξία της στοχαστικής μοντελοποίησης.

Η χρήση μιας γεννήτριας προβλημάτων σε αυτή τη μελέτη υπηρετεί ως πειραματική αξιολόγηση της αντοχής του μοντέλου. Αυτό βοηθά στη συστηματική αξιολόγηση αλγορίθμων καθώς και λύσεων σε περιβάλλοντα ελεγχόμενης τυχαιότητας και μεταβολών. Ταυτόχρονα η πολυπλοκότητα που συνδέεται στενά με την ολοκληρωμένη στοχαστική βελτιστοποίηση αποφεύγεται.

2.2.5 Πολυκριτηριακός σχεδιασμός δικτύων θυρίδων

Στον σχεδιασμό ενός δικτύου, σπάνια υπάρχει ένας μόνο στόχος που να οδηγεί σε μία απόλυτα ιδανική λύση. Συνήθως δημιουργούνται αντιφάσεις μεταξύ της κάλυψης, του κόστους, της προσβασιμότητας και της βέλτιστης αξιοποίησης των πόρων. Οι Che et al.[27] καθορίζουν ξεκάθαρα το δίλημμα που υπάρχει ανάμεσα στην κάλυψη, την επικάλυψη και τη μη ενεργή χωρητικότητα. Παρέχουν λύσεις Pareto, οι οποίες δρουν ως εναλλακτικές προτάσεις στη διαδικασία λήψης αποφάσεων. Αυτό αποκτά σημασία όταν χρειάζεται να δικαιολογηθεί ο σχεδιασμός, όχι μόνο από τεχνική προοπτική (ώστε να επιτευχθεί η καλύτερη τιμή), αλλά επίσης από πολιτική ή

επιχειρηματική άποψη (τα συμβιβαστικά μέτρα που επιλέχθηκαν και οι λόγοι που τα καθόρισαν). Οι Luo et al.[28] ενισχύουν την πολυκριτήρια προσέγγιση, δίνοντας έμφαση στην πρόσβαση. Δηλώνουν ότι η προσβασιμότητα σε σχέση με το κόστος δεν είναι απλώς ζήτημα βαρών. Αρκετά συχνά φαίνεται να λειτουργεί ως κρίσιμος παράγοντας, κυρίως όταν συσχετίζεται με τη γεωγραφική ισότητα ή με την αποδοχή από τους χρήστες. Ο πολυκριτηριακός σχεδιασμός δείχνει ότι δεν υπάρχει μία και μόνο «τέλεια» λύση αλλά πολλές αποδεκτές επιλογές.

2.3 Ευρετικές και Μεθευρετικές Μέθοδοι

2.3.1 Υπολογιστική δυσκολία και περιορισμοί των exact MILP

Οι βασικές μέθοδοι χωροθέτησης, όπως το p-median και η εγκατάσταση υποδομών με περιορισμούς στην χωρητικότητα, καθώς επίσης και η χωροθέτηση με περιορισμένη ικανότητα εξυπηρέτησης, αποτελούν παραλλαγές που επικεντρώνονται στην χωρητικότητα. Αυτές οι προσεγγίσεις ανήκουν στα συνδυαστικά προβλήματα. Η ενσωμάτωση ρεαλιστικών περιορισμών έχει σημαντική επίδραση στον χώρο των πιθανών λύσεων. Οι Hakimi[29] και Krarup & Pruzan[18] εξετάζουν το λόγο για τον οποίο ακόμα και φαινομενικά απλές περιπτώσεις γίνονται υπολογιστικά δύσκολες όταν προστίθενται χωρητικότητες, διάφορα επίπεδα δυναμικότητας ή επιλογές τύπου εγκατάστασης. Στις θυρίδες παράδοσης, τα πράγματα γίνονται πιο σύνθετα: αυτός ο χώρος δεν αποτελεί απλώς ένα σημείο, αλλά έχει χωρητικότητα, κανόνες λειτουργίας, περιορισμούς στα ωράρια και συγκεκριμένα πρότυπα ζήτησης. Ο αλγόριθμος διακλάδωσης και ορίων αποτελεί ένα ουσιαστικό εργαλείο για την αναζήτηση ακριβών λύσεων. Παρ' όλα αυτά, η αποδοτικότητά του εξαρτάται από το πόσο καλά είναι δυνατό να παρέχει αξιόπιστους περιορισμούς και ικανοποιητικές εκτιμήσεις εφικτών λύσεων. Οι Boyd & Mattingley[30] θίγουν ένα σημαντικό θέμα: χωρίς αποτελεσματικούς περιορισμούς και καλή διαχείριση, ο αλγόριθμος μπορεί να περιπλανιέται σε ατελείωτες αναζητήσεις. Εξετάζει απεριόριστους συνδυασμούς χωρίς να φτάσει ποτέ σε μια λύση. Για αυτόν τον λόγο, σε πολύπλοκες διαδικασίες παραλαβής που περιλαμβάνουν χωρητικότητα, χρονικά περιθώρια και επιλογή οδηγού, χρησιμοποιούμε ευρετικές μεθόδους. Δεν είναι σωστό να πούμε ότι το MILP έχει θεωρητικά ελαττώματα, όμως στην περίπτωση πραγματικών δεδομένων, απαι-

τούμε λύσεις που να είναι υψηλής ποιότητας και να παραδίδονται σε ένα προβλέψιμο χρονικό διάστημα. Αυτό δεν συμβαδίζει πάντα με τις υποσχέσεις των ακριβών επιλυτών (Rodríguez-Esparza et al.[12], Zhou et al.[21]). Οι μεθόδοι Branch and Bound, οι οποίες χρησιμοποιούν φράγματα και στρατηγικές διακλάδωσης (Boyd & Mattingley[30]), επιτρέπουν την επίτευξη ακριβών λύσεων σε ικανοποιητικούς χρόνους. Αυτή η μέθοδος αποτελεί μια μορφή αλγορίθμου βελτιστοποίησης που εξισορροπεί την ακρίβεια με την υπολογιστική αποδοτικότητα.

2.3.2 Μεθευρετικές για προβλήματα χωροθέτησης με χωρητικότητα

Η μελέτη των μεθευρετικών σχετικά με τα p-median και τις παραλλαγές τους που περιλαμβάνουν περιορισμούς χωρητικότητας είναι ένα πολύτιμο εργαλείο για τον σχεδιασμό συστημάτων θυρίδων παράδοσης. Συγκεκριμένα, πολλές από τις διατυπώσεις που αφορούν αυτές τις θυρίδες μπορούν στην ουσία να χαρακτηριστούν ως p-median/CFLP, προσθέτοντας επιπλέον περιορισμούς. Οι γενετικοί αλγόριθμοι διατηρούν τη σπουδαιότητα τους λόγω της ευέλικτης κωδικοποίησης των αποφάσεων (π.χ., άνοιγμα/ανάθεση ή επιλογή/κατανομή). Αυτό σαφώς και επιβεβαιώνεται στο κλασικό πρόβλημα p-median με χωρητικότητα (Capacitated p-Median Problem, CPMP) (Shariff et al.[31]) όσο και σε καταστάσεις όπου η ανάλυση συνυπάρχει με δύσκολους περιορισμούς (Rodríguez-Esparza et al.[12]). Ακόμη, οι μέθοδοι εντατικής τοπικής αναζήτησης, όπως για παράδειγμα η Variable Neighborhood Search (VNS) και η Tabu Search (TS), συχνά αποδεικνύονται πιο αποτελεσματικές όταν ο κύριος στόχος τους είναι η γρήγορη βελτίωση από μια καλή αρχική λύση. Το VNS των Fleszar & Hindi[32] αποδεικνύει πως η εναλλαγή γειτονιών επιτυγχάνει σταθερά υψηλές επιδόσεις για προβλήματα p-median με περιορισμούς στη χωρητικότητα. Παράλληλα, η υβριδική Tabu Search που προτείνουν οι Yaghini et al.[33] δείχνει πώς η χρήση μνήμης κινήσεων μειώνει σημαντικά τους κύκλους και βελτιώνει τη διαδικασία σύγκλισης στα μεγάλα προβλήματα. Επίσης, το Iterated Local Search (ILS) των Lourenço et al.[34] αποτελεί μια απλή αλλά πολύ αποτελεσματική μεθευρετική προσέγγιση. Στην πράξη, επαναλαμβάνει τοπική αναζήτηση πάνω σε ελαφρώς διαταραγμένες λύσεις, προκειμένου να ξεφεύγει από τα τοπικά βέλτιστα. Ακόμη, χαρακτηρίζεται από αρχικοποίηση, τοπική αναζήτηση, διαταραχή και κριτήριο αποδοχής. Έτσι, έχει αποδειχθεί ιδιαίτερα αποδοτική κυρίως σε δύσκολα

συνδυαστικά προβλήματα. Το Simulated Annealing (SA) αποτελεί μια αξιόπιστη μέθοδο αναφοράς για την εξασφάλιση ελεγχόμενης απόδρασης από τοπικά ελάχιστα. Όπως παρατηρούμε δηλαδή σε εφαρμογές p-median που αφορούν Σημεία Συλλογής και Παράδοσης (CDPs) (Tadić et al.[35]). Η σχετική βιβλιογραφία δεν επικεντρώνεται στην ιστορική της έκδοση. Πιο συγκεκριμένα, προσφέρει κανόνες σχεδίασης, όπως σύνολα κινήσεων, μηχανισμούς για την αντιμετώπιση περιορισμών καθώς και στρατηγικές για αρχικοποίηση. Όλα αυτά μεταφέρονται σχεδόν χωρίς αλλαγές σε δίκτυα παραδόσεως θυρίδων. Μια πρακτική κυρίως επιβεβαίωση προκύπτει από τη μελέτη περίπτωσης των ταχυδρομικών μεταφορών (Koloch[36]), στην οποία συγκρίνονται οι μέθοδοι Simulated Annealing, Tabu Search και Genetic Algorithm στο πλαίσιο του προβλήματος οχημάτων με διαδρομές (VRP) υπό περιορισμένο χρόνο.

2.3.3 Μεθευρετικές σε δίκτυα Θυρίδων Παράδοσης με περιορισμούς και ετερογένεια

Στις παραδόσεις των θυρίδων, οι μεθευρετικοί αλγόριθμοι δεν προκύπτουν μόνο λόγω του μεγέθους τους. Εμφανίζονται διότι το πρόβλημα αποκτά δομικά πολυκριτηριακό ή μη ομαλό χαρακτήρα όταν εισάγονται κανόνες συμπεριφοράς και λειτουργικοί περιορισμοί. Όταν η χωροθέτηση σχετίζεται με χρονικά περιθώρια παραλαβής και περιορισμούς στη χωρητικότητα, η χρήση του Γενετικού Αλγορίθμου (Rodríguez-Esparza et al.[12]) δεν είναι εναλλακτική. Συχνά αποτελεί την πιο πρακτική μέθοδο για την εξεύρεση λειτουργικών λύσεων σε ρεαλιστικά μεγέθη. Οι πολυκριτηριακές προσεγγίσεις σε προβλήματα αναγνωρίζουν ότι η βέλτιστη λύση είναι ένα σύνολο συμβιβασμών. Οι Che et al.[27] επικεντρώνονται σε ζητήματα κάλυψης, επικάλυψης και ανενεργής χωρητικότητας, παράγοντας έτσι λύσεις του Pareto. Στον τομέα του σχεδιασμού θυρίδων παράδοσης με έμφαση στη προσβασιμότητα, οι Luo et al.[28] χρησιμοποιούν έναν ενεργό μηχανισμό μάθησης (Active Learning Mechanism) μέσα σε ένα πλαίσιο εξελικτικής βελτίωσης βασισμένης στο Pareto. Σκοπός τους είναι να μειώσουν το κόστος των αξιολογήσεων, δεδομένου ότι κάθε αξιολόγηση απαιτεί υπολογιστική δαπάνη. Η ποικιλία και η συμβατότητα της δομής είναι σημαντικοί παράγοντες σε περιβάλλοντα που περιλαμβάνουν διαφορετικούς χρήστες ή τύπους, καθώς και ειδικούς κανόνες κόστους ή ικανοποίησης. Η επιλογή μιας κατάλληλης μεθόδου αναζήτησης εξαρτάται από την αντιστοίχιση με τη δομή του προβλήματος.

Το Scatter Search (SS) των Zhou et al.[21] αποτελεί χαρακτηριστικό παράδειγμα. Στα δίκτυα παραλαβής, η μεθευρετική διευκολύνει την ενσωμάτωση ρεαλιστικών περιορισμών (όπως χωρητικότητα και χρονικά παράθυρα) και τη δημιουργία λύσεων Pareto, οι οποίες εξυπηρετούν στρατηγικές αποφάσεις.

2.3.4 Δυναμική διαχείριση θυρίδων με Reinforcement Learning

Μια νεότερη κατεύθυνση φαίνεται να μετακινεί το επίκεντρο της από την επιλογή θέσεων εγκατάστασης στη λειτουργία αυτών των θυρίδων στην καθημερινότητα. Πιο συγκεκριμένα, οι αποφάσεις γίνονται διαδοχικά υπό αβεβαιότητα: αφίξεις δεμάτων, στοχαστικοί χρόνοι παραλαβής, καθώς και μεταβαλλόμενη διαθεσιμότητα θυρίδων. Σύμφωνα με το πλαίσιο των Sailer et al.[16]—αναλυτική προσέγγιση διαδοχικών αποφάσεων και ενισχυτική μάθηση (Reinforcement Learning)—φαίνεται ότι πολιτικές αποδοχής και ανάθεσης μπορούν να βελτιώσουν σημαντικά την απόδοση έναντι απλών κανόνων. Ειδικά όταν το σύστημα κινείται κοντά σε κορεσμό. Αυτή η κατεύθυνση λειτουργεί ως προέκταση του στρατηγικού σχεδιασμού. Στην πραγματικότητα, ο εκ των προτέρων σχεδιασμός (θέση, τύπος, ανάθεση) προσφέρει τη βάση για τη δομή του δικτύου. Επιπλέον, οι πολιτικές που αφορούν τη λειτουργία διασφαλίζουν την ποιότητα της παρεχόμενης υπηρεσίας και αποτρέπουν επαναδρομολογήσεις λόγω υπερφόρτωσης.

2.4 Προσβασιμότητα, Συμπεριφορά Χρηστών & Χωρική Ισότητα

2.4.1 Προσβασιμότητα ως σχεδιαστική απαίτηση

Η πρόσβαση στις θυρίδες παράδοσης επηρεάζεται όχι μόνο από την απλή ευθεία απόσταση, αλλά και από τη διαδρομή που επέλεξε κάποιος να ακολουθήσει. Οι διαθέσιμες επιλογές εξετάζονται στο ευρύτερο πλαίσιο: πόσο εύκολα μπορεί κάποιος να προσεγγίσει τον χώρο, ποιο είναι το επίπεδο ασφάλειας της περιοχής και πώς συνδέεται η θυρίδα με τις μετακινήσεις μέσα στην πόλη. Τα μοντέλα των "5Ds" δείχνουν ότι η απόσταση δεν είναι ο μόνος παράγοντας που καθορίζει την πραγματική εμπειρία της πρόσβασης. Δύο διαδρομές που φαίνονται παρόμοιες στο χάρτη μπορεί στην πραγματικότητα να έχουν πολύ διαφορετικό επίπεδο δυσκολίας. Αυτό επηρεάζει άμεσα την συχνότητα και την αποτελεσματικότητα της χρήσης τους

(Ma et al.[37]). Οι σχετικές αναλύσεις δείχνουν ότι τα πρότυπα εξυπηρέτησης δεν είναι σταθερά, αλλά ποικίλλουν ανάλογα με τις συνθήκες. Επηρεάζονται από το περιβάλλον καθώς και από τα χαρακτηριστικά του πληθυσμού. Η ίδια πολιτική για τον καθορισμό της μέγιστης απόστασης εφαρμόζεται διαφορετικά σε περιοχές με μεγάλη πληθυσμιακή πυκνότητα και σε εκείνες που είναι λιγότερο κατοικημένες (Campisi & Di Ruocco[4]). Στη βιβλιογραφία, συχνά υιοθετούνται αυστηρά όρια για να απλοποιηθεί η διαδικασία μοντελοποίησης. Ωστόσο, αυτή η μέθοδος παραβλέπει τις σταδιακές αλλαγές στην προθυμία των χρηστών. Στην ανάλυσή μας, η μέγιστη απόσταση για κάθε τρόπο μεταφοράς περιλαμβάνει διάφορους τρόπους πρόσβασης, χωρίς να λαμβάνονται υπόψη οι παράγοντες χρησιμότητας. Αυτή η επιλογή είναι σύμφωνη με την έννοια του κατωφλίου του TLM. Η πρόσβαση περιγράφεται ως ένας αυστηρός περιορισμός που καθορίζει το σύνολο των διαθέσιμων επιλογών.

2.4.2 Συμπεριφορά επιλογής και πιθανότητα χρήσης

Ακόμη και αν υπάρχει μια θυρίδας παραλαβής και είναι προσβάσιμη για τον πελάτη , δεν είναι σίγουρο ότι θα επιλεγεί. Δηλαδή στην πραγματικότητα μια τέτοια επιλογή εξαρτάται σίγουρα από την αντίληψη του χρήστη σχετικά με την άνεση και την αξιοπιστία της, καθώς και από την σχετική ασφάλεια της υποψήφιας περιοχής. Είναι πολύ σημαντικό να συνδυάζεται με τις καθημερινές μετακινήσεις του χρήστη. Οι παράγοντες αυτοί κυρίως αναλύονται μέσω δύο κύριων προσεγγίσεων. Η πρώτη έρευνα εστιάζει στην κατανόηση της χρήσης. Αναδεικνύει μη τεχνικούς παράγοντες που επηρεάζουν την επιλογή, όπως η αξιοπιστία στις λειτουργίες, η ευχρηστία καθώς και η κατάλληλη τοποθεσία. Παρ' όλα αυτά, η προσέγγιση αυτή περιέχει κάποιους περιορισμούς. Τα αποτελέσματα δεν είναι πάντα έτοιμα προς εφαρμογή σε διάφορες συνθήκες και συχνά επηρεάζονται από υποκειμενικές προσωπικές αξιολογήσεις, όπως τονίζουν οι Lemke et al.[8].

Η δεύτερη μέθοδος εδράζεται στη διαδικασία μοντελοποίησης επιλογών (Choice Modeling), όπου τα μοντέλα MNL και TLM αποτελούν χαρακτηριστικά παραδείγματα. Στις περιπτώσεις αυτές, οι αποστάσεις και οι υπηρεσίες μετατρέπονται σε πιθανότητες προτίμησης. Είναι ουσιώδες να κάνουμε τη διάκριση μεταξύ αυτών των δύο προσεγγίσεων. Στο MNL, ο χρήστης διαθέτει πολλές διαφορετικές επιλογές. Αντίθετα, στα μοντέλα κατωφλίου ή περιορισμένων επιλογών (TLM), η προ-

σβασιμότητα λειτουργεί περισσότερο ως φίλτρο: καθορίζει ποιες εναλλακτικές θα εξεταστούν εξ' αρχής, όπως αναφέρουν οι Lin et al.[10].

Στο πλαίσιο της παρούσας μελέτης, η εφαρμογή ενός ανώτατου ορίου απόστασης για κάθε μέσο μεταφοράς εναρμονίζεται ακριβώς με αυτή την αρχή του κατωφλίου. Δεν χρειάζεται μια ολοκληρωμένη εκτίμηση της χρησιμότητας· αντίθετα, υιοθετείται μια ρεαλιστική προσέγγιση που αποτυπώνει τον τρόπο λειτουργίας των πραγματικών χρηστών.

2.4.3 Ετερογένεια χρηστών και παραλαβή "on the way"

Η συμπεριφορά των ανθρώπων δεν είναι πάντοτε ομοιογενής. Μία απόσταση μπορεί να σημαίνει κάτι διαφορετικό για κάποιον που εξερευνά μια νέα περιοχή σε σχέση με άλλον που διασχίζει καθημερινά έναν συγκεκριμένο κόμβο. Σύμφωνα με διάφορα μοντέλα, η ιδανική επιλογή δεν σχετίζεται πάντα με την αρχή του "όσο πιο κοντά γίνεται". Η ενσωμάτωση της παραλαβής στην καθημερινή διαδρομή του χρήστη φαίνεται να μειώνει την επιπλέον απόσταση καθώς και το χρόνο. Δηλαδή, απαιτείται μόνο μια μικρή εκτροπή αντί για ξεχωριστή μετακίνηση (Zhou et al.[21]). Σε μεγάλες αστικές περιοχές, η ευχέρεια πρόσβασης στις θυρίδες παράδοσης σχετίζεται κυρίως με την καθημερινή ρουτίνα. Έαν οι θυρίδες βρίσκονται στην πορεία σου και ανταγωνίζονται τον τρόπο μετακίνησής σου, τότε η διαδικασία παραλαβής γίνεται αρκετά πιο εύκολη (Zhou et al.[21]). Η εξέταση των δικτυακών επεκτάσεων ταυτόχρονα , δείχνει ότι η ύπαρξη τους κοντά σε κατοικίες ή διελεύσεις μπορεί να μειώσει τις αποστάσεις, προωθώντας έτσι το περπάτημα ή τη χρήση του ποδηλάτου. Από την άλλη πλευρά, μια υπερβολική συσσώρευση σημείων παράδοσης μπορεί να ανακόψει τις ροές (Pinchasik et al.[5]). Η πιο καινούργια προσέγγιση που εστιάζει σε δεδομένα (data-driven), στρέφει την προσοχή από τις στατικές τοποθεσίες ζήτησης προς τις πραγματικές ροές και διαδρομές μετακίνησης. Οι Li & Li[38] αναλύουν δεδομένα από ταξί για να μελετήσουν τη συνδυασμένη μεταφορά, επικεντρώνοντας την προσοχή τους στην υπηρεσία ride-pooling κατά την τελευταία φάση της διαδρομής. Τονίζουν ότι η αξιολόγηση πιθανών τοποθεσιών μεταβάλλεται σημαντικά όταν η πρόσβαση θεωρείται ως μια παράκαμψη στη διαδρομή, αντί να εξαρτάται απλώς από την απόσταση.

2.4.4 Χωρική ανισότητα και ασυμφωνία προσφοράς–ζήτησης

Η προσβασιμότητα σχετίζεται άμεσα με την ισότητα. Ένα δίκτυο που μειώνει τις μέσες αποστάσεις μπορεί να έχει ως αποτέλεσμα ορισμένες περιοχές να παραμένουν μόνιμα υποβαθμισμένες ή και ευάλωτες σε υπερφόρτωση. Η εξέταση της ασυμφωνίας μεταξύ προσφοράς και ζήτησης (supply–demand mismatch) σε τοπικό επίπεδο δείχνει ότι η ανισορροπία ανάμεσα στην ικανότητα και τη ζήτηση προκαλεί άνισους όρους παροχής υπηρεσιών. Αυτό δηλαδή μπορεί να προκαλέσει έναν φαύλο κύκλο, όπου η περιορισμένη χρήση οδηγεί σε λιγότερες επενδύσεις στις πιο προβληματικές περιοχές. Στην περίπτωση της Tianjin, το 20% των κατοίκων έχει πρόσβαση στο 80% των έξυπνων θυρίδων παραλαβής (Smart Parcel Lockers, SPLs), ενώ πολλές περιοχές φαίνεται να διαθέτουν υπερβολικό αριθμό τέτοιων θυρίδων με περιορισμένη χρήση (Ding et al.[13]). Οι μελέτες που εξετάζουν τη χωρική ισότητα επισημαίνουν ότι η εστίαση μόνο στην αποδοτική τοποθέτηση μπορεί να εντείνει τον αποκλεισμό. Αυτό συμβαίνει ειδικά όταν η πρόσβαση για ορισμένες ομάδες προϋποθέτει τη χρήση ιδιωτικού αυτοκινήτου ή περιλαμβάνει μακρινές και επικίνδυνες διαδρομές για τους πεζούς (Campisi & Di Ruocco[4]) Η ισότητα δεν αφορά μόνο το ”πού”, αλλά επεκτείνεται και στο ”πώς” λειτουργεί το δίκτυο. Ακόμα κι αν οι θυρίδες παραλαβής είναι σωστά τοποθετημένες, μπορεί να υπάρξουν άνισες εμπειρίες αν η χωρητικότητα και ο σχεδιασμός (διάταξη/αριθμός εγκαταστάσεων) δημιουργούν συμφόρηση σε συγκεκριμένες περιοχές. Ένα συγκεκριμένο παράδειγμα είναι η μέθοδος βελτιστοποίησης διάταξης (layout optimization) που προτείνουν οι Zhang & Wu[39]. Σε αυτή τη μέθοδο, ο καθορισμός του αριθμού ή της διάταξης σε επίπεδο κοινότητας συνδυάζει το κόστος με την ικανοποίηση αξιολογώντας τις αποστάσεις. Αυτού του είδους οι εργασίες συχνά βασίζονται σε απλουστεύσεις, ωστόσο αποκαλύπτουν μια ζωτική αλήθεια: η ισότητα σε τοπικό επίπεδο επηρεάζεται από επιλογές που μπορεί να φαίνονται μικρές, όπως είναι ο αριθμός και η κατανομή των εγκαταστάσεων. Αυτές οι αποφάσεις ουσιαστικά προσδιορίζουν ποιοι άνθρωποι θα έχουν τη δυνατότητα να εξυπηρετούνται με ασφάλεια.

2.5 Βιωσιμότητα, Εκπομπές & Αστική Κινητικότητα

2.5.1 Περιβαλλοντικό ισοζύγιο και όρια συστήματος

Μια καθοριστική πτυχή στη βιβλιογραφία είναι ο προσδιορισμός του τι θεωρείται εκπομπή από το σύστημα. Δηλαδή αν αναφερόμαστε μόνο στην πλευρά του παροχέα, όπως τα οχήματα και τα δρομολόγια, ή εάν περιλαμβάνουμε το συνολικό άθροισμα που αφορά και τον πάροχο αλλά και τις μετακινήσεις των καταναλωτών. Οι συγκρίσεις ανάμεσα σε άμεσες παραδόσεις (direct deliveries) και Αυτόματες Θυρίδες Παράδοσης (APL/lockers) δείχνουν ότι η συσσώρευση των παραδόσεων βοηθά στη μείωση των χιλιομέτρων που διανύει ο στόλος. Ωστόσο, το τελικό αποτέλεσμα εξαρτάται σε μεγάλο βαθμό από τον τρόπο που θα παραλάβει ο παραλήπτης, είτε με τα πόδια, είτε με ποδήλατο, είτε οδηγώντας αυτοκίνητο. Επιπλέον, είναι σημαντικό να εξετάσουμε αν η διαδικασία παράδοσης ενσωματώνεται σε ήδη υπάρχουσες διαδρομές, ένα θέμα που συχνά παραβλέπεται (Gutenschwager et al.[40]). Σύμφωνα με τα εμπειρικά δεδομένα, η δυνατότητα μετατόπισης προς βιώσιμες μορφές μετακίνησης ανέρχεται στο 12%, και η μείωση των συμμετρικών διαδρομών (από το σπίτι στην παραλαβή και πίσω) μπορεί να φτάσει το 27% (Hofer et al.[41]). Η εκπομπή ρύπων ανά μέσο μεταφοράς παίζει σημαντικό ρόλο: ηλεκτρικά οχήματα (0 g/km), δημόσιες συγκοινωνίες (42.5 g/km) και οχήματα εσωτερικής καύσης (162 g/km) (Pinchasik et al.[5]). Αυτή η τετραπλή διαφορά μεταξύ ιδιωτικών αυτοκινήτων και Μέσων Μαζικής Μεταφοράς καθιστά απαραίτητη την ένταξη αποστάσεων και εκπομπών που εξαρτώνται από το μέσο μετακίνησης στο μοντέλο βελτιστοποίησης. Το πλαίσιο του χωροθέτησης και της δρομολόγησης (location-routing) που αναλύει την περιβαλλοντική επίδραση των οικολογικών συμπεριφορών των καταναλωτών, αποδεικνύει ότι η μέγιστη αποδεκτή απόσταση δεν είναι αρκετή. Η περιβαλλοντική απόδοση εξαρτάται από την έκταση των αποστάσεων που καλύπτονται με φιλικά προς το περιβάλλον μέσα μεταφοράς. Το σύστημα γίνεται βιώσιμο όχι μόνο λόγω της αλλαγής στον τόπο παράδοσης, αλλά και εξαιτίας της κινητικότητας που αυτή η αλλαγή προκαλεί. Στην πράξη, όταν η μέγιστη πράσινη απόσταση (dgreen) ξεπερνά τα 1.5 χιλιόμετρα, οι συνολικές εκπομπές μειώνονται μέχρι και κατά 67%. Αντίθετα, σε περιπτώσεις όπου οι καταναλωτές επιλέγουν αυτοκίνητο για κοντινές διαδρομές, η μείωση των εκπομπών είναι σχεδόν αμελητέα (Bonomi et al.[42]). Για αυτή την

εργασία, η μεγαλύτερη απόσταση ανά τρόπο μετακίνησης και το συνολικό κόστος λειτουργούν ως μια χρήσιμη μέθοδος ώστε να ενσωματωθεί αυτός ο περιορισμός του συστήματος στη διαδικασία βελτιστοποίησης.

2.5.2 Αποτελέσματα εφαρμογών σε πόλεις και σενάρια

Οι περιπτώσεις μελέτης αποδεικνύουν ότι οι θυρίδες παράδοσης συμβάλλουν στη μείωση των χιλιομέτρων που διανύουν τα οχήματα, καθώς και του χρόνου οδήγησης. Αυτό συμβαίνει επειδή διευκολύνουν τη συγκέντρωση των στάσεων και περιορίζουν τις επαναλήψεις που προκύπτουν από αποτυχημένες παραδόσεις (van Duin et al.[1]). Στην περίπτωση του Άμστερνταμ, ο αριθμός των στάσεων μειώθηκε από 1475 σε 477. Η τιμή παράδοσης κάθε πακέτου μειώθηκε κατά 16%, ενώ οι εκπομπές CO_2 έχουν περιοριστεί κατά δύο τρίτα, χάρη στη χρήση της συγκεντρωτικής μεθόδου που εφαρμόζεται στις θυρίδες παράδοσης. Ωστόσο, υπάρχει ένα όριο σε αυτό. Η επέκταση ενός δικτύου δεν είναι πάντα χρειαζούμενη ή αποδεδειγμένα ωφέλιμη. Σε ορισμένες μόνο περιπτώσεις, η υπερβολική πυκνότητα μπορεί να επηρεάσει τις ροές και να απειλήσει τα πλεονεκτήματα της οικονομικής κλίμακας, καθώς επίσης να έχει και επιπτώσεις στα περιβαλλοντικά οφέλη (Pinchasik et al.[5]). Μία σημαντική διάσταση είναι η έννοια του κοινωνικού κόστους, η οποία περιλαμβάνει παράγοντες όπως η κυκλοφοριακή συμφόρηση, ο θόρυβος καθώς και την ασφάλεια στους δρόμους. Αυτή η μέθοδος σαφώς, επεκτείνει την έννοια της βιωσιμότητας, προχωρώντας πέρα από μια στενή λογιστική που επικεντρώνεται μόνο στο CO_2, και αναλαμβάνει μια πιο συνολική πολιτική και αστική αξιολόγηση. Η τοποθέτηση και ταυτόχρονα η προσβασιμότητα δεν σχετίζονται αποκλειστικά με τις εκπομπές ρύπων. Αφορά επίσης ποιες μετακινήσεις είναι καλύτερο να αποφεύγονται ή να αναβάλλονται για περιοχές και χρονικές στιγμές που επιβαρύνουν ιδιαίτερα το αστικό δίκτυο (Pinchasik et al.[5]). Στο Όσλο, η αλλαγή από την παράδοση στο σπίτι στις θυρίδες παραδόσεων έχει ως αποτέλεσμα τη μείωση των εκπομπών του διανομικού δικτύου κατά 30-33%. Ταυτόχρονα, οι συνολικές εκπομπές πέφτουν κατά 13-32%, ακόμη και υπό τη συντηρητική υπόθεση χρήσης αυτοκινήτων.

2.5.3 Βιωσιμότητα ως πολυκριτηριακός στόχος

Στη σύγχρονη βιβλιογραφία, η βιωσιμότητα εκλαμβάνεται ως ένας συνδυασμός πρακτικών και περιβαλλοντικών παραμέτρων. Δεν αντιμετωπίζεται μόνο ως μια απλή μετρήσιμη αξία. Αυτό γίνεται αντιληπτό σε πολυκριτήρια περιβάλλοντα, όπου πέρα από τις εκπομπές ή τις αποστάσεις, είναι καθοριστικής σημασίας η ποιότητα της χωρικής κάλυψης, η επικάλυψη καθώς και η υποχρησιμοποίηση της χωρητικότητας. Αυτοί οι παράγοντες συνδέονται άμεσα με την κακή αξιοποίηση υποδομών και, ως αποτέλεσμα, το περιβαλλοντικό αποτύπωμα που δημιουργείται από την επένδυση και τη λειτουργία (Che et al.[27]). Έρευνες που αναλύουν τη λειτουργικότητα αυτών των δικτύων θυρίδων παράδοσης από πρακτική και επιχειρησιακή προοπτική δείχνουν ότι, αν και ένα δίκτυο μπορεί να φαίνεται βιώσιμο στη θεωρία, ίσως δεν αποδίδει ικανοποιητικά όσον αφορά την ποιότητα υπηρεσίας και την αξιοπιστία. Αυτό οδηγεί σε ανακατευθύνσεις, αποτυχίες και έμμεσες επιπτώσεις στο περιβάλλον (Sawik et al.[7]). Σε πιο ευρείες ποσοτικές προσεγγίσεις, το κόστος και οι περιβαλλοντικές επιδράσεις θεωρούνται αλληλένδετα στοιχεία. Η καλύτερη επιλογή για τη χωροθέτηση δεν είναι απλώς αυτή που μειώνει τις εκπομπές CO_2, αλλά ταυτόχρονα και εκείνη που περιορίζει το συνολικό κόστος σε σχέση με τις εκπομπές. Αυτό πρέπει να γίνεται λαμβάνοντας υπόψη ρεαλιστικούς παράγοντες ανάθεσης και τη συμπεριφορά της ζήτησης (Peppel & Spinler[9]). Για το γενικευμένο κόστος (Generalised Cost) ενός ταξιδιού δίνονται οι τιμές 0.36 έως 0.77 €/χλμ για μια αστική μετακίνηση με ένα ιδιωτικό αυτοκίνητο. Στην πραγματικότητα, οι συγκεκριμένες τιμές επηρεάζονται άμεσα από την ταχύτητα καθώς και από την απόσταση της διαδρομής. Επίσης, το κόστος ανά επιβάτη μπορεί να είναι πολύ χαμηλότερο στις δημόσιες συγκοινωνίες. Πιο συγκεκριμένα, είναι σχεδόν 0.05 €/χλμ και ισχύει για όσους από αυτούς διαθέτουν κάρτα απεριορίστων διαδρομών. Τέλος, η επιβολή χρεώσεων λόγω κυκλοφοριακής συμφόρησης (π.χ η χρέωση των £2 ανά είσοδο στο Cardiff) έχει στόχο να ενσωματώσει εξωτερικές παραμέτρους, όπως οι εκπομπές των ρύπων, η κυκλοφοριακή συμφόρηση και ο θόρυβος (Santos & Caranzo[43], OECD/ITF[44]). Οι εκπομπές CO_2 διαφέρουν σημαντικά ανάλογα με τον τύπο του μέσου μεταφοράς. Για παράδειγμα, τα ηλεκτρικά οχήματα (BEVs) δεν εκπέμπουν ρύπους (0 g/km), ενώ άλλες επιλογές ενδέχεται να έχουν διαφορετικούς δείκτες. Οι δημόσιες συγκοινωνίες μπορεί να φτάσουν τα 42.5 g/km, ενώ ορισμένα αυτοκίνητα

με εσωτερική καύση μπορεί να εκπέμπουν έως και 162 g/km (Pinchasik et al.[5]). Η εκτίμηση τους ποικίλλει από περίπου 100 €/tCO_2 (συντηρητική εκδοχή) έως και 1.056 €/tCO_2 (υψηλή αξία SCC σύμφωνα με τους Bilal & Känzig[45]). Για την παρούσα εργασία, η επιλογή μιας υψηλής τιμής CO_2 (1000 €/tCO_2) υποδεικνύει μια στοχευμένη προσέγγιση που προάγει τη βιωσιμότητα. Ενσωματώνει από την αρχή τον περιβαλλοντικό παράγοντα στη διαδικασία βελτιστοποίησης, προσφέροντας λύσεις που είναι "βιώσιμες κατά το σχεδιασμό". Παρ' όλα αυτά, η βιωσιμότητα δεν βασίζεται μόνο στη στρατηγική του σχεδιασμού. Η πρόληψη αναδρομολογήσεων και η αξιόπιστη λειτουργία είναι επίσης πολύ σημαντικές.

2.5.4 Έμμεση βιωσιμότητα μέσω λειτουργικής αξιοπιστίας

Ένας τομέας που συχνά παραβλέπεται όσον αφορά τα περιβαλλοντικά οφέλη είναι η αποφυγή δευτερευόντων μετακινήσεων, οι οποίες προκύπτουν από λειτουργική αστάθεια, όπως η υπερχείλιση, οι επαναλήψεις προσπαθειών, οι ανακατευθύνσεις καθώς και από ξαφνικές αναθέσεις. Μελέτες που λαμβάνουν υπόψη περιορισμούς χωρητικότητας και χρονοδιαγράμματα δείχνουν ότι η ιδανική επιλογή για την τοποθέτηση και τη διανομή πόρων αλλάζει όταν εξετάζονται οι πιθανότητες συμφόρησης και η αξιοπιστία της υπηρεσίας. Αυτό ισχύει ακόμα κι αν το δίκτυο φαίνεται ικανοποιητικό από γεωγραφικής άποψης (Rodríguez-Esparza et al.[12]). Από τη θέση της λειτουργικής πολιτικής, οι δυνατότητες που προσφέρει η δυναμική διαχείριση ζήτησης μπορούν και παρέχουν μια εναλλακτική οπτική. Μπορείς να βελτιώνεις την περιβαλλοντική απόδοση χωρίς καν να χρειαστεί να κάνεις μεγάλες αλλαγές στη χωρική διάταξη του δικτύου, απλά μειώνοντας τις αποτυχίες και τις αναδρομολογήσεις. Αυτό μπορεί να γίνει μέσω πιο αποτελεσματικών κανόνων για την αποδοχή και την κατανομή της χωρητικότητας (Sailer et al.[16]).

2.6 Προσομοίωση, Δυναμικά Μοντέλα και Στοχαστική Συμπεριφορά Συστήματος

2.6.1 Πολυπεριοδικός σχεδιασμός με προσομοίωση

Στη βιβλιογραφία, η σχεδίαση ενός δικτύου θυρίδων παράδοσης δεν είναι στατική. Η ζήτηση αλλάζει με την πάροδο του χρόνου και η αξιοπιστία εξαρτάται διαρ-

κώς από το πού είναι τοποθετημένες οι θυρίδες καθώς και από τον τρόπο λειτουργίας τους. Τα πολυπεριοδικά (multi-period) σχήματα συχνά περιλαμβάνουν διαδικασίες για τη χωροθέτηση και τη διαστασιολόγηση (πχ. CFLP/ILP) με προσομοιώσεις ή σενάρια Monte Carlo. Δηλαδή, αντί να επικεντρώνονται αποκλειστικά σε αυτά που είναι "καλά" σήμερα, αναλύουν επίσης ποια από τα στοιχεία διατηρούν την αξία τους κατά τη διάρκεια εξελίξεων και αβεβαιοτήτων (Rabe et al.[24],Ottaviani et al.[25]). Μελέτες που βασίζονται σε πραγματικά δεδομένα ή πιο αντιπροσωπευτικές συνθήκες υποδεικνύουν συνεχώς ότι η διάσταση της επιχείρησης μπορεί να επηρεάσει την εικόνα. Οι καθυστερήσεις, τα προβλήματα στην εξυπηρέτηση καθώς και οι αλλαγές στη διαχείριση μπορούν να μετατρέψουν μια λύση που φαίνεται τέλεια σε ένα στατικό μοντέλο. Συγκεκριμένα, σε κάτι μέτριο κατά την εφαρμογή της στον πραγματικό κόσμο. Η αξιολόγηση μέσω λειτουργικών δοκιμών, είτε με τη βοήθεια σεναρίων είτε μέσω προσομοιώσεων, αποτελεί καθοριστικό βήμα ανάμεσα στον στρατηγικό σχεδιασμό και την πραγματική υλοποίηση υπηρεσιών (Sawik et al.[7],Raviv & Gansterer[20]).

2.6.2 Στοχαστική και ανθεκτική μοντελοποίηση ζήτησης και χωρητικότητας

Στα δίκτυα των θυρίδων παράδοσης, η αβεβαιότητα επηρεάζει τη ζήτηση καθώς και τη διαθέσιμη χωρητικότητα. Αυτές οι θυρίδες μπορεί να παραμείνουν κλειστές για περισσότερες ώρες από ό,τι είχε υπολογιστεί συνήθως, με αποτέλεσμα την υπερχείλιση και την ανάγκη για αναδρομολογήσεις. Η βιβλιογραφία αντιμετωπίζει αυτό το θέμα μέσω δύο κύριων κατευθύνσεων, οι οποίες συνήθως λειτουργούν συνδετικά η μία στην άλλη. Η πρώτη έννοια είναι η Ανθεκτική Βελτιστοποίηση (Robust Optimization), που εισάγει έναν ελεγχόμενο παράγοντα "ασφάλειας" προκειμένου να διασφαλίσει ότι η λύση θα παραμείνει αποτελεσματική, ακόμα και αν οι απαιτήσεις αποκλίνουν από τις αρχικές εκτιμήσεις. Μειώνει την χαμένη ζήτηση υπό τυχαίες συνθήκες (Wang et al.[23]). Η δεύτερη προσέγγιση είναι η Στοχαστική Προγραμματιστική Μέθοδος (Scenario-based Stochastic Programming) που χρησιμοποιεί σενάρια και η αβεβαιότητα ενσωματώνεται μέσω αυτών. Ο σκοπός της προσέγγισης είναι να διατηρείται μια μέση επίδοση και να αποφεύγεται η συνεχείς κατάρρευση σε προκλητικές συνθήκες (Mancini et al.[26]). Πολλές από τις αναλυτικές προσεγγίσεις δεν αξιολογούν όλους τους στόχους με ίσο τρόπο. Συνήθως, επικεντρώνονται στο

"να καλυφθούν όσο το δυνατόν περισσότερες ανάγκες" και στη συνέχεια εξετάζουν τις αποστάσεις. Ο λόγος είναι απλός: λειτουργικά, το γεγονός ότι δεν μπορείς να καλύψεις τη ζήτηση λόγω περιορισμένης χωρητικότητας σε συνθήκες αβεβαιότητας συνιστά πιο σοβαρό πρόβλημα από μια μικρή αύξηση της μέσης απόστασης (Mancini et al.[26],Wang et al.[23]). Όταν οι περιπτώσεις αυξάνονται, η ακριβής λύση γίνεται υπολογιστικά πιο απαιτητική. Σε αυτές τις περιπτώσεις χρησιμοποιούμε συχνά μαθευρετικές ή υβριδικές προσεγγίσεις για να κρατήσουμε τον χρόνο υπολογισμού σε διαχειρίσιμα επίπεδα (Mancini et al.[26], Ottaviani et al.[25]).

2.6.3 Μικροδυναμική λειτουργία θυρίδων και στενά σημεία ροών

Ακόμα κι αν ένας στρατηγικός σχεδιασμός φαίνεται ελκυστικός σε θεωρητικό επίπεδο, μικρές αλλά συνεχείς μεταβολές - όπως οι χρόνοι παράδοσης των πακέτων, οι στιγμές άφιξης ή οι ώρες αιχμής - μπορούν γρήγορα να προκαλέσουν συμφόρηση και υπερφόρτωση. Αυτές οι καταστάσεις συχνά παραμένουν απαρατήρητες όταν εξετάζουμε ένα στατικό μοντέλο. Μεθόδους που βασίζονται σε πράκτορες (Agent-based) ή στη Monte Carlo προσομοίωση δείχνουν ότι η φέρουσα ικανότητα από μόνη της δεν είναι αρκετή για να αξιολογήσει την αξιοπιστία ενός συστήματος. Στην πραγματικότητα, η κατανομή των χρόνων παραλαβής καθώς και τα χαρακτηριστικά της έντονης και "εκρηκτικής" ζήτησης ασκούν σίγουρα σημαντική επιρροή (Rosca et al.[15], Raviv & Gansterer[20]). Η Προσομοίωση Διακριτών Γεγονότων (Discrete-Event Simulation) δείχνει επίσης ότι ορισμένες από τις πολιτικές λειτουργίας, όπως οι κανόνες εξυπηρέτησης, η διαχείριση των καθυστερημένων παραλαβών καθώς και η επανακατανομή πόρων, μπορούν να επιδράσουν σημαντικά στην πιθανότητα κορεσμού. Σε τέτοιες περιπτώσεις, ίσως να χρειαστεί να γίνουν επιπλέον μεταφορές ή αλλαγές στα δρομολόγια. Αυτές οι προσαρμογές συχνά συνεπάγονται ένα πρόσθετο κόστος και μπορεί να προκαλέσουν και ζητήματα βιωσιμότητας (Rosca et al.[46], Sawik et al.[7]).

Στην πραγματικότητα του σχεδιασμού δικτύων, το κύριο σημείο είναι ότι μια αποδοτική στρατηγική δεν χρειάζεται να συνεπάγεται και πλήρη προσομοίωση πολλών παραμέτρων, όπως συμβαίνει με τα ABM, ούτε και ενδελεχή ανάλυση συγκεκριμένων γεγονότων (DES). Αυτό που μετράει κυρίως είναι να αναγνωριστούν οι τομείς όπου η ροή διακόπτεται (bottlenecks), γιατί ακριβώς εκεί είναι που προκύ-

πτουν τα προβλήματα και οι καθυστερήσεις. Σύμφωνα με αυτό, είναι κρίσιμο να δημιουργηθεί ένα πειραματικό περιβάλλον που θα συμπεριλαμβάνει μια ποικιλία σεναρίων και παραμετρικών αλλαγών. Με αυτόν τον τρόπο, αποτυπώνεται πιο ρεαλιστικά η λειτουργική αβεβαιότητα και μπορεί να εξεταστεί η ανθεκτικότητα του συστήματος υπό συνθήκες πίεσης.

2.6.4 Δυναμική διαχείριση ζήτησης και αποφάσεις σε πραγματικό χρόνο

Μία πιο πρόσφατη προσέγγιση εξετάζει τις θυρίδες παράδοσης ως ένα σύστημα που λειτουργεί σε πραγματικό χρόνο, τα αιτήματα προκύπτουν διαδοχικά κατά τη διάρκεια της ημέρας και οι θυρίδες μπορεί να εκκενώνονται σε απρόβλεπτες χρονικές στιγμές. Η απόφαση για αποδοχή καθώς και ανάθεση θα πρέπει να λαμβάνεται σε πραγματικό χρόνο, στηριζόμενη στην παρούσα κατάσταση και τις δοσμένες εκτιμήσεις σχετικά με τυχόν μελλοντικά αιτήματα. Έρευνες που εφαρμόζουν αναλυτική προσέγγιση διαδοχικών αποφάσεων (Sequential Decision Analytics) και ενισχυτική μάθηση (Reinforcement Learning) αποκαλύπτουν ότι ένα δίκτυο με τις ίδιες θέσεις και χωρητικότητα μπορεί να επιδεικνύει εντελώς διαφορετικά επίπεδα απόδοσης, απλά λόγω των διαφορών στην πολιτική παραχώρησης που χρησιμοποιείται. Για παράδειγμα, μια πολιτική θα μπορούσε να δεσμεύει συγκεκριμένες θυρίδες για πιθανά μελλοντικά αιτήματα από πιο κοντινές περιοχές, αντί να ανατίθενται αμέσως—ιδιαίτερα όταν παρατηρούνται αυξημένες ανάγκες και η διαθεσιμότητα μειώνεται (Sailer et al.[16], Rosca et al.[15]). Η παρούσα διπλωματική εργασία επικεντρώνεται στον σχεδιασμό που περιλαμβάνει θέσεις, τύπους και αναθέσεις. Η λήψη αποφάσεων σε πραγματικό χρόνο καταγράφεται ως μελλοντική κατεύθυνση που θα μπορούσε να ενισχύσει την απόδοση κατά τις ώρες αιχμής (Sailer et al.[16], Raviv & Gansterer[20]).

2.7 Σύνθεση Βιβλιογραφίας & Τοποθέτηση της Διπλωματικής

2.7.1 Θυρίδες παράδοσης και ετερογένεια

Η σύγχρονη βιβλιογραφία επισημαίνει ότι τα δίκτυα θυρίδων διακρίνονται για την ετερογένεια τους. Συγκεκριμένα διαφέρουν τόσο στην χωρητικότητα όσο και στη συμπεριφορά των χρηστών. Τα μοντέλα χωρητικότητας πολλαπλών επιπέδων,

όπως το Multi-Capacitated Location Problem (MCLP), προτείνουν ότι οι εγκαταστάσεις θα πρέπει να έχουν τη δυνατότητα να λειτουργούν σε διάφορες κλίμακες και με διαφορετικά κόστη. Με αυτόν τον τρόπο, θα μπορούν να ανταποκριθούν στις μεταβαλλόμενες ανάγκες της αγοράς (El Amrani et al.[22]). Επιπρόσθετα, η ποικιλία στους όγκους των θυρίδων και η συμμετοχή διαφορετικών χρηστών, οι οποίοι παρουσιάζουν ξεχωριστές συμπεριφορές και προτιμήσεις, επηρεάζουν άμεσα την αποδοτικότητα και την αποδοχή του συστήματος (Zhou et al.[21]). Επιπλέον, οι εμπειρικές μελέτες δείχνουν ότι οι πραγματικές διαστάσεις και η ποικιλία στη λειτουργία των θυρίδων παίζουν καθοριστικό ρόλο στο σχεδιασμό αποτελεσματικών δικτύων (van Duin et al.[1]). Για την παρούσα εργασία χρησιμοποιούνται τρεις διαφορετικοί τύποι θυρίδων. Πιο συγκεκριμένα έχουμε μικρό, μεσαίο και μεγάλο με βάση τις διαστάσεις που παρουσιάζονται στη μελέτη των van Duin et al[1]. Στην πράξη, με βάση αυτό το άρθρο οι διαστάσεις των θυρίδων (πλάτος 0.410 m, βάθος 0.525 m και ύψος μεταβαλλόμενο ανά τον τύπο) οδηγούν σε υπολογισμό όγκου ίσο με 0.0521 m^3 για τις μικρές, 0.1082 m^3 για τις μεσαίες καθώς και 0.1633 m^3 για τις μεγάλες θυρίδες. Οι συγκεκριμένες τιμές ενσωματώνονται στο μοντέλο ως απαιτήσεις όγκου ανά τύπο θυρίδας. Με αυτόν τον τρόπο, διασφαλίζουμε ρεαλιστική αντιστοίχιση με τα δίκτυα θυρίδων. Η ανάλυση του κόστους των θυρίδων εξετάζεται σε ένα διάστημα 15 ετών, συνδυάζοντας την αρχική επένδυση, τα έξοδα υποδομής (υπολογιστής για κάθε ομάδα θυρίδων) καθώς και τα ετήσια λειτουργικά έξοδα. Αυτή η προσέγγιση στηρίζεται στην μέθοδο της Καθαρής Παρούσας Αξίας (NPV), σύμφωνα με τους Schnieder et al.[19]. Το κόστος εγκατάστασης και λειτουργίας των θυρίδων υπολογίζεται μέσω ενός σταθερού επιτοκίου 1% για μια μακροχρόνια περίοδο. Στην εργασία μας, εξετάζουμε το συνολικό κόστος σε παρούσες αξίες για κάθε είδους θυρίδα. Με αυτόν τον τρόπο, αποφασίζεται πού και ποιο είδος θα επιλεγεί, βασιζόμενοι σε εφικτές και μακροχρόνιες οικονομικές παραμέτρους. Ακόμη, η ζήτηση του κάθε πελάτη μοντελοποιείται με τρόπο που προτείνουν οι Ottaviani et al.[25] για τη διαχείριση της στοχαστικής ζήτησης. Για αρχή, ο χώρος χωρίζεται σε οκτώ διακριτές περιοχές με διαφορετικά ποσοστά ζήτησης, ώστε να αποτυπώνεται η χωρική συγκέντρωση των παραγγελιών. Στη συνέχεια, ακολουθεί η ατομική ζήτηση κάθε ενός πελάτη. Αυτή, προσεγγίζεται μέσω εκθετικής κατανομής με παράμετρο λ, επιλογή που ευθυγραμμίζεται με την προσέγγιση στοχαστικής

ζήτησης και επιτρέπει την ύπαρξη πολλών μικρών αλλά και σποραδικών υψηλών τιμών ζήτησης, όπως συμβαίνει και στα σενάρια προσομοίωσης που χρησιμοποιούν οι συγγραφείς για την αξιολόγηση της χωροθέτησης APLs.

2.7.2 Θυρίδες παράδοσης, συμπεριφορά μετακίνησης και βιωσιμότητα

Η έρευνα σχετικά με τις θυρίδες παράδοσης υποδεικνύει ότι αυξάνουν την αποτελεσματικότητα των διανομών. Δηλαδή μειώνουν τις αποτυχημένες παραδόσεις και βελτιώνουν τη συγκέντρωση των στάσεων. Αυτό ισχύει ιδιαίτερα σε πραγματικές αστικές συνθήκες και στα πλαίσια συγκεκριμένων περιπτώσεων (van Duin et al.[1], Pinchasik et al.[5], Hovi et al.[47]). Ωστόσο, το αποτέλεσμα δεν είναι απόλυτα εγγυημένο. Η συνολική απόδοση εξαρτάται από το πώς οι χρήστες ενσωματώνουν την παραλαβή στις καθημερινές τους μετακινήσεις και ποια μέσα μεταφοράς επιλέγουν. Από την πλευρά του καταναλωτή, υπάρχει η δυνατότητα να επιστραφεί ένα μέρος των κερδών που αποκομίζει ο μεταφορέας στο σύστημα. Στην πραγματικότητα, εάν ο χρήστης επιλέξει να χρησιμοποιήσει το αυτοκίνητό του για να φτάσει στη θυρίδα αντί να περπατήσει, το περιβαλλοντικό όφελος μπορεί να αυξηθεί σε μεγάλο βαθμό (Gutenschwager et al.[40], Hofer et al.[41], Pinchasik et al.[5]). Για αυτόν τον λόγο, αυτή η εργασία περιλαμβάνει πρόσβαση που εξαρτάται από πέντε μέσα μεταφοράς που βασίζονται στα εμπειρικά δεδομένα (Pinchasik et al.[5]) και συνδυασμένο κόστος (combined cost: ταξίδι + CO_2). Πιο συγκεκριμένα, για κάθε ένα από τα μέσα μεταφοράς ορίζεται και διαφορετικό όριο προσβασιμότητας. Αυτό, αντικατοπτρίζει με μεγαλύτερη ακρίβεια την πραγματική προθυμία των χρηστών. Οι τιμές που ενσωματώνονται είναι 0.5 χιλιόμετρα για περπάτημα, 1.0 για ποδήλατο, 2.0 για μέσα μαζικής μεταφοράς, 3.5 για οχήματα εσωτερικής καύσης καθώς και 6.0 για ηλεκτρικά οχήματα. Ακόμη, οι τιμές κόστους μετακίνησης που χρησιμοποιούνται στο μοντέλο με εύρος από 0–0.2 €/km ανάλογα με το μέσο, βασίζονται στα εύρη που αναφέρονται στη βιβλιογραφία για αστικές μετακινήσεις. Πιο συγκεκριμένα, το κόστος για δημόσιες συγκοινωνίες (0.05 €/km) ευθυγραμμίζεται με τις εκτιμήσεις του OECD/ITF[44] και των Santos & Caranzo[43]. Από την άλλη, οι τιμές για ιδιωτικό όχημα με κινητήρα εσωτερικής καύσης καθώς και ηλεκτρικό όχημα προσαρμόζονται ώστε να αποτυπώνουν το καθαρό χρηματικό κόστος χωρίς να ενσωματώνουν εξωτερικότητες. Ακόμη, οι εκπομπές CO_2 ανά μέσο μεταφοράς προέρχονται από

τα δεδομένα των Pinchasik et al.[5], και μετατρέπονται σε χρηματικό κόστος μέσω συντελεστή 0.001 €/g CO_2. Αυτός, αντιστοιχεί σε υψηλή τιμή κοινωνικού κόστους άνθρακα (1000 €/tCO_2) σύμφωνα με τους Bilal & Känzig[45]. Το συνδυασμένο κόστος που εισάγεται το υπολογίζουμε ως το άθροισμα του κόστους μετακίνησης και του κόστους των εκπομπών CO_2 που μετατρέπονται σε ευρώ. Με αυτόν τον τρόπο, ενσωματώνουμε το οικονομικό αλλά και το περιβαλλοντικό αποτύπωμα της μετακίνησης. Έτσι, εξασφαλίζεται ότι η βελτιστοποίηση λαμβάνει υπόψη τη βιωσιμότητα από την φάση του σχεδιασμού καθώς και τις πραγματικές συμπεριφορές των χρηστών.

Δεν αρκεί απλώς να βελτιώσεις τη δομή του δικτύου. Είναι πολύ σημαντικό να εξετάσεις και τη συμπεριφορά πρόσβασης από την αρχή. Οι ανασκοπήσεις καταλήγουν ότι το πεδίο βρίσκεται σε μια ισορροπία μεταξύ λειτουργικής αποδοτικότητας, περιβαλλοντικών επιπτώσεων και της εμπειρίας του χρήστη. Υπάρχει ανάγκη να συνδυαστούν τα συμπεριφορικά ευρήματα με αυστηρά σχεδιαστικά μοντέλα (Raviv & Gansterer[20]).

2.7.3 Θυρίδες παράδοσης και Αλγοριθμικός Σχεδιασμός

Η ενσωμάτωση της ετερογένειας αυξάνει κατά πολύ την υπολογιστική πολυπλοκότητα και απαιτεί τη χρήση πιο εξειδικευμένων αλγορίθμων. Πιο συγκεκριμένα, οι Hakimi[29] και οι Krarup & Pruzan[18] αποκάλυψαν ότι ακόμη και οι πολύ απλές εκδοχές των p-median και CFLP είναι NP-hard. Αυτό το γεγονός, καθιστά σχεδόν αδύνατη την επίλυση αυτών σε ρεαλιστικά μεγέθη χρησιμοποιώντας εξαντλητικές μεθόδους. Η βιβλιογραφία καταλήγει σε δύο βασικές κατευθύνσεις. Ειδικότερα περιλαμβάνονται ακριβείς τεχνικές όπως ο Branch and Bound για μικρά και μεσαία σενάρια, καθώς επίσης και ευρετικές ή μεθευρετικές προσεγγίσεις για μεγαλύτερες περιπτώσεις, όπου η πολυπλοκότητα αυξάνεται με εκθετικό ρυθμό (Fleszar & Hindi[32], Yaghini et al.[33]). Στη εργασία αυτή προτείνεται μια αναβαθμισμένη εκδοχή της μεθόδου Branch and Bound. Έχει καθιερωθεί ως ακριβής τεχνική για προβλήματα MILP χωροθέτησης που περιλαμβάνουν δυαδικές αποφάσεις καθώς και περιορισμούς στη χωρητικότητα (Deutsch & Golany[17], El Amrani et al.[22]). Στην πραγματικότητα, προσφέρει αποδεδειγμένα την καλύτερη δυνατή λύση και επιτρέπει την παρακολούθηση της διαφοράς στην βέλτιστη κατάσταση, ενώ ταυτόχρονα

αξιοποιεί και αποτελεσματικά τη δομή UFLP/CFLP με τη βοήθεια ειδικών στρατηγικών διακλάδωσης. Η επιλογή αυτή ευθυγραμμίζεται με τη σύγχρονη προσέγγιση που συνδυάζει ακριβείς και ευρετικές τεχνικές για την επίλυση ζητημάτων χωροθέτησης (Mancini et al.[26]). Ταυτόχρονα, η εφαρμογή ευρετικών προσεγγίσεων κρίνεται απαραίτητη για την ταχεία παραγωγή αποτελεσματικών και ποιοτικών λύσεων που ενισχύουν την απόδοση του αλγορίθμου Branch and Bound. Οι κατασκευαστικές διαδικασίες που βασίζονται σε αποστάσεις, κόστος και κάλυψη ταιριάζουν με τις προσεγγίσεις που έχουν προταθεί για το πρόβλημα p-median καθώς και τη χωρητικότητα στις θέσεις εγκαταστάσεων (El Amrani et al.[22], Tadić et al.[35], Schnieder et al.[19]). Επιπλέον, οι ευρετικές μέθοδοι αποκατάστασης εφικτότητας, όπως οι διαδικασίες επαναανάθεσης και ανταλλαγής, θεωρούνται μια καθιερωμένη πρακτική για την αντιμετώπιση περιορισμών στη χωρητικότητα και στην κάλυψη (Fleszar & Hindi[32], Yaghini et al.[33], Raviv & Gansterer[20]). Με αυτόν τον τρόπο ο ευρετικός σχεδιασμός της εργασίας βασίζεται σε τεχνικές που έχουν αποδεδειγμένα επιτυχία στην επίλυση πολύπλοκων προβλημάτων χωροθέτησης. Επιπλέον, για να βελτιωθεί η ποιότητα των λύσεων και να διερευνηθεί αποτελεσματικά ο χώρος αναζήτησης, αυτή η εργασία χρησιμοποιεί και μεθευρετικές τεχνικές. Αυτές, είναι καθιερωμένες για τα μεγάλης κλίμακας προβλήματα χωροθέτησης. Η εφαρμογή μεθόδων τοπικής αναζήτησης σε πολλές γειτονιές, όπως για παράδειγμα η προσομοιωμένη ανόπτηση και η επαναληπτική τοπική αναζήτηση, ταιριάζει απόλυτα με τις συστάσεις της βιβλιογραφίας για τα προβλήματα p-median και χωρητικότητας εγκαταστάσεων. Στην πράξη, έχουν αποδειχθεί ότι είναι ιδιαίτερα αποτελεσματικές στο συγκεκριμένο πεδίο για τη βελτίωση κατασκευαστικών λύσεων καθώς και την αποφυγή των τοπικών ελαχίστων (Fleszar & Hindi[32], Yaghini et al.[33], Lourenço et al.[34] σχετικά με ILS). Να σημειωθεί επίσης πως η χρήση τους σε θέματα που αφορούν τη συλλογή και παράδοση έχει επιβεβαιωθεί μέσω πρόσφατων μελετών, όπως συμβαίνει με τα Κέντρα Δεδομένων (CDPs) και με τα δίκτυα θυρίδων. Αυτές οι μελέτες αξιοποιούν προσομοιωμένη ανόπτηση καθώς και πολυγειτονικές δομές για να διαχειριστούν ζητήματα χωρητικότητας και προσβασιμότητας (Tadić et al.[35]). Έτσι, η χρήση μεθευρετικών τεχνικών ενδυναμώνει σε μεγάλο βαθμό την ικανότητα του μοντέλου να επιτυγχάνει ποιοτικές λύσεις σε πολύπλοκα και σε υπολογιστικά απαιτητικά περιβάλλοντα.

Κεφάλαιο 3

Υλοποίηση

3.1 Κατασκευή μαθηματικού μοντέλου

Η συγκεκριμένη εργασία επικεντρώνει το ενδιαφέρον της στο να εντοπίσουμε τον βέλτιστο τρόπο συνδυασμού θέσης, τύπου και χωρητικότητας για κάθε εγκατάσταση. Στην πραγματικότητα, θέλουμε να επιλέξουμε P θέσεις και να αναθέσουμε κατάλληλα τους πελάτες, προκειμένου να ελαχιστοποιηθεί η συνολική απόσταση που θα πρέπει να διανύσουν για εξυπηρέτηση. Οι περιορισμοί που εντάσσονται είναι ποικίλοι και αφορούν κυρίως την χωρητικότητα, την ζήτηση, την προσβασιμότητα καθώς και τον διαθέσιμο προϋπολογισμό. Ταυτόχρονα, η διατύπωση του ζητήματος γίνεται ως μικτό ακέραιο γραμμικό πρόγραμμα (MILP) και η επίλυση του πραγματοποιείται μέσω του εμπορικού λύτη Gurobi. Ο συγκεκριμένος λύτης, χρησιμοποιεί τη μέθοδο Branch and Bound για να βρει άμεσα ακέραιες λύσεις. Στην πράξη, λύνει σε κάθε κόμβο το χαλαρωμένο γραμμικό πρόγραμμα μέσω Dual Simplex. Με αυτόν τον τρόπο, επιτυγχάνεται μια αποδοτική κυρίως επιλογή τοποθεσιών καθώς και μια σωστή ανάθεση πελατών, λαμβάνοντας υπόψη οικονομικούς και λειτουργικούς περιορισμούς.

3.1.1 Μεταβλητές Απόφασης και Παράμετροι

Οι μεταβλητές απόφασης του μοντέλου περιλαμβάνουν τις x_{ij} και y_{ki}, οι οποίες είναι δυαδικές και μπορούν να πάρουν μόνο τις τιμές 0 ή 1. Αυτές, εξαρτώνται από δύο δείκτες. Συγκεκριμένα, η μεταβλητή x_{ij} υποδηλώνει εάν θα ανοίξει μια εγκατάσταση τύπου j στη θέση i, ενώ η y_{ki} σχετίζεται με την ανάθεση του πελάτη k σε μία από τις διαθέσιμες θέσεις εγκαταστάσεων i.

$$x_{ij} = \begin{cases} 1, & \text{αν στη θέση } i \text{ ανοίξει εγκατάσταση τύπου } j \\ 0, & \text{διαφορετικά} \end{cases}$$

$$y_{ki} = \begin{cases} 1, & \text{αν ο πελάτης } k \text{ εξυπηρετείται από τη θέση } i \\ 0, & \text{διαφορετικά} \end{cases}$$

Πίνακας 3.1: Βασικές παράμετροι προβλήματος

Παράμετρος	**Τύπος/Διάσταση**	**Περιγραφή**
N	Ακέραιος / 1	Αριθμός υποψήφιων θέσεων εγκατάστασης
E	Ακέραιος / 1	Αριθμός τύπων εγκαταστάσεων (=3)
CL	Ακέραιος / 1	Αριθμός πελατών
P	Ακέραιος / 1	Αριθμός εγκαταστάσεων που θα ανοίξουν
$facility_types$	Ακέραιος / N	Μέγιστος επιτρεπτός τύπος ανά θέση
$capacities$	Ακέραιος / E	Χωρητικότητα ανά τύπο (δέματα)
$demand$	Ακέραιος / CL	Ζήτηση ανά πελάτη (δέματα)
$cl_sp_distances$	Πραγματικός / CL×N	Απόσταση πελάτη-εγκατάστασης (km)
$max_distance$	Πραγματικός / CL	Μέγιστη απόσταση ανά πελάτη (km)
$cost_types$	Πραγματικός / E	NPV κόστος ανά τύπο (€)
C_{max}	Πραγματικός / 1	Διαθέσιμο κεφάλαιο (€)

Πίνακας 3.2: Παράμετροι μεταφοράς και εκπομπών

Παράμετρος	**Τύπος/Διάσταση**	**Περιγραφή**
transport_mode	Ακέραιος / CL	Μέσο μεταφοράς ανά πελάτη (0-4)
emission_values	Πραγματικός / 5	Εκπομπές CO_2 ανά mode (g/km)
transport_cost_per_km	Πραγματικός / 5	Κόστος μετακίνησης ανά mode (€/km)
co2_cost_factor	Πραγματικός / 1	Μετατροπή CO_2 σε κόστος (€/g)

Σημείωση: Οι παράμετροι *combined_cost_per_km* και *total_transport_cost* υπολογίζονται από τις παραπάνω τιμές.

3.1.2 Αντικειμενική συνάρτηση και περιορισμοί

Αντικειμενική Συνάρτηση

$$\text{Minimize} \quad \sum_{k=1}^{CL} \sum_{i=1}^{N} y_{ki} \cdot cl_sp_distances_{ki}$$

Περιορισμοί:

$$\sum_{j=1}^{E} x_{ij} \leq 1, \quad i = 1, \ldots, N \tag{3.1}$$

$$\sum_{i=1}^{N} \sum_{j=1}^{E} x_{ij} = P \tag{3.2}$$

$$x_{ij} = 0, \quad i = 1, \ldots, N, \; j = 1, \ldots, E \text{ όταν } facility_types_i < j \tag{3.3}$$

$$\sum_{i=1}^{N} y_{ki} = 1, \quad k = 1, \ldots, CL \tag{3.4}$$

$$y_{ki} \leq \sum_{j=1}^{E} x_{ij}, \quad k = 1, \ldots, CL, \; i = 1, \ldots, N \tag{3.5}$$

$$y_{ki} = 0, \quad k = 1, \ldots, CL, \; i = 1, \ldots, N \text{ όταν } cl_sp_distances_{ki} > max_distance_k \tag{3.6}$$

$$\sum_{k=1}^{CL} demand_k \cdot y_{ki} \leq \sum_{j=1}^{E} capacities_j \cdot x_{ij}, \quad i = 1, \ldots, N \tag{3.7}$$

$$\sum_{i=1}^{N} \sum_{j=1}^{E} (x_{ij} \cdot cost_types_j) + total_transport_cost \leq C_{\max} \tag{3.8}$$

Περιορισμοί μη αρνητικότητας

$$x_{ij} \geq 0, \quad i = 1, \ldots, N, \; j = 1, \ldots, E$$

$$y_{ki} \geq 0, \quad k = 1, \ldots, CL, \; i = 1, \ldots, N$$

Επεξήγηση Αντικειμενικής Συνάρτησης: Ελαχιστοποίηση της συνολικής απόστασης που διανύουν οι πελάτες για να εξυπηρετηθούν από την πλησιέστερη εγκατάσταση (πρόβλημα p-median). Η τιμή της αντικειμενικής συνάρτησης είναι δεκαδικός αριθμός.

Επεξήγηση περιορισμών:

- **Περιορισμός (3.1)**: Μόνο ένας τύπος εγκατάστασης μπορεί να εγκατασταθεί σε κάθε υποψήφια τοποθεσία.
- **Περιορισμός (3.2)**: Εγγυάται ότι ακριβώς P από τις εγκαταστάσεις θα ανοίξουν.
- **Περιορισμός (3.3)**: Μια εγκατάσταση μπορεί να ανοίξει μόνο αν ο τύπος της είναι επιτρεπτός για τη συγκεκριμένη θέση. Δηλαδή, για κάθε θέση i επιτρέπεται να ανοίξει μόνο ένας τύπος εγκατάστασης j που δεν ξεπερνά το μέγιστο επιτρεπτό τύπο της θέσης.
- **Περιορισμός (3.4)**: Κάθε πελάτης εξυπηρετείται από ακριβώς μία εγκατάσταση.
- **Περιορισμός (3.5)**: Κάθε πελάτης εξυπηρετείται μόνο από ανοιχτές εγκαταστάσεις. Δεν μπορεί να συνδεθεί δηλαδή με κλειστή εγκατάσταση.

- **Περιορισμός (3.6)**: Κάθε πελάτης μπορεί να εξυπηρετηθεί μόνο από εγκατάσταση που βρίσκεται εντός της μέγιστης αποδεκτής απόστασης για τον τρόπο μεταφοράς του. Ο πίνακας $max_distance$ χρησιμοποιείται για τον υπολογισμό της μέγιστης επιτρεπόμενης απόστασης για κάθε πελάτη k για τη μεταφορά του $max_distance[k]$. (Πίνακας 3.1).

- **Περιορισμός (3.7)**: Η συνολική ζήτηση των πελατών που εξυπηρετείται από μία εγκατάσταση δεν μπορεί να υπερβαίνει τη χωρητικότητα της εγκατάστασης, ανάλογα με τον τύπο της.

- **Περιορισμός (3.8)**: Το συνολικό κόστος του συστήματος δεν υπερβαίνει το διαθέσιμο κεφάλαιο C_{max} και περιλαμβάνει: (α) κόστος εγκατάστασης και λειτουργίας θυρίδων ($cost_types$), και (β) κόστος μετακίνησης πελατών.

  Το κόστος μετακίνησης υπολογίζεται με βάση το μέσο μεταφοράς κάθε πελάτη και ενσωματώνει τόσο το άμεσο οικονομικό κόστος όσο και το περιβαλλοντικό κόστος από εκπομπές CO_2. Για κάθε μέσο m, το συνδυαστικό κόστος ανά χιλιόμετρο ορίζεται ως:

$$\begin{aligned}\text{combined_cost_per_km}(m) = \text{transport_cost_per_km}(m) \\ + \text{emission_values}(m) \cdot \text{co2_cost_factor}\end{aligned} \tag{3.9}$$

  όπου $co2_cost_factor$ = 0.001 €/g (ισοδύναμο με 1000 €/tCO_2). Το συνολικό κόστος μετακίνησης είναι:

$$\begin{aligned}\text{total_transport_cost} = \sum_{k=1}^{CL}\sum_{i=1}^{N} y_{ki} \cdot \text{cl_sp_distances}_{ki} \\ \cdot \text{combined_cost_per_km}(\text{transport_mode}_k)\end{aligned} \tag{3.10}$$

3.2 Επίλυση μέσω Gurobi

Για την επίλυση του μαθηματικού μοντέλου, γίνεται χρήση του εμπορικού λύτη Gurobi Optimizer έκδοσης 11.0.3. Αυτός, θεωρείται από τους πιο αποδοτικούς για μικτά ακέραια γραμμικά προγράμματα (MILP). Ο λύτης, χρησιμοποιεί τον αλγόριθμο Branch and Bound προκειμένου να αναζητήσει την καλύτερη ακέραια λύση.

Στην πράξη, σε κάθε κόμβο του δέντρου, επιλύει το αντίστοιχο χαλαρωμένο πρόβλημα (LP relaxation) μέσω της μεθόδου Dual Simplex. Πιο συγκεκριμένα, η εφαρμογή αυτού του αλγορίθμου έγινε σε Python 3.10 χρησιμοποιώντας το API gurobipy. Το μοντέλο δημιουργείται δυναμικά, διαβάζοντας τις πληροφορίες από τα αρχεία που παράγονται από τον γεννήτορα προβλημάτων. Πιο συγκεκριμένα:

- Οι μεταβλητές απόφασης x_{ij} και y_{ki} ορίζονται ως δυαδικές (`GRB.BINARY`)
- Οι περιορισμοί (2.1)-(2.8) προστίθενται με τη συνάρτηση `model.addConstr()`
- Η αντικειμενική συνάρτηση ορίζεται με `model.setObjective()` σε κατάσταση ελαχιστοποίησης (`GRB.MINIMIZE`)

Για την εκτέλεση ορίστηκε ως χρονικό όριο η μία ώρα (TimeLimit 3600 δευτερόλεπτα). Ο Gurobi εφαρμόζει αυτόματα προεπεξεργασία (presolve) καθώς και τεχνικές αποκοπής (cuts) για την επιτάχυνση της σύγκλισης.

Μετά την επίλυση, ελέγχεται η κατάσταση του μοντέλου:

- `GRB.OPTIMAL`: Βρέθηκε η βέλτιστη λύση
- `GRB.INFEASIBLE`: Το πρόβλημα είναι μη εφικτό
- `GRB.TIME_LIMIT`: Έληξε το χρονικό όριο χωρίς να βρεθεί βέλτιστη λύση

Για τα εφικτά προβλήματα, εξάγονται και αποθηκεύονται: η τιμή της αντικειμενικής συνάρτησης, οι ενεργές μεταβλητές ($x_{ij} = 1$, $y_{ki} = 1$) καθώς και ο χρόνος επίλυσης.

Παρακάτω παρουσιάζονται στιγμιότυπα από την εκτέλεση του μοντέλου με τον λύτη Gurobi για ένα ένα υποπρόβλημα της κλάσης 6_36_12_5. Στην πράξη, το υποπρόβλημα αυτό αποτελείται από μέγεθος πλέγματος 6 (grid_size), 36 συνολικά σημεία (grid_size x grid_size), 12 πελάτες (CL), 24 πιθανές θέσεις εγκατάστασης (συνολικά σημεία - CL) και έχει ως στόχο να ανοίξουν 5 εγκαταστάσεις. Στο Σχήμα 3.1 εμφανίζονται οι τεχνικές λεπτομέρειες του μοντέλου που επιλύθηκε. Συγκεκριμένα, περιλαμβάνει 486 περιορισμούς, 360 μεταβλητές (όλες δυαδικές) και 2320 μη μηδενικούς συντελεστές στον πίνακα περιορισμών. Κατά την προεπεξεργασία (0.01 δευτερόλεπτα), οι περιορισμοί μειώθηκαν σε 191, οι μεταβλητές σε 199 μεταβλητές και οι μη μηδενικούς συντελεστές σε 900. Τέλος, εντοπίστηκε ευρετική λύση με αντικειμενική τιμή 6.55.

```
PS C:\Users\marka\Desktop\Python\Diplomatiki_Ergasia\snyk> python improved_model.py problems/6_36_12_5/ --problems 0.txt


==================== Gurobi Facility Location Solver ====================


****************Solving problem 0.txt****************


Set parameter Username
Academic license - for non-commercial use only - expires 2026-11-03
Set parameter TimeLimit to value 3600
Gurobi Optimizer version 11.0.3 build v11.0.3rc0 (win64 - Windows 10.0 (19045.2))

CPU model: AMD Ryzen 3 3200U with Radeon Vega Mobile Gfx, instruction set [SSE2|AVX|AVX2]
Thread count: 2 physical cores, 4 logical processors, using up to 4 threads

Optimize a model with 486 rows, 360 columns and 2320 nonzeros
Model fingerprint: 0x4163ab35
Variable types: 0 continuous, 360 integer (360 binary)
Coefficient statistics:
  Matrix range     [4e-02, 2e+02]
  Objective range  [1e-01, 6e+00]
  Bounds range     [1e+00, 1e+00]
  RHS range        [1e+00, 9e+02]
Found heuristic solution: objective 11.0500000
Presolve removed 295 rows and 161 columns
Presolve time: 0.02s
Presolved: 191 rows, 199 columns, 990 nonzeros
Variable types: 0 continuous, 199 integer (199 binary)
Found heuristic solution: objective 6.5500000
```

Σχήμα 3.1: Στιγμιότυπο 1 επίλυσης με Gurobi

Το Σχήμα 3.2 δείχνει την πρόοδο επίλυσης του προβλήματος από τον λύτη Gurobi. Η αρχική χαλάρωση (root relaxation) έδωσε αντικειμενική τιμή 3.94, η οποία είναι βέλτιστη αφού το GAP έφτασε στο 0.00%. Επίσης, εξερευνήθηκε μόνο 1 κόμβος με 46 simplex iterations σε 0.02 δευτερόλεπτα.

```
Root relaxation: objective 3.940000e+00, 44 iterations, 0.00 seconds (0.00 work units)

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

H    0     0                       3.9400000    0.17000  95.7%     -    0s
     0     0           -    0         3.94000    3.94000  0.00%     -    0s

Explored 1 nodes (46 simplex iterations) in 0.02 seconds (0.00 work units)
Thread count was 4 (of 4 available processors)

Solution count 3: 3.94 6.55 11.05
```

Σχήμα 3.2: Στιγμιότυπο 2 επίλυσης με Gurobi

Στη λύση του Σχήματος 3.3, παρουσιάζουμε τις εγκαταστάσεις που πρέπει να ανοίξουν και τις αναθέσεις των πελατών σε αυτές προκειμένου να επιτευχθεί η ελάχιστη τιμή στόχου (3.94). Οι μεταβλητές απόφασης ερμηνεύονται στην υποενότητα 3.1.1. Το πρόβλημα λύθηκε πλήρως με μηδενικό κενό.

Τέλος, στο Σχήμα 3.4 δίνονται τα αποτελέσματα που αποθηκεύονται για περαιτέρω ανάλυση: μέση αντικειμενική τιμή, μέσος χρόνος εκτέλεσης, χρόνος ανά πρόβλημα και οι λίστες εφικτών, μη εφικτών και μη ολοκληρωμένων προβλημάτων. Για το παράδειγμα μας (αρχείο 0.txt) που είναι εφικτό, έχουμε μέση τιμή αντικειμενικής ίση με 3.94 και χρόνο εκτέλεσης ίσο με 0.033 δευτερόλεπτα.

```
Optimal solution found (tolerance 1.00e-04)
Best objective 3.940000000000e+00, best bound 3.940000000000e+00, gap 0.0000%
Model solved to optimality
Objective value = 3.94


Εγκαταστάσεις που θα ανοίξουν (x[i, j]):
x[6, 2] = 1.0
x[15, 0] = 1.0
x[16, 0] = 1.0
x[21, 2] = 1.0
x[23, 0] = 1.0

Πελάτες που εξυπηρετούνται (y[k, i]):
y[0, 23] = 1.0
y[1, 21] = 1.0
y[2, 16] = 1.0
y[3, 21] = 1.0
y[4, 15] = 1.0
y[5, 21] = 1.0
y[6, 21] = 1.0
y[7, 15] = 1.0
y[8, 23] = 1.0
y[9, 21] = 1.0
y[10, 16] = 1.0
y[11, 6] = 1.0
```

Σχήμα 3.3: Στιγμιότυπο 3 επίλυσης με Gurobi

```
============================== RESULTS ================================


Average objective value for all feasible problems: 3.94.
Average run time for all feasible problems: 0.033 s.

Time of every feasible problem: [0.033] s.


Infeasible problems: []
Feasible problems: ['0.txt']
Aborted problems: []

=======================================================================
```

Σχήμα 3.4: Στιγμιότυπο 4 επίλυσης με Gurobi

3.2.1 Ψευδοκώδικας κατασκευής και επίλυσης μοντέλου με τον λύτη Gurobi

Στον Αλγόριθμο 1 δίνεται ο ψευδοκώδικας κατασκευής και επίλυσης του μοντέλου με την χρήση του λύτη Gurobi.

3.3 Αλγόριθμος Branch and Bound

Η μέθοδος Branch and Bound είναι μια διαδικασία που χρησιμοποιείται για την επίλυση δύσκολων προβλημάτων βελτιστοποίησης. Με λίγα λόγια, οργανώνει μεθοδικά τον χώρο λύσεων και ακολουθεί τη γνωστή λογική ”διαίρει και βασίλευε”. Στην πράξη, το κύριο πρόβλημα διαχωρίζεται σε πιο μικρά υποπροβλήματα. Αυτά, τοποθετούνται σε μια δομή δέντρου αναζήτησης και εξετάζονται με επιλεκτικό τρόπο. Πρόσθετα, για καθένα από τα υποπρόβλημα, υπολογίζεται ένα όριο που δείχνει

Αλγόριθμος 1: Μοντελοποίηση προβλήματος χωροθέτησης με λύτη Gurobi

Input: filename containing problem data
Output: model.status, label, objVal, solvingTime
model = `gp.Model("FacilityLocation");`
$x[i,j]$: binary $(N \times E)$ – facility type j at location i;
$y[k,i]$: binary $(CL \times N)$ – customer k assigned to site i;

for $i = 0$ **to** $N-1$ **do**
  $\sum_{j=0}^{E-1} x[i,j] \leq 1;$
  $\sum_{k=0}^{CL-1} \text{demand}[k] \cdot y[k,i] \;\leq\; \sum_{j=0}^{E-1} \text{capacity}[j] \cdot x[i,j];$
  for $j = 0$ **to** $E-1$ **do**
    if *facility_types*$[i] < j$ **then**
      $x[i,j] = 0;$

for $k = 0$ **to** $CL-1$ **do**
  $\sum_{i=0}^{N-1} y[k,i] = 1;$
  for $i = 0$ **to** $N-1$ **do**
    $y[k,i] \leq \sum_{j=0}^{E-1} x[i,j];$
    if $cl_sp_distances[k,i] >$ *max_distance*$[k]$ **then**
      $y[k,i] = 0;$

$$\sum_{i=0}^{N-1}\sum_{j=0}^{E-1} \text{cost_types}[j] \cdot x[i,j] \;+\; \sum_{k=0}^{CL-1}\sum_{i=0}^{N-1} y[k,i] \cdot \text{cl_sp_distances}[k,i] \cdot \text{combined_cost_per_km}[\text{transport_mode}[k]] \;\leq\; C_{\max};$$

$$\sum_{i=0}^{N-1}\sum_{j=0}^{E-1} x[i,j] = P;$$

minimize $\sum_{k=0}^{CL-1}\sum_{i=0}^{N-1} d[k,i]\, y[k,i];$

```
model.setObjective(MINIMIZE);
model.setParam(TimeLimit, 3600);
model.optimize();
```

Return *model.status*, *label*, *objVal*, *solvingTime*

αν μπορεί να οδηγήσει σε καλύτερη λύση. Αν αυτό δεν είναι εφικτό, απορρίπτεται άμεσα χωρίς περαιτέρω εξέταση. Αυτό, μειώνει δραστικά τον όγκο της αναζήτησης. Ο τρόπος με τον οποίο θα εξερευνηθούν τα υποπροβλήματα επηρεάζει κατά μεγάλο βαθμό την αποδοτικότητα της μεθόδου. Κάποιες από τις στρατηγικές που χρησιμοποιούνται είναι η αναζήτησης κατά βάθος (Depth First Search) και η βέλτιστη αναζήτηση πρώτα (Best First Search). Η μέθοδος εφαρμόζεται σε πληθώρα προβλημάτων, μεταξύ των οποίων ο ακέραιος προγραμματισμός, τα προβλήματα χωροθέτησης καθώς και οι διάφορες παραλλαγές του p-median. [48]

3.3.1 Υλοποίηση

Κατά την υλοποίηση του αλγορίθμου Branch and Bound με στρατηγική αναζήτησης κατά βάθος (Depth First Search), χρησιμοποιείται μια δομή στοίβας. Η στοίβα λειτουργεί με τη λογική Last-In, First-Out (LIFO). Ξεκινά η διαδικασία με τη λύση του χαλαρωμένου προβλήματος στο root, δηλαδή του μοντέλο χωρίς περιορισμούς ακεραιότητας. Η τιμή της αντικειμενικής συνάρτησης από τη χαλαρωμένη λύση γραμμικού προγραμματισμού στο root είναι το πρώτο κατώτερο όριο. Το κατώτερο όριο είναι το καλύτερο όριο για όλες τις ακέραιες λύσεις που μπορεί να εμφανιστούν μετά. Αν η λύση του root είναι ακέραια, η διαδικασία τερματίζει αμέσως.

Όταν η λύση σε έναν κόμβο δεν είναι ακέραια, επιλέγουμε μια μεταβλητή που έχει κλασματική τιμή και δημιουργούμε δύο νέα υποπροβλήματα: στο πρώτο περιορίζουμε τη μεταβλητή στο κατώτερο ακέραιο μέρος της (αριστερό παιδί), ενώ στο δεύτερο την περιορίζουμε στο ανώτερο ακέραιο μέρος της (δεξί παιδί). Αυτή η διαδικασία διακλάδωσης εφαρμόζεται σε κάθε μη ακέραιο κόμβο, και όχι μόνο στο αρχικό πρόβλημα. Τα δύο παιδιά προστίθενται στη στοίβα με τέτοιον τρόπο ώστε το αριστερό υποπρόβλημα να εξετάζεται πρώτο, σύμφωνα με τη λογική του DFS.

Σε κάθε νέο κόμβο, το μοντέλο ανανεώνεται με τα καθορισμένα όρια της διακλάδωσης και γίνεται νέα επίλυση. Για να επιταχυνθεί η διαδικασία επίλυσης, χρησιμοποιείται προσέγγιση warm start μέσω των βάσεων (VBasis και CBasis) που μεταφέρονται από τον γονικό κόμβο. Αν το υποπρόβλημα δεν είναι εφικτό, ο συγκεκριμένος κόμβος απορρίπτεται. Εάν η λύση είναι ακέραια, συγκρίνεται με το τρέχον ανώτατο όριο (upper bound) για το πρόβλημα ελαχιστοποίησης. Αν υπάρ-

χει βελτίωση, τότε ενημερώνεται το ανώτατο όριο. Σε περίπτωση που η λύση είναι κλασματική και το κατώτατο όριο (lower bound) του κόμβου είναι χειρότερο από το υπάρχον ανώτατο όριο, ο κόμβος αποκόπτεται χωρίς να γίνουν περισσότερες διακλαδώσεις.

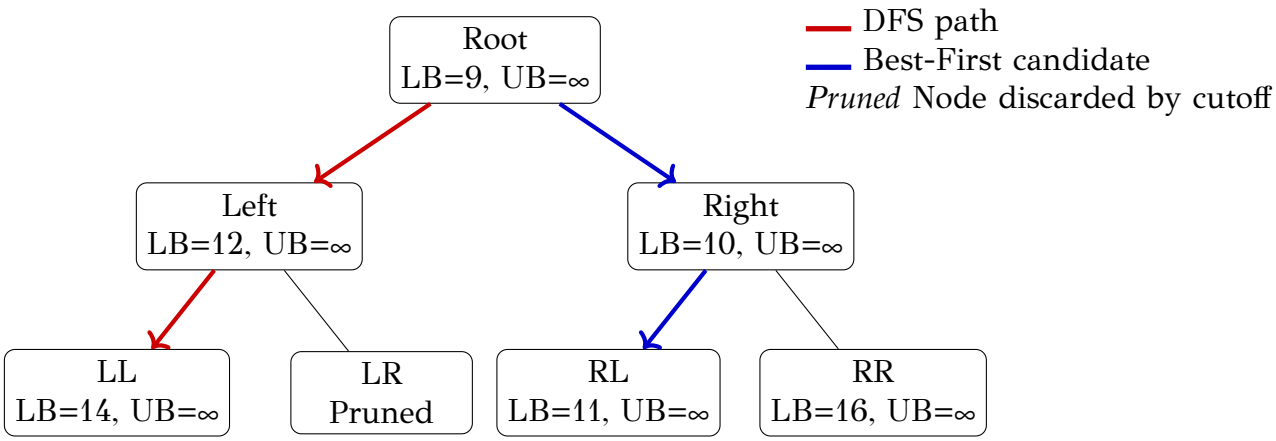
Η διαδικασία επαναλαμβάνεται συνεχώς, επιλέγοντας πάντοτε τον κόμβο στην κορυφή της στοίβας. Αυτό γίνεται μέχρι η στοίβα να αδειάσει ή να έχουμε ισότητα μεταξύ του global lower bound και του upper bound. Σε αυτή την περίπτωση, επιβεβαιώνεται η βέλτιστη επιλογή της λύσης και η διαδικασία αναζήτησης σταματά.

Στον Branch and Bound με βέλτιστη αναζήτηση πρώτα (Best First Search), η διαδικασία είναι παρόμοια με τη Depth First Search, αλλά αλλάζει το πώς διαλέγουμε τον επόμενο κόμβο. Αντί για στοίβα, χρησιμοποιείται μια δομή λίστας που διατηρεί όλους τους ενεργούς κόμβους ταξινομημένους με βάση την «υποσχόμενη» συμπεριφορά τους. Σε κάθε βήμα επιλέγεται για επέκταση ο κόμβος με το χαμηλότερο εκτιμώμενο κάτω όριο (lower bound), καθώς θεωρείται ότι έχει τις μεγαλύτερες πιθανότητες να οδηγήσει γρήγορα σε βέλτιστη λύση. Σε περίπτωση ισοβαθμίας, η επιλογή γίνεται με βάση τη σειρά δημιουργίας των κόμβων, δίνοντας προτεραιότητα σε εκείνον με το μικρότερο αναγνωριστικό (node_id). Με αυτόν τον τρόπο, η αναζήτηση κατευθύνεται συστηματικά προς τις πιο υποσχόμενες περιοχές του χώρου λύσεων. Σημειώνεται ότι στο χαλαρωμένο πρόβλημα (LP relaxation) οι δυαδικές μεταβλητές του μοντέλου αντιμετωπίζονται ως συνεχείς, επιτρέποντας την αποδοτική επίλυση κάθε κόμβου μέσω γραμμικού προγραμματισμού.

Για τη μέθοδο Branch and Bound υλοποιήθηκαν δύο τεχνικές. Η πρώτη (απλή) αφορά τον αλγόριθμο Branch and Bound με στρατηγική DFS και σειριακή επιλογή μεταβλητής. Η δεύτερη (βελτιωμένη) τεχνική αφορά τον αλγόριθμο Branch and Bound με υβριδική στρατηγική, που συνδυάζει DFS και Best First Search. Ο σκοπός είναι να αξιοποιηθούν τα πλεονεκτήματα και των δύο στρατηγικών. Ακόμη, στην δεύτερη τεχνική εφαρμόζεται μια σειρά από βελτιωτικές μεθόδους, οι οποίες θα περιγραφούν στις επόμενες ενότητες.

Το Σχήμα 3.5 δείχνει ένα απλό δέντρο Branch and Bound, στο οποίο συγκρίνονται οι στρατηγικές εξερεύνησης DFS και Best-First. Η κόκκινη διαδρομή δείχνει τη συμπεριφορά του DFS, το οποίο ακολουθεί πάντα το αριστερότερο διαθέσιμο μονοπάτι, προχωρώντας σε βάθος μέχρι να φτάσει σε φύλλο. Αντίθετα, η μπλε δια-

δρομή αντιστοιχεί στη στρατηγική Best First, η οποία επιλέγει για επέκταση τον κόμβο με το μικρότερο εκτιμώμενο κάτω όριο (LB), ανεξάρτητα από τη θέση του στο δέντρο. Οι κόμβοι που χαρακτηρίζονται ως Pruned αποκόπτονται επειδή το κάτω όριό τους δεν μπορεί να οδηγήσει σε καλύτερη λύση. Το παράδειγμα αναδεικνύει τη διαφορά προτεραιοτήτων των δύο μεθόδων και τον τρόπο με τον οποίο επηρεάζουν την πορεία αναζήτησης.



Σχήμα 3.5: Παράδειγμα δέντρου Branch and Bound με Depth First Search και Best First Search

3.4 Απλή Έκδοση Branch and Bound

3.4.1 Υλοποίηση

Η υλοποποίση της πρώτης έκδοσης Branch and Bound δημιουργήθηκε με τη γλώσσα προγραμματισμού Python έκδοσης 11.0.3. Παράλληλα, για την επίλυση των γραμμικών χαλαρώσεων σε κάθε κόμβο του δέντρου αναζήτησης χρησιμοποιήθηκε η βιβλιοθήκη του Gurobi. Να σημειωθεί, ότι όλες οι μεταβλητές απόφασης που συμμετέχουν στο πρόβλημα συγκεντρώνονται σε ένα ενιαίο μονοδιάστατο διάνυσμα. Αυτό, φαίνεται να απλοποιεί σε μεγάλο βαθμό την παρακολούθηση των ανωτέρων και κατωτέρων ορίων κατά τη διάρκεια όλης της διαδικασίας διακλάδωσης. Για την εξερεύνηση του δέντρου εφαρμόζεται η τεχνική αναζήτησης βάθους (Depth First Search), όπως εξηγείται στην υποενότητα 3.3.1. Στην πράξη, για κάθε κάθε κόμβο που αναπαρίσταται από την κλάση Node, αποθηκεύονται τα όρια των μεταβλητών, το βάθος του κόμβου και οι βάσεις simplex (VBasis/CBasis) που βοηθούν για τη θερμή εκκίνηση. Επίσης, περιέχει τη μεταβλητή στην οποία έχει γίνει η διακλάδωση

καθώς και την αντίστοιχη λεζάντα της (Left/Right). Η διαδικασία επιλογής της μεταβλητής για τη διακλάδωση είναι απλή και διαφανής. Δηλαδή, μετά την επίλυση της χαλάρωσης, οι ακέραιες μεταβλητές εξετάζονται κατά την σειρά που εμφανίζονται στο μοντέλο. Η πρώτη μεταβλητή που αποκτά μη ακέραια τιμή επιλέγεται.

3.4.2 Ψευδοκώδικας Απλής Έκδοσης Branch and Bound

Στον Αλγόριθμο 2 και 3 παρουσιάζεται ο ψευδοκώδικας που αφορά τον αλγόριθμο Branch and Bound με στρατηγική DFS και σειριακή επιλογή μεταβλητής.

Αλγόριθμος 2: Απλή Έκδοση Branch and Bound (Μέρος 1ο)

Input: $model$, ub, lb, $integer_var$, $best_bound_per_depth$, $nodes_per_depth$,$start$, $vbasis$, $cbasis$, $depth$

Output: $solutions$, $best_sol_idx$, $solutions_found$

Initialize stack $\leftarrow \emptyset$; solutions $\leftarrow [\ ]$; $solutions_found \leftarrow 0$; $best_sol_idx \leftarrow 0$;

Set $best_sol_obj \leftarrow -\infty$ if $isMax$ else $+\infty$;

Create $root_node \leftarrow Node(ub, lb, depth, vbasis, cbasis)$;

Update $nodes_per_depth[0] \leftarrow nodes_per_depth[0] - 1$;

Solve LP relaxation at root;

if *model is infeasible* **then**

  if $isMax$ **then**

    Return $(\emptyset,\ -\infty,\ depth)$;

  else

    Return $(\emptyset,\ +\infty,\ depth)$;

Store LP solution x, objective z;

Update $best_bound_per_depth[0] \leftarrow z$;

Check integrality and select first fractional variable k;

if x *integer feasible* **then**

  Store solution;

else

  Update global bound ($lower$ for min, $upper$ for max);

Retrieve simplex basis $(VBasis, CBasis)$ for $root_node$;

Create two child nodes with $x_k \leq \lfloor x_k \rfloor$ and $x_k \geq \lceil x_k \rceil$;

Push children to stack;

Αλγόριθμος 3: Απλή Έκδοση Branch and Bound (Μέρος 2ο)

```
while stack ≠ ∅ do
    Pop current_node, increment explored nodes, apply bounds (lb, ub);
    if current_node has basis then
        Warm-start LP using (VBasis, CBasis);
    Solve LP relaxation at current_node;
    if infeasible then
        Skip node and update nodes-per-depth;
    else
        Store LP solution x, objective z;
        Update best_bound_per_depth at current depth;
        Update global bound if all nodes at this depth;
        visited (lower for min, upper for max);
    if all integer variables in x are integers then
        if isMax and x.z > lower_bound then
            Update lower_bound and store solution (x, z);
            Update best solution index if necessary;
            if solution optimal (lower ≈ upper) then
                Return solutions, best_sol_idx, solutions_found
        else if isMin and x.z < upper_bound then
            Update upper_bound and store solution (x, z);
            Update best solution index if necessary;
            if solution optimal (lower ≈ upper) then
                Return solutions, best_sol_idx, solutions_found
        Prune children nodes at future depths (do not branch further);
    else if fractional solution cannot improve incumbent: then
        Prune node(do not branch);
    else
        Continue branching on first fractional variable r;
        Retrieve simplex basis (VBasis, CBasis) for current_node;
        Create two child nodes with x_r ≤ ⌊x_r⌋ and x_r ≥ ⌈x_r⌉;
        Push children to stack;
Return solutions, best_sol_idx, solutions_found
```

3.5 Βελτιωμένη Έκδοση Branch and Bound

3.5.1 Υλοποίηση

Η ενότητα αυτή αναλύει τις διάφορες βελτιώσεις που έχουν εφαρμοστεί στον αλγόριθμο Branch and Bound. Αυτές συγκεκριμένα αφορούν τη δομή των δεδομένων στους κόμβους, τη τεχνική που ακολουθείται για την εξερεύνηση του δέντρου καθώς και την επιλογή της μεταβλητής με βάση την οποία γίνονται οι διακλαδώσεις. Ακόμη, την εφαρμογή περιορισμών, τη διαχείριση των βασικών πληροφοριών και τον προσδιορισμό μιας αρχικής εφικτής λύσης μέσω ευρετικών τεχνικών και μευθερετικών. Όλα τα παραπάνω υλοποιούνται για τη βελτίωση της ποιότητας των λύσεων καθώς και την ενίσχυση της υπολογιστικής αποδοτικότητας.

Η δομή της ενότητας έχει ως εξής: Στην υποενότητα 3.5.2, γίνεται ανάλυση της στρατηγικής εξερεύνησης. Η υποενότητα 3.5.3 εστιάζει στις δομές δεδομένων που χρησιμοποιούνται για τους κόμβους και στα κριτήρια που καθορίζουν την εξερεύνηση. Ταυτόχρονα, στην υποενότητα 3.5.4 εξετάζεται η διαδικασία προετοιμασίας των απαιτούμενων δεδομένων και πραγματοποιείται ο υπολογισμός διαφόρων μετρικών που είναι απαραίτητες για τη διακλάδωση. Έπειτα, στην υποενότητα 3.5.5 αναλύεται η στρατηγική επιλογής της μεταβλητής διακλάδωσης. Ακολουθεί η υποενότητα 3.5.6, η οποία περιλαμβάνει τεχνικές που συμβάλλουν στην ενίσχυση των περιορισμών. Στην υποενότητα 3.5.7 δίνονται οι βοηθητικές συναρτήσεις που χρησιμοποιούνται για την ανάπτυξη ευρετικών και μεθευρετικών αλγορίθμων. Στο τέλος, οι υποενότητες 3.5.8 και 3.5.9 εξετάζουν τις ευρετικές και μεθευρετικές προσεγγίσεις που χρησιμοποιούνται για τον εντοπισμό και τη βελτίωση της αρχικής εφικτής λύσης.

3.5.2 Στρατηγική Εξερεύνησης

Η διερεύνηση του δέντρου στηρίζεται σε μια υβριδική στρατηγική με δύο φάσεις. Στην αρχή, και μόνο εφόσον δεν έχει βρεθεί καμία ακέραια λύση μέσω ευρετικών ή μεθευρετικών προσεγγίσεων, ξεκινάει μια διαδικασία αναζήτησης βάθους (DFS). Αυτή, αποσκοπεί στην πιο γρήγορη εύρεση της πρώτης εφικτής λύσης. Στην πράξη, αυτή η φάση διαρκεί για μια χρονική περίοδο που επιλέγει ο χρήστης, ώστε να διατηρείται ισορροπία ανάμεσα στην ταχύτητα εύρεσης μιας αρχικής λύσης και τον

συνολικό χρόνο αναζήτησης. Σ' αυτή τη φάση, οι κόμβοι διερευνούνται ακολουθώντας αυστηρά την εξερεύνηση κατά βάθος, χρησιμοποιώντας στοίβα (LIFO). Σε κάθε διακλάδωση επιλέγεται ως προτιμώμενο παιδί (preferred child) αυτό που σχετίζεται κυρίως με τη φυσική στρογγυλοποίηση της κλασματικής τιμής της μεταβλητής στο χαλαρωμένο πρόβλημα . Το προτιμώμενο αυτό παιδί τοποθετείται στην κορυφή της στοίβας και εξερευνάται άμεσα, ενώ το εναλλακτικό παιδί αποθηκεύεται προσωρινά σε βοηθητική λίστα (deferred list) για μεταγενέστερη επεξεργασία.

Μετά την ολοκλήρωση της αναζήτησης βάθους, οι κόμβοι που έχουν αποθηκευτεί μεταφέρονται φιλτραρισμένοι σε μια δομή λίστας. Έτσι απορρίπτονται όσοι δεν μπορούν να βελτιώσουν την τρέχουσα καλύτερη λύση. Οι κόμβοι ταξινομούνται στη λίστα με βάση συγκεκριμένο κριτήριο που αναλύεται στην υποενότητα 3.5.3 . Έπειτα, ο αλγόριθμος συνεχίζει αποκλειστικά με στρατηγική αναζήτησης Best First, εξασφαλίζοντας συστηματική εξερεύνηση και απόδειξη βελτιστότητας.

Η υβριδική προσέγγιση επιλέγεται ώστε να συνδυαστούν τα συμπληρωματικά πλεονεκτήματα των δύο μεθόδων: η αναζήτηση βάθους προσφέρει γρήγορη εύρεση πρώτης ακέραιας λύσης και άμεση αρχικοποίηση του άνω ορίου (upper_bound), ενώ η αναζήτηση κατά προτεραιότητα (Best First) εξασφαλίζει συστηματική εξερεύνηση των πιο υποσχόμενων κόμβων με βάση την τιμή της αντικειμενικής συνάρτησης που προκύπτει από τη χαλαρωμένη εκδοχή του προβλήματος (LP relaxation). Η τιμή αυτή λειτουργεί ως φράγμα για το ακέραιο πρόβλημα και δείχνει πόσο «καλό» μπορεί να γίνει το υποπρόβλημα αν συνεχίσουμε την εξερεύνηση, ενισχύοντας έτσι το κλάδεμα και επιταχύνοντας την απόδειξη βελτιστότητας.

3.5.3 Δομή Κόμβου και Κριτήριο Εξερεύνησης

Για κάθε κόμβο του δέντρου αναζήτησης αποθηκεύονται όλες οι πληροφορίες που περιγράφουν πλήρως την κατάσταση του αντίστοιχου υποπροβλήματος. Για λόγους αποδοτικότητας μνήμης, η υλοποίηση βασίζεται στον μηχανισμό __slots__ της Python, ο οποίος αποτρέπει την αποθήκευση των ιδιοτήτων σε λεξικό και μειώνει σημαντικά το επιπλέον κόστος μνήμης. Η επιλογή αυτή είναι κρίσιμη, καθώς κατά την εκτέλεση δημιουργούνται χιλιάδες κόμβοι. Επίσης, όλες οι μεταβλητές απόφασης αναδιατάσσονται σε ένα ενιαίο μονοδιάστατο διάνυσμα για την απλούστερη διαχείριση των άνω και κάτω ορίων κατά τη διακλάδωση..

Ο κόμβος περιλαμβάνει τα άνω και κάτω όρια των μεταβλητών, το βάθος του κόμβου στο δέντρο και τις βάσεις του simplex (VBasis/CBasis) για warm-start κατά την επίλυση του χαλαρωμένου προβλήματος (LP relaxation). Ακόμη, περιέχει τον δείκτη της μεταβλητής που έχει επιλεχθεί για διακλάδωση και την τιμή της στη λύση του χαλαρωμένου προβλήματος, την τιμή της αντικειμενικής συνάρτησης στο υποπρόβλημα, καθώς και μια λεζάντα αναγνώρισης (Left/Right). Τέλος, κάθε κόμβος διαθέτει μοναδικό αναγνωριστικό (node_id), το οποίο αυξάνεται αυτόματα με τη χρήση ενός εσωτερικού μετρητή.

Για να εφαρμοστεί η Best First Search, χρειάζεται να καθοριστεί ένα κριτήριο που θα αποφασίζει ποιος κόμβος θεωρείται πιο υποσχόμενος και άρα ποιος θα εξερευνηθεί πρώτος. Το κριτήριο αυτό βασίζεται στην τιμή της αντικειμενικής συνάρτησης που προκύπτει από την επίλυση του χαλαρωμένου προβλήματος, η οποία λειτουργεί ως φράγμα για το ακέραιο πρόβλημα (κάτω φράγμα για ελαχιστοποίηση, άνω φράγμα για μεγιστοποίηση). Έτσι, προτεραιότητα δίνεται στον κόμβο με την καλύτερη εκτίμηση (χαμηλότερη τιμή για ελαχιστοποίηση), ενώ σε περίπτωση ισοπαλίας επιλέγεται ο κόμβος με το μικρότερο αναγνωριστικό node_id.

3.5.4 Προεπεξεργασία και Υπολογισμός Μετρικών για Διακλάδωση

Πριν την εκτέλεση του αλγορίθμου Branch and Bound εκτελείται μια διαδικασία προεπεξεργασίας δεδομένων, με στόχο να μειωθεί ο υπολογιστικός φόρτος και να επιταχυνθεί η διαδικασία επιλογής μεταβλητών διακλάδωσης. Ακολουθεί η επεξήγηση διάφορων μετρικών που υπολογίζονται στο στάδιο της προεπεξεργασίας. Αυτές, είναι σταθερές ανά πρόβλημα και εκφράζουν με διαφορετική προσέγγιση την εφικτότητα και τη σημασία μιας ανάθεσης πελατών σε εγκαταστάσεις. Χρησιμοποιούνται για να καθοδηγήσουν αποτελεσματικά τη διαδικασία διακλάδωσης. Συγκεκριμένα, το dist_feasible φιλτράρει μη ρεαλιστικές αναθέσεις βάσει μέγιστης απόστασης, το cust_scar εντοπίζει «δύσκολους» πελάτες με λίγες επιλογές (λόγω μεγάλης ζήτησης ή προσβασιμότητας), ενώ τα feas_cnt και feas_dem μετρούν πόσους πελάτες και πόση ζήτηση μπορεί να καλύψει κάθε εγκατάσταση. Επίσης, το invd ευνοεί εγκαταστάσεις κοντά σε πελάτες με υψηλή ζήτηση, μειώνοντας το κόστος/απόσταση και το scarcity_site προτεραιοποιεί εγκαταστάσεις που εξυπηρετούν σπάνιους, «κρίσιμους» πελάτες. Οι παραπάνω υπολογισμοί, κανονικοποιούνται εντός

του διαστήματος [0,1]. Με αυτόν τον τρόπο συμβάλλουν ισότιμα στη συνολική βαθμολογία για την επιλογή μεταβλητών διακλάδωσης. Παράλληλα, κατά τη διάρκεια της φάσης προεπεξεργασίας, ενσωματώνεται η διαδικασία Distance-Based Pruning που εφαρμόζεται για να περιοριστεί ο χώρος αναζήτησης. Στην πραγματικότητα, η βασική ιδέα είναι να αποκλείσουμε εκ των προτέρων τις μεταβλητές ανάθεσης y[k,i] που παραβιάζουν τον περιορισμό της μέγιστης απόστασης, καθορίζοντας το ανώτατο όριο τους σε μηδέν. Αυτή η προσέγγιση αποφεύγει εξαρχής μη εφικτές λύσεις, κάνοντάς την υλοποίηση πιο αποτελεσματική.

3.5.5 Επιλογή Μεταβλητής Διακλάδωσης

Η επιλογή της μεταβλητής για την διακλάδωση έχει σημαντική επίδραση στην αποτελεσματικότητα του αλγορίθμου Branch and Bound. Επομένως, αντί να χρησιμοποιήσουμε τη σειριακή μέθοδο που περιγράφεται στην υποενότητα 3.4.1 προτείνουμε μια εναλλακτική προσέγγιση. Εφαρμόζουμε μια υβριδική στρατηγική που προσαρμόζεται κυρίως στις ανάγκες του συγκεκριμένου μοντέλου.

Να σημειωθεί, πως οι αποφάσεις που σχετίζονται με το άνοιγμα εγκαταστάσεων x[i,j] είναι πιο σημαντικοί σε σύγκριση με τις μεταβλητές ανάθεσης πελατών y[k,i]. Επιπλέον, υπάρχει μια υποστηρικτική συνάρτηση που βασίζεται στη κλασματικότητα, δηλαδή πόσο απέχει μια μεταβλητή από την πιο κοντινή ακέραια τιμή της. Είναι σημαντικό οι μεταβλητές με κλασματικότητα κάτω από 0.05 να παραλείπονται ώστε να αποφεύγονται οι διακλαδώσεις σε σχεδόν ακέραιες τιμές.

Στις μεταβλητές x[i,j], η στρατηγική δεν αναλύει κάθε μεταβλητή χωριστά. Αντίθετα, αξιολογεί αρχικά ποια εγκατάσταση είναι πιο «σημαντική». Στην πραγματικότητα, θεωρείται ότι θα επιλεγεί η εγκατάσταση που διαθέτει τη μεγαλύτερη συνολική κλασματικότητα σε όλους τους τύπους της, υποστηριζόμενη από μια πολυκριτηριακή αξιολόγηση. Αυτό το σκορ, συνδυάζει τέσσερις μετρικές που προαναφέρθηκαν στην υποενότητα 3.5.4: feas_cnt (κάλυψη πελατών), feas_dem (συνολική ζήτηση), invd (ευνοεί εγκαταστάσεις κοντά σε πελάτες υψηλής ζήτησης) και scarcity_site (βαθμός εξυπηρέτησης σπάνιων πελατών), με βάρη που δίνουν μεγαλύτερη έμφαση στη συνολική ζήτηση που μπορεί να καλύψει μια εγκατάσταση και στην εξυπηρέτηση πελατών με περιορισμένες επιλογές. Για την επιλεγμένη εγκατάσταση, ο τύπος της επιλέγεται με σκορ που συνδυάζει την κλασματικότητα της

μεταβλητής με έναν συντελεστή χωρητικότητας, ο οποίος ενισχύεται όταν υπάρχει έλλειμμα χωρητικότητας (capacity pressure). Η ένταση της ενίσχυσης ρυθμίζεται από τα βάρη wcap_dive και wcap_best: στη φάση αναζήτηση βάθους η χωρητικότητα έχει μεγαλύτερη επίδραση για γρήγορη εφικτότητα, ενώ στη φάση Best First αναζήτησης η επίδραση είναι πιο ήπια για ισορροπία με την κλασματικότητα. Ο συντελεστής capacity pressure αυξάνει κυρίως το βάρος σε τύπους με μεγαλύτερη χωρητικότητα όταν η λύση του χαλαρωμένου προβλήματος υπολείπεται της συνολικής ζήτησης, έτσι ώστε να επιτυγχάνεται πιο γρήγορα εφικτότητα. Έτσι, σε περιπτώσεις πίεσης προτιμώνται τύποι με μεγαλύτερη χωρητικότητα, ενώ χωρίς πίεση η επιλογή βασίζεται κυρίως στην κλασματικότητα.

Αν δεν βρεθεί μη ακέραια x[i,j] μεταβλητή, η στρατηγική συνεχίζει με τις y[k,i] μεταβλητές, δίνοντας προτεραιότητα σε πελάτες με περιορισμένες επιλογές ανάθεσης. Αρχικά, ελέγχεται ποιες από τις εγκαταστάσεις μπορούν ακόμα να ανοίξουν στον τρέχοντα κόμβο. Δηλαδή να έχουν τουλάχιστον έναν τύπο με μη μηδενικό άνω όριο. Στη συνέχεια, για κάθε πελάτη καθορίζεται πόσες επιλογές ανάθεσης είναι διαθέσιμες. Πιο συγκεκριμένα υπολογίζεται ένα σκορ για κάθε μεταβλητή y[k,i], το οποίο συνδυάζει την κλασματικότητα της μεταβλητής με δύο δείκτες σπανιότητας: τον node-specific (λίγες επιλογές στο τρέχον node) και τον instance-constant (cust_scar). Στην πράξη, οι πελάτες με περιορισμένες επιλογές αποκτούν μεγαλύτερη σημασία, ώστε να προτεραιοποιούνται αναθέσεις που είναι κρίσιμες για την υλοποίηση της λύσης και να ελαχιστοποιούνται οι πιθανότητες αποτυχίας.

Ψευδοκώδικας Επιλογής Μεταβλητής Διακλάδωσης

Στον Αλγόριθμο 4 και 5 δίνεται ο ψευδοκώδικας για την επιλογή μεταβλητής διακλάδωσης με υβριδική στρατηγική.

Αλγόριθμος 4: Επιλογή Μεταβλητής με Υβριδική Στρατηγική Μέρος 1ο

Input: Solution LP x, $integer_var$, N, E, CL, $facility_types$, $capacities$, $demand$, $dist_feasible$, $cust_scar$, $feas_cnt$, $feas_dem$, $invd$, $scarcity_site$, ub, $phase$, eps_int, $frac_threshold$, $pressure_scale$, $pressure_max$, w_{cnt}, w_{dem}, w_{invd}, w_{scar_site}, w_{cap_dive}, w_{cap_best}

Output: $(best_idx, best_val)$

```
// Fractionality
Function frac(v):
    v' ← max(0, min(1, v));
    Return min(v', 1 − v')
```

$x_{open} \leftarrow x[0{:}N \cdot E]$ as array $N \times E$;

$expected_capacity \leftarrow \sum_{i,j} x_{open}[i,j] \cdot capacities[j]$;

$gap \leftarrow \max(0, \sum_k demand[k] - expected_capacity)$;

// $pressure_scale$ controls how fast pressure grows with the capacity gap, while $pressure_max$ caps the pressure to avoid extreme values

$cap_pressure \leftarrow \min\left(pressure_max,\ 1 + \frac{gap}{pressure_scale \cdot \sum_k demand[k] + 10^{-9}}\right)$;

$w_cap \leftarrow w_cap_dive$ **if** $phase$ = "dive" **else** w_cap_best;

```
// Step 2: X-Branching
```

for $i = 0, \ldots, N-1$ **do**

  $site_frac[i] \leftarrow \sum_{j=0}^{facility_types[i]} \text{frac}(x_{open}[i,j])$;

end

for $i = 0, \ldots, N-1$ **do**

  if $site_frac[i] < frac_threshold$ **then**

    continue

  end

  $impact[i] = 1 + w_cnt \cdot feas_cnt[i] + w_dem \cdot feas_dem[i] + w_invd \cdot invd[i] + w_scar_site \cdot scarcity_site[i]$;

  $score_i \leftarrow site_frac[i] \cdot impact$;

  if $i = 0$ ***or*** *score_i is best so far* **then**

    $best_site \leftarrow i$;

  end

end

Αλγόριθμος 5: Επιλογή Μεταβλητής με Υβριδική Στρατηγική Μέρος 2ο

```
if best_site found then
    // Filter before scoring: declares as integer in the model, not
       almost integer (|v − round(v)| > eps_int), fractionality
       ≥ frac_threshold
       For each feasible type j at the site best_site:
        score_j ← frac(x_ij) · (1 + w_cap · cap_pressure · capacities[j] / max_j capacities[j]);
       (best_idx, best_val) ← arg max_j{score_j};
       if a valid variable was found then
           Return (best_idx, best_val);
       end
end
// Step 3: Y-Branching
if ub = None then
    Return variable y[k, i] with max fractionality;
end
for i = 0, ..., N − 1 do
    can_open[i] ← True if ∃j ∈ [0, ..., facility_types[i]] : ub[i · E + j] > eps_int,
     else False;
end
for every customer k = 0, ..., CL − 1 do
    options_node ← max (1, |{i : dist_feasible[k, i] ∧ can_open[i]}|);
    node_scar ← 1/options_node ;                    // node-specific scarcity
    // Filter before scoring: declared as integer in the model, not
       almost integer (|v - round(v)| > eps_int), fractionality ≥
       frac_threshold
    For each feasible site i for customer k:
       score_ki ←
        (1 + pressure_max · node_scar + w_scar_cite · cust_scar[k]) · frac(y_ki);
end
(best_idx, best_val) ← arg max_{k,i}{score_ki} with frac(y_ki) ≥ frac_threshold;
Return (best_idx, best_val);
```

3.5.6 Τεχνικές Διάδοσης Περιορισμών

Κατά τη διαδικασία του αλγορίθμου Branch and Bound, πέρα από το κλάδεμα βάση ορίων ενσωματώνονται τρεις τεχνικές προώθησης περιορισμών. Εφαρμόζονται κατά τη δημιουργία των παιδιών-κόμβων του αλγορίθμου και αφορούν τόσο τις μεταβλητές εγκατάστασης x[i,j] όσο και τις μεταβλητές ανάθεσης y[k,i]. Είναι κατασκευασμένες σύμφωνα με τη δομή και τους περιορισμούς του μοντέλου, ώστε να διατηρείται η εφικτότητα και να μειώνεται ο χώρος αναζήτησης. Συγκεκριμένα:

(A) αν μια μεταβλητή x[i,j] έχει «κλειδώσει» στο 1, δηλαδή έχει ανοίξει εγκατάσταση τύπου j στο σημείο i, τότε όλες οι υπόλοιπες μεταβλητές x[i,j'] ($\forall j' \neq j$) για αυτή την εγκατάσταση τίθενται στο 0. Δηλαδή δεν μπορεί να ανοίξει στο ίδιο σημείο εγκατάσταση άλλου τύπου. (περιορισμός 3.1, υποενότητα 3.1.2)

(B) αν μια μεταβλητή y[k,i] έχει «κλειδώσει» στο 1, δηλαδή ο συγκεκριμένος πελάτης k εξυπηρετείται από την εγκατάσταση στο σημείο i, τότε όλες οι υπόλοιπες μεταβλητές y[k,i'] ($\forall i' \neq i$) για αυτόν τον πελάτη τίθενται στο 0. Έτσι, εξασφαλίζεται ότι κάθε πελάτης μπορεί να εξυπηρετηθεί μόνο από μια εγκατάσταση. (περιορισμός 3.4, υποενότητα 3.1.2)

(C) αν μια μεταβλητή x[i,j] έχει «κλειδώσει» στο 0 και όλες οι μεταβλητές x[i,*] είναι 0 για αυτό το σημείο i, τότε δεν μπορεί να ανοίξει κανένας τύπος εγκατάστασης στο σημείο αυτό. Το σημείο i θεωρείται κλειστή εγκατάσταση οπότε όλες οι μεταβλητές ανάθεσης y[k,i] που σχετίζονται με αυτήν την εγκατάσταση τίθενται στο 0. (περιορισμός 3.5, υποενότητα 3.1.2)

3.5.7 Βοηθητικές Συναρτήσεις Ευρετικών και Μεθευρετικών Αλγορίθμων

Για την υλοποίηση των ευρετικών και μεθευρετικών αλγορίθμων αναπτύχθηκε ένα σύνολο βοηθητικών συναρτήσεων, οι οποίες επιτελούν σημαντικές λειτουργίες για την επίτευξη εφικτών λύσεων. Ακόμη, οι συναρτήσεις αυτές δεν αποτελούν ανεξάρτητους αλγορίθμους, αλλά βοηθητικοί μηχανισμοί που συμβάλλουν στη σωστή και αποδοτική εκτέλεση των κύριων διαδικασιών. Έτσι, η πολυπλοκότητα αυτών των διαδικασιών διατηρείται σε οργανωμένα και διαχειρίσιμα μέρη.

Οι παρακάτω βοηθητικές συναρτήσεις αξιοποιούνται από τους ευρετικούς αλγορίθμους για την διευκόλυνση της διαδικασίας τους. Αρχικά, η συνάρτηση _allowed_types επιστρέφει τους επιτρεπτούς τύπους εγκατάστασης για το σημείο που θα δοθεί

σαν όρισμα. Η συνάρτηση build_xvec μετατρέπει την λύση σε μονοδιάστατο διάνυσμα για ευκολότερη διαχείριση. Η συνάρτηση _evaluate υπολογίζει την τιμή της αντικειμενικής συνάρτησης (συνολική απόσταση) καθώς και το συνολικό κόστος (κόστος εγκατάσταση, λειτουργίας, μεταφοράς). Η συνάρτηση _choose_min_type_for_load επιλέγει τον μικρότερο επιτρεπτό τύπο εγκατάστασης, του οποίου η χωρητικότητα καλύπτει το συνολικό φορτίο ζήτησης που δίνεται ως όρισμα. Η συνάρτηση _scarcity_order ταξινομεί τους πελάτες με βάση τη δυσκολία εξυπηρέτησης. Συγκεκριμένα, πρώτα τοποθετούνται οι πελάτες με λιγότερες επιλογές εγκατάστασης, ενώ σε περίπτωση ισοπαλίας προηγούνται οι πελάτες με μεγαλύτερη ζήτηση. Η συνάρτηση _assign_greedy αναθέτει πελάτες σε εγκαταστάσεις με άπληστο τρόπο. Η συνάρτηση αυτή, σε κάθε βήμα της παίρνει την τοπικά καλύτερη απόφαση χωρίς να εξετάζει τις συνέπειες αυτής της επιλογής. Ειδικότερα, κάθε πελάτης εξυπηρετείται από την πιο κοντινή διαθέσιμη εγκατάσταση που έχει επαρκή χωρητικότητα. Παρόμοια, η συνάρτηση _greedy_open_sites_cover επιλέγει άπληστα εγκαταστάσεις. Κάθε εγκατάσταση προσπαθεί να καλύψει όσο το δυνατόν περισσότερους μη εξυπηρετημένους πελάτες, λαμβάνοντας υπόψη τη ζήτηση και την απόσταση. Συγκεκριμένα, γίνεται χρήση βαρών που διαφοροποιούνται ανά ευρετικό αλγόριθμο. Η διαδικασία επαναλαμβάνεται μέχρι να συμπληρωθεί ο απαιτούμενος αριθμός εγκαταστάσεων P ή να μην υπάρχουν εφικτές επιλογές. Η συνάρτηση _swap_repair αποτελεί μηχανισμό επιδιόρθωσης που προσπαθεί να διορθώσει μη εφικτές λύσεις ή να καλύψει πελάτες που παραμένουν χωρίς εγκατάσταση. Στην πράξη, προσπαθεί να αντικαταστήσει τις ήδη ανοιχτές εγκαταστάσεις με νέες υποψήφιες, έτσι ώστε να εξασφαλιστεί πλήρης κάλυψη. Η επιλογή της εγκατάστασης που θα αφαιρεθεί γίνεται δίνοντας προτεραιότητα σε εκείνες που εξυπηρετούν τον μικρότερο αριθμό πελατών. Σε περίπτωση ισοπαλίας, επιλέγονται οι εγκαταστάσεις που καλύπτουν πιο απομακρυσμένους πελάτες. Τέλος, η συνάρτηση _empty_sol επιστρέφει κενή λύση σε περίπτωση που αποτύχει κάποιος ευρετικός αλγόριθμος.

Σε αντίθεση με τις βοηθητικές συναρτήσεις των κατασκευαστικών ευρετικών αλγορίθμων, η βοηθητική συνάρτηση decode_repair_from_open_sites χρησιμοποιείται από τους μεθευρετικούς αλγορίθμους. Αυτή, λαμβάνει ως είσοδο ένα σύνολο P ανοικτών εγκαταστάσεων και κατασκευάζει μια πλήρη εφικτή λύση. Αρχικά, με δεδομένο το P σύνολο εγκαταστάσεων, επιχειρεί άπληστη ανάθεση πελατών λαμ-

βάνοντας τη μέγιστη επιτρεπτή χωρητικότητα για κάθε εγκατάσταση. Αν αυτή η ανάθεση αποτύχει, εφαρμόζεται ο μηχανισμός επιδιόρθωσης ώστε να αντικαταστήσει εγκαταστάσεις μέχρι να γίνει εφικτή η ανάθεση. Στη συνέχεια, επιλέγεται για κάθε εγκατάσταση ο ελάχιστος επιτρεπτός τύπος που επαρκεί για την ζήτηση των πελατών που εξυπηρετεί (ώστε να μειωθεί το κόστος εγκατάστασης). Έπειτα, υπολογίζει τη συνολική απόσταση και το συνολικό κόστος και εκτελεί έλεγχο στον περιορισμό προϋπολογισμού. Τέλος, αν όλα ικανοποιούνται, επιστρέφει ένα ολοκληρωμένο λεξικό λύσης, αλλιώς επιστρέφει κενή λύση.

3.5.8 Ευρετικοί Αλγόριθμοι

Πριν την εφαρμογή του αλγορίθμου Branch and Bound, πραγματοποιούνται διάφοροι ευρετικοί αλγόριθμοι, οι οποίοι έχουν ως στόχο να εντοπίσουν μία αρχική εφικτή λύση. Αυτή η προσέγγιση λειτουργεί ως αρχικό ανώτατο όριο για το πρόβλημα ελαχιστοποίησης. Με αυτόν το τρόπο πετυχαίνουμε αποτελεσματική αποκοπή των μη υποσχόμενων κόμβων καθώς και ταχύτερη σύγκλιση προς την καλύτερη δυνατή λύση. Οι ευρετικοί αλγόριθμοι αποσκοπούν στη δημιουργία εφικτών λύσεων σε λογικό χρόνο υπολογισμού, χωρίς να εξασφαλίζουν όμως την ανεύρεση της βέλτιστης λύσης.

Οι ευρετικοί αλγόριθμοι που αναπτύχθηκαν ανήκουν στην κατηγορία των κατασκευαστικών άπληστων αλγορίθμων (constructive greedy). Κατασκευαστικοί διότι χτίζουν σταδιακά την λύση, δημιουργώντας την από το μηδέν. Άπληστοι γιατί σε κάθε βήμα επιλέγεται η καλύτερη επιλογή βάσει κάποιου κριτηρίου, χωρίς να εξετάζεται η συνολική βέλτιστη λύση. Να σημειωθεί πως ο τέταρτος ευρετικός αλγόριθμος ενσωματώνει στοχαστική προσέγγιση. Δηλαδή, σε κάθε βήμα επιλέγει τυχαία ανάμεσα στις καλύτερες διαθέσιμες επιλογές. Αυτή, η στοχαστική προσέγγιση ενισχύεται ακόμη περισσότερο μέσω πολλαπλών εκκινήσεων (multistart), όπου ο ευρετικός εκτελείται επανειλημμένα από διαφορετικές τυχαίες αρχικές επιλογές. Στην πράξη, αυτό αυξάνει την ποικιλία των λύσεων και αποτρέπει τη παγίδευση σε τοπικά βέλτιστα. Τέλος, και οι τέσσερις ευρετικοί αλγόριθμοι εκτελούνται διαδοχικά και η λύση με τη μικρότερη συνολική απόσταση δίνεται ως αρχική λύση στους μεθευρετικούς αλγορίθμους για περαιτέρω βελτίωση. Στο Σχήμα 3.6 δίνεται η κοινή δομή που εμφανίζουν οι τέσσερις ευρετικοί αλγόριθμοι, η οποία διαφοροποιείται κυρίως

στο κριτήριο επιλογής των αρχικών εγκαταστάσεων.

Γενική Δομή Ευρετικών Αλγορίθμων

1. Αρχική επιλογή εγκαταστάσεων σύμφωνα με κάποιο κριτήριο (ζήτηση, κόστος μεταφοράς, μέγιστη χωρητικότητα × κάλυψη, ελκυστικότητα από 1/απόσταση), μέσω **_greedy_open_sites_cover**.
2. Ανάθεση πελατών στις επιλεγμένες εγκαταστάσεις με άπληστη προσέγγιση μέσω **_assign_greedy**.
3. Εφαρμογή επιδιόρθωσης σε περίπτωση μη εφικτής ανάθεσης ή μη καλυμμένων πελατών μέσω **_swap_repair**.
4. Επιλογή κατάλληλου τύπου εγκατάστασης για κάθε ανοικτή θέση ώστε να καλύπτεται το φορτίο (ζήτηση) μέσω **_choose_min_type_for_load.**
5. Αξιολόγηση λύσης μέσω **_evaluate** και δημιουργία μονοδιάστατου διανύσματος μέσω **_build_xvec.**
6. Έλεγχος του περιορισμού προϋπολογισμού.

Σχήμα 3.6: Κοινή δομή των ευρετικών αλγορίθμων.

Ευρετικός Αλγόριθμος Distance Greedy Cover

Για τον πρώτο ευρετικό αλγόριθμο εφαρμόζεται η γενική διάταξη που απεικονίζεται στο Σχήμα 3.6. Η βασική διαφορά του είναι στο κριτήριο επιλογής των αρχικών εγκαταστάσεων. Στην πράξη, για κάθε υποψήφια θέση υπολογίζει μια βαθμολογία ελκυστικότητας. Αυτή, είναι ανάλογη της ζήτησης των πελατών που δεν εξυπηρετούνται και αντιστρόφως ανάλογη της απόστασης τους από την εγκατάσταση. Με βάση αυτό, είναι προφανές ότι η βαθμολογία μιας τοποθεσίας αυξάνεται όταν είναι κοντά σε πολλούς πελάτες που έχουν αυξημένη ζήτηση. Έτσι, επιλέγονται οι θέσεις που ελαχιστοποιούν όσο περισσότερο μπορούν τη συνολική απόσταση για τους πιο "σημαντικούς" πελάτες. Σε κάθε επανάληψη, γίνεται η επιλογή των θέσεων με την υψηλότερη βαθμολογία μέχρι να πετύχουμε τον αριθμό των P εγκαταστάσεων. Ακολουθεί στη συνέχεια, η ανάθεση των πελατών στις κοντινότερες εγκαταστάσεις που μπορούν να καλύψουν πλήρως τις ανάγκες τους σε χωρητικότητα. Ταυτόχρονα, δίνεται προτεραιότητα σε αυτούς που διαθέτουν λιγότερες επιλογές ή έχουν μεγαλύτερη ανάγκη ζήτησης. Σε περίπτωση που αποτύχει η ανάθεση εφαρμόζεται ο μηχανισμός επιδιόρθωσης που εκτελεί ανταλλαγές εγκαταστάσεων, προκειμένου να καλυφθούν όλοι οι πελάτες. Έπειτα, γίνεται επιλογή ελάχιστου επιτρεπτού τύπου εγκατάστασης για κάθε ανοικτή θέση προκειμένου η χωρητικότητα να καλύπτει το συνολικό φορτίο ζήτησης. Μετά, υπολογίζεται η τιμή της αντικειμενικής συνάρτησης

(συνολική απόσταση) καθώς και το συνολικό κόστος. Τέλος, ελέγχεται αν πληρείται ο περιορισμός προϋπολογισμού και κατασκευάζεται το μονοδιάστατο διάνυσμα της λύσης. Στόχος του αλγορίθμου είναι να παράγει γρήγορα μια εφικτή αρχική λύση που ελαχιστοποιεί τη συνολική απόσταση, καλύπτει όλους τους πελάτες και παραμένει εντός προϋπολογισμού. Στον Αλγόριθμο 6 και 7 δίνεται ο ψευδοκώδικας του Distance Greedy Cover.

Αλγόριθμος 6: Ευρετικός Αλγόριθμος: Distance Greedy Cover Μέρος 1ο

Είσοδος: P, N, E, CL,cl_sp,$maxd$,$demand$, $facility_types$,$capacities$, $cost_types$,C_{max}, $transport_mode$, $combined_cost_per_km$

Έξοδος: $feasible$, $distance_obj$, $total_cost$, $open_sites$, $open_types$, $assign$, $xvec$

```
dist_feasible ← (cl_sp ≤ maxd[:, None]);
(open_sites, uncovered) ←
 _greedy_open_sites_cover(P, dist_feasible, cl_sp, facility_types,
E, weights = demand)
if |open_sites| ≠ P then
  | Return _empty_sol()
end
if uncovered ≠ ∅ then
  | (open_sites, assign, ok) ←
  |  _swap_repair(open_sites, dist_feasible, cl_sp, demand, facility_types,
  | capacities, E, P) if ¬ok then
  |   | Return _empty_sol()
  | end
end
else
  | max_cap ← 0^N ;
  | foreach i ∈ open_sites do
  |   | j^max ← min(facility_types[i], E − 1);
  |   | max_cap[i] ← capacities[j^max];
  | end
  | (assign, ok) ←
  |  _assign_greedy(open_sites, max_cap, dist_feasible, cl_sp, demand)
  | if ¬ok then
  |   | (open_sites, assign, ok) ←
  |   |  _swap_repair(open_sites, dist_feasible, cl_sp, demand, facility_types,
  |   | capacities, E, P)
  |   | if ¬ok then
  |   |   | Return _empty_sol()
  |   | end
  | end
end
```

Αλγόριθμος 7: Ευρετικός Αλγόριθμος: Distance Greedy Cover Μέρος 2ο

```
load[i] ← 0; ∀i ∈ open_sites
for k = 0 to CL − 1 do
    i ← assign[k];
    load[i] ← load[i] + demand[k];
end
open_types ← {};
foreach i ∈ open_sites do
    j* ← _choose_min_type_for_load(i, load[i], facility_types, capacities, E)
    if j* = None then
        Return _empty_sol()
    end
    open_types[i] ← j*;
end
(distance_obj, total_cost) ←
 _evaluate(open_types, assign, cl_sp, transport_mode, combined_cost_per_km,
cost_types)
if total_cost > C_max + 10^−9 then
    Return _empty_sol()
end
xvec ← _build_xvec(open_types, assign, N, E, CL)
Return feasible, distance_obj, total_cost, open_sites, open_types, assign, xvec
```

Ευρετικός Αλγόριθμος Cost-aware Greedy

Ο δεύτερος ευρετικός αλγόριθμος ακολουθεί επίσης τη γενική δομή του Σχήματος 3.6 αλλά διαφοροποιείται στο κριτήριο επιλογής των αρχικών εγκαταστάσεων. Πιο συγκεκριμένα, εστιάζει στη μείωση του κόστους μεταφοράς αντί της συνολικής απόστασης. Αρχικά, για κάθε πελάτη υπολογίζεται ένα βάρος ίσο με το κόστος μεταφοράς ανά χιλιόμετρο του μέσου που χρησιμοποιεί (συνδυασμός οικονομικού κόστους και κόστους CO_2). Ακόμη ελέγχουμε ποιες θέσεις είναι εφικτές για τον καθένα με βάση τη μέγιστη αποδεκτή απόσταση διαδρομής. Έπειτα, για κάθε υποψήφια θέση υπολογίζεται ένα σκορ ελκυστικότητας, το οποίο προκύπτει από τη συνεισφορά των μη καλυμμένων πελατών. Η συνεισφορά κάθε πελάτη είναι ανάλογη με το κόστος μεταφοράς ανά χιλιόμετρο του μέσου που χρησιμοποιεί και αντιστρόφως ανάλογη με την απόσταση από την εγκατάσταση. Το σκορ μιας υποψήφιας θέσης αυξάνεται όταν είναι κοντά σε πελάτες με υψηλό κόστος μεταφοράς ανά χιλιόμετρο, ώστε να μειώνεται το συνολικό κόστος μετακίνησης. Σε κάθε βήμα, ανοίγουμε τη θέση με το μεγαλύτερο σκορ μέχρι να ανοίξουν P θέσεις. Στη συνέχεια ακολουθεί η ίδια διαδι-

κασία. Δηλαδή, άπληστη ανάθεση πελατών με προτεραιότητα σε πελάτες που έχουν λιγότερες εφικτές επιλογές (σε ισοβαθμία, υψηλότερη ζήτηση), μηχανισμός επιδιόρθωσης αν χρειαστεί, επιλογή ελάχιστου επιτρεπτού τύπου εγκατάστασης με επαρκή χωρητικότητα, υπολογισμός της αντικειμενικής συνάρτησης και του συνολικού κόστους, έλεγχος του περιορισμού προϋπολογισμού καθώς και τελική κατασκευή του μονοδιάστατου διανύσματος λύσης. Στον Αλγόριθμο 8 δίνεται ο ψευδοκώδικας του Cost-aware Greedy.

Ευρετικός Αλγόριθμος Feasibility-First

Για τον τρίτο ευρετικό αλγόριθμο εφαρμόζεται μια διαφορετική τεχνική σε σχέση με τους προηγούμενους. Δηλαδή, δεν στηρίζεται στην άπληστη επαναληπτική επιλογή τοποθεσιών. Αντίθετα, δίνει έμφαση στην άμεση λήψη αποφάσεων μέσω της βοήθειας μιας βαθμολογίας εφικτότητας. Συγκεκριμένα, υπολογίζει για κάθε υποψήφια εγκατάσταση μια τιμή η οποία μας δείχνει πόσο αποτελεσματικά ικανή είναι να εξυπηρετήσεις τους πελάτες. Η τιμή αυτή, προκύπτει από το γινόμενο της μέγιστης αποδεκτής χωρητικότητας (δηλαδή τη χωρητικότητα του μεγαλύτερου επιτρεπτού τύπου εγκατάστασης) με το πλήθος των πελατών που έχουν τη δυνατότητα να προσεγγίσουν αυτήν την τοποθεσία εντός της μέγιστης ανεκτής απόστασής τους. Προφανώς και οι θέσεις που διαθέτουν την μεγαλύτερη χωρητικότητα και ευρύτερη κάλυψη αποκτούν κύρια προτεραιότητα. Ακόμη, γίνεται η κατάταξη τους κατά φθίνουσα σειρά βαθμολογίας και επιλέγονται τελικά οι P καλύτερες από αυτές. Ακολουθεί η διαδικασία διόρθωσης που διασφαλίζει ότι όλοι οι πελάτες καλύπτονται. Στη συνέχεια, επιλέγεται ο μικρότερος τύπος εγκατάστασης που πληροί τις απαιτήσεις χωρητικότητας. Στο τελευταίο στάδιο, υπολογίζουμε τη συνάρτηση στόχου και το συνολικό κόστος, ενώ ελέγχουμε παράλληλα ότι δεν παραβιάζεται ο περιορισμός του προϋπολογισμού. Με βάση τις παραπάνω διαδικασίες ολοκληρώνουμε την κατασκευή του μονοδιάστατου διανύσματος λύσης. Στον Αλγόριθμο 9 δίνεται ο ψευδοκώδικας του Feasibility-First.

Ευρετικός Αλγόριθμος GRASP Multistart

Ο τέταρτος ευρετικός αλγόριθμος ενσωματώνει μια στοχαστική προσέγγιση πολλαπλών εκκινήσεων που στηρίζεται στη μεθοδολογία GRASP (Greedy Randomized

Αλγόριθμος 8: Ευρετικός Αλγόριθμος: Cost-Aware Greedy

Είσοδος: P, N, E, CL,cl_sp,$maxd$,$demand$, $facility_types$,$capacities$, $cost_types$,C_{max}, $transport_mode$, $combined_cost_per_km$

΄Εξοδος: $feasible$, $distance_obj$, $total_cost$, $open_sites$, $open_types$, $assign$, $xvec$

```
dist_feasible ← (cl_sp ≤ maxd[:, None]);
weights[k] ← combined_cost_per_km[transport_mode[k]]; ∀k ∈ {0, ..., CL − 1}
(open_sites, _) ←
  _greedy_open_sites_cover(P, dist_feasible, cl_sp, facility_types, E, weights)
if |open_sites| ≠ P then
  | Return _empty_sol()
end
(open_sites, assign, ok) ←
  _swap_repair(open_sites, dist_feasible, cl_sp, demand, facility_types,
capacities, E, P)
if ¬ok then
  | Return _empty_sol()
end
load[i] ← 0; ∀i ∈ open_sites
for k = 0 to CL − 1 do
  | i ← assign[k];
  | load[i] ← load[i] + demand[k];
end
open_types ← {};
foreach i ∈ open_sites do
  | j* ← _choose_min_type_for_load(i, load[i], facility_types, capacities, E)
  | if j* = None then
  |   | Return _empty_sol()
  | end
  | open_types[i] ← j*;
end
(distance_obj, total_cost) ←
  _evaluate(open_types, assign, cl_sp, transport_mode, combined_cost_per_km,
cost_types)
if total_cost > C_max + 10^−9 then
  | Return _empty_sol()
end
xvec ← _build_xvec(open_types, assign, N, E, CL)
Return feasible, distance_obj, total_cost, open_sites, open_types, assign, xvec
```

Αλγόριθμος 9: Ευρετικός Αλγόριθμος: Feasibility-First

Είσοδος: P, N, E, CL,cl_sp,$maxd$,$demand$, $facility_types$,$capacities$, $cost_types$,C_{max}, $transport_mode$, $combined_cost_per_km$

Έξοδος: $feasible$, $distance_obj$, $total_cost$, $open_sites$, $open_types$, $assign$, $xvec$

```
dist_feasible ← (cl_sp ≤ maxd[:, None]);
site_list ← [];
for i = 0 to N − 1 do
    j^max ← min(facility_types[i], E − 1);
    if j^max < 0 then
        continue
    end
    cover ← Σ_{k=0}^{CL−1} dist_feasible[k, i];
    if cover = 0 then
        continue
    end
    max_cap ← capacities[j^max];
    score ← −(max_cap × cover) site_list.append((score, i));
end
site_list.sort();
open_sites ← { i | (_, i) ∈ site_list[: P] };
if |open_sites| ≠ P then
    Return _empty_sol()
end
(open_sites, assign, ok) ← _swap_repair(open_sites, dist_feasible, cl_sp,
demand, facility_types, capacities, E, P)
if ¬ok then
    Return _empty_sol()
end
load[i] ← 0; ∀i ∈ open_sites
for k = 0 to CL − 1 do
    i ← assign[k];
    load[i] ← load[i] + demand[k];
end
open_types ← {};
foreach i ∈ open_sites do
    j* ← _choose_min_type_for_load(i, load[i], facility_types, capacities, E)
    if j* = None then
        Return _empty_sol()
    end
    open_types[i] ← j*;
end
(distance_obj, total_cost) ←
 _evaluate(open_types, assign, cl_sp, transport_mode,
combined_cost_per_km, cost_types)
if total_cost > C_max + 10^−9 then
    Return _empty_sol()
end
xvec ← _build_xvec(open_types, assign, N, E, CL)
Return feasible, distance_obj, total_cost, open_sites, open_types, assign, xvec
```

Adaptive Search Procedure). Αυτή συγκεκριμένα, συνδυάζει άπληστη επιλογή με ελεγχόμενη τυχαιότητα προκειμένου να αποφύγει τα τοπικά βέλτιστα. Στην πράξη, για κάθε μία υποψήφια θέση υπολογίζει ένα σκορ ελκυστικότητας, το οποίο είναι αντιστρόφως ανάλογο των αποστάσεων των πελατών που μπορεί να εξυπηρετήσει. Με αυτόν τον τρόπο βέβαια, το σκορ μιας θέσης αυξάνεται όταν καλύπτει πολλούς πελάτες σε κοντινές αποστάσεις. Αυτό συμβαίνει διότι οι μικρότερες αποστάσεις συνεισφέρουν καλύτερα στο τελικό άθροισμα. Στη συνέχεια, οι υποψήφιες θέσεις κατατάσσονται κατά φθίνουσα σειρά με βάση το σκορ. Οι καλύτερες από αυτές τις θέσεις τοποθετούνται σε μια Περιορισμένη Λίστα Υποψηφίων (Restricted Candidate List, RCL). Πιο συγκεκριμένα, το πλήθος αυτών που θα εισαχθούν καθορίζεται από τον χρήστη. Έπειτα, ο αλγόριθμος πραγματοποιεί πολλές επαναλήψεις, όπου σε κάθε βήμα επιλέγει τυχαία P εγκαταστάσεις από την Περιορισμένη Λίστα Υποψηφίων. Ακολούθως, για κάθε τυχαία επιλεγμένο σύνολο εγκαταστάσεων εφαρμόζεται ο μηχανισμός επιδιόρθωσης για την εξασφάλιση εφικτής ανάθεσης όλων των πελατών, η επιλογή του ελάχιστου επιτρεπτού τύπου ανά θέση και ο υπολογισμός της αντικειμενική συνάρτησης και του συνολικού κόστους. Τέλος, αν η λύση περάσει και τον έλεγχο του περιορισμού προϋπολογισμού, συγκρίνεται με την καλύτερη λύση που έχει βρεθεί στις επαναλήψεις. Αν αυτή είναι μικρότερη, ενημερώνεται ως νέα καλύτερη λύση. Αν σε οποιοδήποτε στάδιο (επιδιόρθωση, επιλογή τύπων ή έλεγχος προϋπολογισμού) υπάρξει αποτυχία, η επανάληψη παραλείπεται και ο αλγόριθμος συνεχίζει στην επόμενη. Μετά την ολοκλήρωση όλων των επαναλήψεων, επιστρέφεται η καλύτερη εφικτή λύση που βρέθηκε. Στον Αλγόριθμο 10 και 11 δίνεται ο ψευδοκώδικας του GRASP Multistart.

3.5.9 Μεθευρετικοί Αλγόριθμοι

Ο αλγόριθμος Branch and Bound αποτελεί ακριβής μέθοδος βελτιστοποίησης, αφού εγγυάται την εύρεση βέλτιστης λύσης μέσω συστηματικής εξερεύνησης του χώρου λύσεων. Παρόλα αυτά, στο πρόβλημα μας ο χώρος αναζήτησης αυξάνεται εκθετικά λόγω της συνδυαστικής φύσης του προβλήματος και των ποικίλων περιορισμών, καθιστώντας τον υπολογιστικά δαπανηρό. Για αυτόν τον λόγο, αναπτύσσουμε μεθευρετικούς αλγορίθμους, οι οποίοι αποτελούν ευέλικτες στρατηγικές βελτιστοποίησης. Στόχος τους είναι να βρουν καλές λύσεις σε λογικό χρόνο, χωρίς όμως να

Αλγόριθμος 10: Ευρετικός Αλγόριθμος: GRASP Multistart Μέρος 1ο

Είσοδος: P, N, E, CL, cl_sp, $maxd$, $demand$, $facility_types$, $capacities$, $cost_types$, C_{max}, $transport_mode$, $combined_cost_per_km$, $iters$, rcl_size, $seed$

Έξοδος: $feasible$, $distance_obj$, $total_cost$, $open_sites$, $open_types$, $assign$, $xvec$

```
rng ← RandomGenerator(seed);
dist_feasible ← (cl_sp ≤ maxd[:, None]);
base_score ← [−∞]^N;
for i = 0 to N − 1 do
    cover ← dist_feasible[:, i];
    if ∃k : cover[k] = True then
        base_score[i] ← Σ_{k:cover[k]} 1/(cl_sp[k,i]+10^−6);
    end
end
cand_order ← argsort(−base_score);
best ← _empty_sol()

for iter = 1 to iters do
    top ← cand_order[: min(rcl_size, N)];
    chosen ← ∅;
    open_sites ← [];
    while |open_sites| < P do
        i ← rng.choice(top);
        if i ∈ chosen then
            continue
        end
        if |_allowed_types(i, facility_types, E)| = 0 then
            continue
        end
        chosen ← chosen ∪ {i};
        open_sites.append(i);
    end
    (open_sites, assign, ok) ←
     _swap_repair(open_sites, dist_feasible, cl_sp, demand, facility_types,
    capacities, E, P)
    if ¬ok then
        continue
    end
end
```

Αλγόριθμος 11: Ευρετικός Αλγόριθμος: GRASP Multistart Μέρος 2ο

```
load[i] ← 0;  ∀i ∈ open_sites
for k = 0 to CL − 1 do
    i ← assign[k];
    load[i] ← load[i] + demand[k];
end
open_types ← {};
ok_types ← True;
foreach i ∈ open_sites do
    j* ← _choose_min_type_for_load(i, load[i], facility_types, capacities, E)
    if j* = None then
        ok_types ← False;
        break
    end
    open_types[i] ← j*;
end
if ¬ok_types then
    continue
end
(distance_obj, total_cost) ←
 _evaluate(open_types, assign, cl_sp, transport_mode, combined_cost_per_km,
cost_types)
if total_cost > C_max + 10^−9 then
    continue
end
if (¬best["feasible"]) ∨ (distance_obj < best["distance_obj"] − 10^−9) then
    xvec ← _build_xvec(open_types, assign, N, E, CL)
    best["feasible"] ← True;
    best["distance_obj"] ← distance_obj;
    best["total_cost"] ← total_cost;
    best["open_sites"] ← open_sites;
    best["open_types"] ← open_types;
    best["assign"] ← assign;
    best["xvec"] ← xvec;
end
Return best
```

εγγυώνται την βέλτιστη λύση. Συγκεκριμένα, στο πρόβλημα μας χρησιμοποιούνται για τη βελτίωση της λύσης που προκύπτει από τους κατασκευαστικούς αλγορίθμους που περιγράφηκαν, η οποία στη συνέχεια χρησιμοποιείται ως αρχικό άνω φράγμα στον ακριβή αλγόριθμο Branch and Bound. Με αυτόν τον τρόπο μειώνεται δραστικά ο χώρος αναζήτησης και επιταχύνεται η σύγκλιση στην βέλτιστη λύση.

Για την παρούσα εργασία, αξιολογούνται τρεις βασικές κατηγορίες μεθευρετικών αλγορίθμων. Ειδικότερα, η πρώτη είναι η τοπική αναζήτηση (Variable Neighborhood Descent), που επιτυγχάνει σταδιακή βελτίωση των αναθέσεων στους πελάτες, ενισχύοντας με αυτόν τον τρόπο τη συστηματική πρόοδο στην διαδικασία. Η δεύτερη κατηγορία αποτελεί τις στοχαστικές μεθόδους (Simulated Annealing), οι οποίες επιτρέπουν την αποδοχή λιγότερο αποτελεσματικών λύσεων προκειμένου να αποφευχθούν τα τοπικά βέλτιστα, δίνοντας έτσι έμφαση στην εξερεύνηση. Η τρίτη κατηγορία περιλαμβάνει μεθόδους που βασίζονται σε διαταραχή (Iterated Local Search), οι οποίες ενσωματώνουν τυχαίες διαταραχές και επαναλαμβάνουν τη διαδικασία τοπικής αναζήτησης. Στο τέλος, θα πρέπει να αναφερθεί ότι εκτελούνται διαδοχικά οι μεθευρετικοί αλγόριθμοι και η λύση που έχει τη μικρότερη συνολική απόσταση επιλέγεται ως το αρχικό ανώτερο όριο για τον αλγόριθμο Branch and Bound.

Μεθευρετικός Αλγόριθμος Variable Neighborhood Descent

Ο πρώτος μεθευρετικός αλγόριθμος εκτελεί τοπική αναζήτηση τύπου Variable Neighborhood Descent στις αναθέσεις πελατών, διατηρώντας σταθερό το σύνολο ανοικτών εγκαταστάσεων P. Η διαδικασία ξεκινά μόνο όταν υπάρχει αρχική εφικτή λύση. Προϋπολογίζει για κάθε πελάτη το κόστος ανά χιλιόμετρο με βάση το μέσο μεταφοράς του. Επίσης, για κάθε πελάτη δημιουργεί τη λίστα των εφικτών του εγκαταστάσεων με βάση τη μέγιστη αποδεκτή απόσταση, ταξινομημένη σε αύξουσα σειρά απόστασης. Υπολογίζει για κάθε εγκατάσταση το συνολικό φορτίο ζήτησης των πελατών που της έχουν ανατεθεί στην αρχική λύση καθώς και τους ελάχιστους επιτρεπτούς τύπους που χωρούν αυτά τα φορτία. Στον βασικό βρόχο του αλγορίθμου, επιλέγονται πελάτες με τυχαία σειρά και για καθέναν από αυτούς δοκιμάζονται μετακινήσεις προς πιο κοντινές ανοικτές εγκαταστάσεις. Η διαδικασία αυτή σταματά πρόωρα όταν η νέα απόσταση δεν είναι καλύτερη ή όταν ξεπεραστεί το όριο δοκιμών ανά πελάτη. Για κάθε υποψήφια μετακίνηση, υπολογίζονται τα νέα

φορτία ζήτησης στις δύο εγκαταστάσεις καθώς και οι ελάχιστοι επιτρεπτοί τύποι που καλύπτουν αυτά τα φορτία. Ακόμη, για κάθε υποψήφια μετακίνηση προσδιορίζονται οι μεταβολές στο κόστος εγκατάστασης και μεταφοράς. Προκειμένου, να γίνει αποδεκτή μια μετακίνηση θα πρέπει να βελτιώνει την απόσταση του πελάτη χωρίς να υπερβαίνει τον προϋπολογισμό. Με τη στρατηγική First Improvement, η αναζήτηση αποδέχεται την πρώτη διαθέσιμη βελτίωση και την εφαρμόζει άμεσα, χωρίς να εξετάζει όλες τις πιθανές μετακινήσεις. Μόλις βρεθεί μια εφικτή μετακίνηση που να μειώνει την τιμή της αντικειμενικής συνάρτησης, ενημερώνονται σταδιακά (χωρίς πλήρη επανυπολογισμό) οι αναθέσεις, τα φορτία ζήτησης, οι ελάχιστοι επιτρεπτοί τύποι και το συνολικό κόστος. Η διαδικασία του βασικού βρόχου επαναλαμβάνεται από την αρχή μέχρι να μην υπάρχει άλλη βελτίωση, καταλήγοντας σε ένα τοπικό βέλτιστο, δηλαδή σε μια κατάσταση όπου κανένας πελάτης δεν μπορεί να μετακινηθεί σε άλλη εγκατάσταση ώστε να μειωθεί περισσότερο η συνολική απόσταση. Τέλος, επιστρέφεται ένα λεξικό που περιέχει την πλήρη βελτιωμένη λύση. Στον Αλγόριθμο 12 , 13 και 14 δίνεται ο ψευδοκώδικας του Variable Neighborhood Descent.

Μεθευρετικός Simulated Annealing

Ο δεύτερος μεθευρετικός αλγόριθμος εφαρμόζει τη μέθοδο Simulated Annealing σε γειτονικές εγκαταστάσεις. Συγκεκριμένα, κάθε γειτονική λύση δημιουργείται μέσω της ανταλλαγής μιας εγκατάστασης που κλείνει με μία που ανοίγει. Στην πράξη, η αποδοχή μιας γειτονικής λύσης γίνεται σύμφωνα με τη δεδομένη θερμοκρασία και την αλλαγή που επιφέρει αυτή στη συνολική απόσταση. Η συγκεκριμένη διαδικασία εκκινεί μόνο όταν υπάρχει αρχική εφικτή λύση. Σε κάθε επανάληψη, επιλέγεται τυχαία μια εγκατάσταση από τις ήδη ανοιχτές της τρέχουσας λύσης για να αφαιρεθεί καθώς και μία επιτρεπτή εγκατάσταση από αυτές που είναι κλειστές για να ανοίξει (διατηρείται σταθερός ο αριθμός P των ανοιχτών εγκαταστάσεων). Για το νέο σύνολο εγκαταστάσεων που προκύπτει χρησιμοποιείται η βοηθητική συνάρτηση decode_repair_from_open_sites για την κατασκευή πλήρους λύσης. Μετά την εφαρμογή εάν η λύση που προκύπτει δεν είναι εφικτή, απορρίπτεται και συνεχίζει στην επόμενη δοκιμή. Από την άλλη, εάν είναι εφικτή, υπολογίζει τη διαφορά (Δ) μεταξύ της συνολικής απόστασης της γειτονικής λύσης και της τρέχουσας λύσης. Αν

Αλγόριθμος 12: Μεθευρετικός Αλγόριθμος: Variable Neighborhood Descent (VND) Μέρος 1ο

Είσοδος: sol_init, P, N, E, CL, cl_sp, $maxd$, $demand$, $facility_types$, $capacities$, $cost_types$, C_{max}, $transport_mode$, $combined_cost_per_km$, max_iters, $top_scan_per_client$, $first_improvement$, $seed$

΄Εξοδος: $feasible$, $distance_obj$, $total_cost$, $open_sites$, $open_types$, $assign$, $xvec$

$rng \leftarrow RandomGenerator(seed)$;
if $sol_init = \texttt{None} \vee \neg sol_init.feasible$ **then**
  Return $_empty_sol()$
end

$open_sites \leftarrow sol_init.open_sites$;
$open_set \leftarrow \{i \mid i \in open_sites\}$;
$assign \leftarrow sol_init.assign$.copy();
$mode_cost[k] \leftarrow combined_cost_per_km[transport_mode[k]];\ \forall k \in [0, CL-1]$;

$sorted_sites \leftarrow [\,]$
for $k = 0$ **to** $CL - 1$ **do**
  $cand \leftarrow \{i \in [0, N-1] : cl_sp[k][i] \leq maxd[k]\}$;
  Sort $cand$ in ascending order according to $cl_sp[k][i]$;
  $sorted_sites[k] \leftarrow cand$;
end

$load[i] \leftarrow 0;\ \forall i \in open_sites$
for $k = 0$ **to** $CL - 1$ **do**
  $i \leftarrow assign[k]$;
  $load[i] \leftarrow load[i] + demand[k]$;
end
$open_types \leftarrow \{\}$;
foreach $i \in open_sites$ **do**
  $j \leftarrow _choose_min_type_for_load(i, load[i], facility_types, capacities, E)$
  if $j = \texttt{None}$ **then**
    Return sol_init
  end
  $open_types[i] \leftarrow j$;
end

$(cur_dist, _) \leftarrow$
$_evaluate(open_types, assign, cl_sp, transport_mode, combined_cost_per_km, cost_types)$
$cur_transport \leftarrow \sum_{k=0}^{CL-1} cl_sp[k, assign[k]] \times mode_cost[k]$;
$cur_install \leftarrow \sum_{i \in open_sites} cost_types[open_types[i]]$;
$cur_cost \leftarrow cur_install + cur_transport$;

Αλγόριθμος 13: Μεθευρετικός Αλγόριθμος: VND (Μέρος 2ο)

```
it ← 0;   improved ← True;
while (improved = True) ∧ (it < max_iters) do
    it ← it + 1;   improved ← False;
    client_order ← Shuffle({0, ..., CL − 1}, rng);   best_delta ← 0;
    best_move ← None;
    foreach k ∈ client_order do
        i_old ← assign[k];   d_old ← cl_sp[k][i_old];   dk ← dem[k]; scanned ← 0;
        foreach i_new ∈ sorted_sites[k] do
            if i_new = i_old ∨ i_new ∉ open_set then
                continue
            end
            d_new ← cl_sp[k][i_new]; if d_new ≥ d_old then
                break
            end
            scanned ← scanned + 1; if scanned > top_scan_per_client then
                break
            end
            load_old^new ← load[i_old] − dk;   load_new^new ← load[i_new] + dk; if load_old^new < 0
             then
                continue
            end
            j_old^new ←
             _choose_min_type_for_load(i_old, load_old^new, facility_types, capacities, E);
             j_new^new ←
             _choose_min_type_for_load(i_new, load_new^new, facility_types, capacities, E);
             if j_old^new = None ∨ j_new^new = None then
                continue
            end
            ΔC_inst ← cost_types[j_old^new] − cost_types[open_types[i_old]] +
             cost_types[j_new^new] − cost_types[open_types[i_new]];
             ΔC_transp ← (d_new − d_old) · mode_cost[k];
             C_new ← cur_cost + ΔC_inst + ΔC_transp; if C_new > C_max then
                continue
            end
            Δd ← d_new − d_old; if Δd < best_delta then
                best_delta ← Δd;
                 best_move ← (k, i_new, j_old^new, j_new^new, ΔC_inst, ΔC_transp)
                if first_improvement = True then
                    break
                end
            end
        end
        if first_improvement = True ∧ best_move ≠ None then
            break
        end
    end
    if best_move = None then
        break
    end
    ApplyBestMoveAndUpdate(best_move, assign, load, open_types, cur_dist,
     cur_install, cur_transport, cur_cost, demand), improved ← True;
end
```

Αλγόριθμος 14: Μεθευρετικός Αλγόριθμος: VND (Μέρος 3ο) — Διαδικασία ενημέρωσης

Function `ApplyBestMoveAndUpdate`($best_move$, $assign$, $load$, $open_types$, cur_dist, $cur_install$, $cur_transport$, cur_cost, dem, $best_delta$):

$(k, i_{new}, j_{old}^{new}, j_{new}^{new}, \Delta C_{inst}, \Delta C_{transp}) \leftarrow best_move$
$i_{old} \leftarrow assign[k]$;
$dk \leftarrow dem[k]$;
$assign[k] \leftarrow i_{new}$;
$load[i_{old}] \leftarrow load[i_{old}] - dk$;
$load[i_{new}] \leftarrow load[i_{new}] + dk$;
$open_types[i_{old}] \leftarrow j_{old}^{new}$;
$open_types[i_{new}] \leftarrow j_{new}^{new}$;
$cur_dist \leftarrow cur_dist + best_delta$;
$cur_install \leftarrow cur_install + \Delta C_{inst}$;
$cur_transport \leftarrow cur_transport + \Delta C_{transp}$;
$cur_cost \leftarrow cur_install + cur_transport$;

$xvec \leftarrow _build_xvec(open_types, assign, N, E, CL)$
Return $feasible$, cur_dist, cur_cost, $open_sites$, $open_types$, $assign$, $xvec$

η διαφορά αυτή είναι μικρότερη ή ίση του μηδενός έχουμε βελτίωση και η ανταλλαγή γίνεται δεκτή, αλλιώς γίνεται αποδεκτή με πιθανότητα Metropolis. Συγκεκριμένα, ο κανόνας Metropolis P (accept) = $\exp\left(-\frac{\Delta}{T}\right)$ δείχνει πως η πιθανότητα αποδοχής μιας χειρότερης λύσης αυξάνεται όσο μεγαλύτερη είναι η θερμοκρασία T και μειώνεται όσο μεγαλύτερη είναι η αρνητική επίδραση Δ. Με τον τρόπο αυτό, επιτρέπει την εξερεύνηση του χώρου λύσεων και αποφεύγουμε τα τοπικά βέλτιστα. Παράλληλα, η θερμοκρασία μειώνεται εκθετικά με συντελεστή α: $T \leftarrow \alpha T$ (T0 αρχική θερμοκρασία), ώστε στην αρχή να αποδέχεται με μεγαλύτερη πιθανότητα μια χειρότερη λύση και στη συνέχεια αυτή η πιθανότητα μειώνεται. Να σημειωθεί πως σε όλη τη διαδικασία της αναζήτησης παρακολουθείται και ενημερώνεται η καλύτερη λύση που έχει βρεθεί και στο τέλος επιστρέφεται το καλύτερο σύνολο ανοικτών εγκαταστάσεων και οι αντίστοιχες αναθέσεις που πραγματοποιήθηκαν. Στον Αλγόριθμο 15 δίνεται ο ψευδοκώδικας του Simulated Annealing.

Μεθευρετικός Iterated Local Search

Ο τέταρτος μεθευρετικός αλγόριθμος συνδυάζει στοχαστική διαταραχή στο σύνολο των ανοικτών εγκαταστάσεων και μεθοδική τοπική αναζήτηση στις αναθέσεις των πελατών προκειμένου να αποφύγει τα τοπικά βέλτιστα. Η διαδικασία ξεκινά

Αλγόριθμος 15: Μεθευρετικός Αλγόριθμος: Simulated Annealing

Είσοδος: sol_init, P, N, E, CL, cl_sp, $maxd$, $demand$, $facility_types$, $capacities$, $cost_types$, C_{max}, $transport_mode$, $combined_cost_per_km$, $iters$, T_0, α, $candidate_pool$, $seed$

Έξοδος: improved solution

```
rng ← RandomGenerator(seed);
if sol_init = None ∨ ¬sol_init.feasible then
    Return _empty_sol()
end
cur ← sol_init; best ← sol_init; cur_sites ← cur.open_sites;
cur_set ← {i | i ∈ cur_sites};
openable_set ← {i ∈ [0, N − 1] : |_allowed_types(i, facility_types, E)| > 0};
T ← T_0; evals ← 0; accepts ← 0; improvements ← 0;
for iter = 1 to iters do
    closed_set ← openable_set − cur_set;
    if closed_set = ∅ then
        break
    end
    rem ← rng.RandomChoice(cur_sites);
    if |closed_set| ≤ candidate_pool then
        pool ← closed_set;
    else
        pool ← Randomly select candidate_pool elements from closed_set;
    end
    add ← rng.RandomChoice(pool);
    neigh_sites ← (cur_sites \ {rem}) ∪ {add};
    Remove duplicates from neigh_sites;
    if |neigh_sites| ≠ P then
        continue
    end
    neigh ← decode_repair_from_open_sites( neigh_sites, P, N, E, CL, cl_sp,
      maxd, demand facility_types, capacities, cost_types, C_max,
      transport_mode, combined_cost_per_km)
    if ¬neigh.feasible then
        continue
    end
    evals ← evals + 1;
    Δ ← neigh.distance_obj − cur.distance_obj;
    if Δ ≤ 0 or rng.random() < e^(−Δ/T) then
        cur ← neigh; cur_sites ← neigh.open_sites;
        cur_set ← {i | i ∈ cur_sites}; accepts ← accepts + 1;
        if cur.distance_obj < best.distance_obj then
            best ← cur; improvements ← improvements + 1;
        end
    else
    end
    T ← T × α;
end
Return best
```

μόνο όταν υπάρχει αρχική εφικτή λύση. Κάθε επανάληψη του μεθευρετικού αποτελείται από δύο φάσεις: τη φάση διαταραχής (perturbation) και τη φάση τοπικής βελτίωσης (local search). Σε κάθε εξωτερική επανάληψη, εφαρμόζεται η φάση της διαταραχής στο σύνολο των ανοικτών εγκαταστάσεων. Συγκεκριμένα, για την τρέχουσα λύση πραγματοποιείται συγκεκριμένος αριθμός ανταλλαγών εγκαταστάσεων. Δηλαδή, επιλέγεται τυχαία μια εγκατάσταση από τις ανοιχτές για να κλείσει και μια εγκατάσταση από τις κλειστές για να ανοίξει (σταθερός ο αριθμός P των ανοικτών εγκαταστάσεων). Μετά τη φάση της διαταραχής, εκτελείται η συνάρτηση decode_repair_from_open_sites στο νέο σύνολο ανοιχτών εγκαταστάσεων, έτσι ώστε να κατασκευαστεί μια πλήρης εφικτή λύση. Στη συνέχεια, σε αυτή τη λύση εφαρμόζεται τοπική αναζήτηση Variable Neighborhood Descent για να βελτιωθούν οι αναθέσεις των πελατών με τη διερεύνηση πολλαπλών γειτονιών. Η καλύτερη λύση που προκύπτει συγκρίνεται με την καλύτερη μέχρι στιγμής και αν είναι καλύτερη, την ενημερώνει. Τέλος, η διαδικασία επαναλαμβάνεται εώς ότου εξαντληθεί ο αριθμός εξωτερικών επαναλήψεων ή μέχρι να μην υπάρχει βελτίωση για ένα συγκεκριμένο αριθμό βημάτων (early stopping). Όταν ικανοποιηθεί μία από αυτές τις συνθήκες, επιστρέφεται η καλύτερη λύση που βρέθηκε. Στον αλγόριθμο 16 δίνεται ο ψευδοκώδικας του Iterated Local Search.

Συγκεντρωτικός Πίνακας Διαφορών Απλής και Βελτιωμένης Έκδοσης

Στον Πίνακα 3.3 παρουσιάζονται συγκεντρωτικά οι διαφορές ανάμεσα στην απλή και στην βελτιωμένη έκδοση του Branch and Bound.

Αλγόριθμος 16: Μεθευρετικός Αλγόριθμος: Iterated Local Search (ILS)

Είσοδος: *sol_init*, Problem parameters, *outer_iters*, *perturb_swaps*, *vnd_iters*, *patience*, *seed*

Έξοδος: improved solution

```
rng ← RandomGenerator(seed);
if sol_init = None ∨ ¬sol_init.feasible then
    Return empty_sol()
end
best ← sol_init; cur ← sol_init;
openable_set ← {i ∈ [0, N − 1] : |_allowed_types(i, facility_types, E)| > 0};
no_improvement ← 0;

for it = 0 to outer_iters − 1 do
    cur_sites ← cur.open_sites; trial_sites ← cur_sites.copy();
    trial_set ← {i | i ∈ trial_sites}; perturb_success ← True;

    for s = 1 to perturb_swaps do
        rem ← rng.RandomChoice(trial_set);
        closed_set ← openable_set − trial_set;
        if closed_set = ∅ then
            perturb_success ← False;
            break
        end
        add ← rng.RandomChoice(closed_set);
        trial_sites ← (trial_sites \ {rem}) ∪ {add};
        Remove duplicate elements from trial_sites;
        if |trial_sites| ≠ P then
            perturb_success ← False;
            break
        end
        trial_set ← {i | i ∈ trial_sites};
    end
    trial ← decode_repair_from_open_sites( trial_sites, P, N, E, CL, cl_sp,
      maxd, demand facility_types, capacities, cost_types, C_max,
      transport_mode, combined_cost_per_km)
    if ¬trial.feasible then
        continue
    end
    trial2 ← meta_vnd_assignments( trial, P,N, E, CL, cl_sp, maxd, demand
      facility_types, capacities, cost_types, C_max transport_mode,
      combined_cost_per_km,vnd_iters)

    cur ← (trial2 if  trial2.feasible else  trial)
    if cur.distance_obj < best.distance_obj then
        best ← cur; no_improvement ← 0;
    else
        no_improvement ← no_improvement + 1;
    end
    if no_improvement ≥ patience then
        break
    end
end

Return best
```

Πίνακας 3.3: Διαφορές Απλής και Βελτιωμένης Έκδοσης Branch and Bound

Κατηγορία	**Απλή Έκδοση**	**Βελτιωμένη Έκδοση**
Στρατηγική Εξερεύνησης	DFS σε όλο το δέντρο	Υβριδική στρατηγική: DFS μόνο όταν δεν υπάρχει αρχική εφικτή λύση αλλιώς Best First
Δομή Κόμβου	Απλή κλάση Node με όρια μεταβλητών και simplex βάσεις	Βελτιωμένη κλάση Node με `__slots__` για μείωση μνήμης και ταχύτερη πρόσβαση, μοναδικό ID και πλήρη μεταδεδομένα κόμβου
Προεπεξεργασία	Δεν υπάρχει προεπεξεργασία	Μετρικές προεπεξεργασίας και distance-based pruning για αποδοτικότερο pruning
Επιλογή Μεταβλητής Διακλάδωσης	Σειριακή επιλογή: πρώτη κλασματική μεταβλητή	Πολυκριτηριακή επιλογή (site-first) με μετρικές, capacity pressure, fractional thresholds
Τεχνικές Διάδοσης Περιορισμών	Δεν εφαρμόζονται	Propagation σε x και y για αποκλεισμό τύπων και μη εφικτών αναθέσεων
Ευρετικοί Αλγόριθμοι	Δεν εφαρμόζονται	Κατασκευαστικοί ευρετικοί (distance-based, cost-based, coverage-based) και repair ευρετικοί (reassignment, swap)
Μεθευρετικοί Αλγόριθμοι	Δεν εφαρμόζονται	Χρήση VND, SA και ILS τεχνικών για βελτίωση αρχικής λύσης και ενίσχυση του pruning

Κεφάλαιο 4

Υπολογιστική μελέτη

4.1 Παραγωγή και Οργάνωση Σεναρίων

Αναπτύσσουμε μια γεννήτρια προβλημάτων με στόχο τη δημιουργία σύνθετων σεναρίων αυτόματα. Ο χρήστης καθορίζει ορισμένες βασικές παραμέτρους εισόδου του σεναρίου, όπως το μέγεθος του πλέγματος (grid_size), τον συνολικό αριθμό σημείων που χρησιμοποιούνται (T), τον αριθμό των πελατών (CL), τον αριθμό των υποψήφιων τοποθεσιών εγκατάστασης (N), τον αριθμό των εγκαταστάσεων που θα ανοίξουν (P) και τον αριθμό των προβλημάτων (PR) που θα δημιουργηθούν με αυτά τα δεδομένα. Έπειτα, η γεννήτρια δημιουργεί το υπόλοιπο σύνολο δεδομένων χρησιμοποιώντας μια μέθοδο στοχαστικής δειγματοληψίας που διαχειρίζεται τη μεταβλητότητα. Η διαδικασία παραγωγής κάθε προβλήματος περιλαμβάνει μια ροή οκτώ βημάτων:

Βήμα 1 – Κατασκευή Χωρικού Πλέγματος Η αρχική φάση είναι η δημιουργία ενός πλέγματος μεγέθους (grid_size × grid_size) από το οποίο ορίζονται τυχαία N μοναδικές τοποθεσίες ως υποψήφιες εγκαταστάσεις και CL θέσεις ως πελάτες.

Βήμα 2 – Υπολογισμός Απόστασης Υπολογίζονται οι πιο σύντομες διαδρομές (Shortest paths) για όλα τα ζεύγη πελάτη-εγκατάστασης και εγκατάστασης-εγκατάστασης, καθώς και οι ευκλείδιες αποστάσεις (Manhattan).

Βήμα 3 – Ανάθεση Τύπων Εγκαταστάσεων Για κάθε υποψήφια τοποθεσία επιλέγεται τυχαία διαθέσιμος όγκος που βρίσκεται στο εύρος 0.06-0.2 m^3. Με αυτόν τον τρόπο, θα προσδιοριστεί ο μέγιστος τύπος θυρίδας που μπορεί να φιλοξενήσει

: small (≥0.0521 m^3), medium (≥0.1082 m^3), ή large (≥0.1633 m^3). Έτσι, λαμβάνουμε σωστά υπόψη τους χωρικούς περιορισμούς στο πρόβλημα.

Βήμα 4 – Στοχαστική Ζήτηση με Χωρική Ετερογένεια Μια ζήτηση για κάθε πελάτη δημιουργείται με εκθετική κατανομή ($\lambda = 0.023$) και κλιμακώνεται στην γεωγραφική του περιοχή (8 ζώνες με διαφορετική ένταση ζήτησης). Η τελική ζήτηση στρογγυλοποιείται σε ακέραιο για 1–15 δέματα ανά πελάτη.

Βήμα 5 – Χωρητικότητα Εγκαταστάσεων Κάθε κατηγορία εγκατάστασης αποκτά τυχαίες χωρητικότητες δέματων μέσα σε συγκεκριμένα εύρη: Small (2–80), Medium (80–180) και Large (180–260) δέματα. Επιπλέον, οι χωρητικότητες μπορούν να ρυθμιστούν με έναν συντελεστή αυστηρότητας ώστε να παραχθούν προβλήματα με διαφορετικά επίπεδα δυσκολίας.

Βήμα 6 – Μέσα Μεταφοράς και Όρια Προσβασιμότητας Κάθε πελάτης τοποθετείται προσεκτικά σε ένα από τα πέντε διαθέσιμα μέσα μεταφοράς, με βάση τις εμπειρικές στατιστικές χρήσης. Καθορίζεται επίσης, διαφορετικό μέγιστο όριο απόστασης για κάθε μέσο μεταφοράς: 0.5 χιλιόμετρα για τα πόδια, 1.0 χιλιόμετρα για το ποδήλατο, 2.0 χιλιόμετρα για τις δημόσιες συγκοινωνίες, 3.5 χιλιόμετρα για το αυτοκίνητο και 6.0 χιλιόμετρα για τα ηλεκτρικά οχήματα.

Βήμα 7 – Υπολογισμός Κόστους Η τιμή κάθε κατηγορίας θυρίδας προσδιορίζεται βάσει της καθαρής παρούσας αξίας (NPV) για διάστημα 15 ετών. Στον υπολογισμό αυτό λαμβάνονται υπόψη τα έξοδα εγκατάστασης, εξοπλισμού και λειτουργίας. Τα τελικά κόστη καταγράφονται ως εξής: Μικρή (102.2€), Μεσαία (130.8€) και Μεγάλη (165.8€). Το διαθέσιμο κεφάλαιο ορίζεται ως $C_{max} = 180 \cdot P$.

Βήμα 8 – Αποθήκευση Το πρόβλημα καταγράφεται σε ένα αρχείο κειμένου, το οποίο περιέχει όλες τις αναγκαίες παραμέτρους για την αυτόματη ανάγνωση από το μαθηματικό μοντέλο. Συγκεκριμένα, στη πρώτη σειρά του αρχείου αναγράφονται τέσσερις αριθμοί. Ο πρώτος αριθμός αντιστοιχεί στο συνολικό αριθμό σημείων, ο δεύτερος στον αριθμό των πελατών, ο τρίτος στο σύνολο των υποφήφιων θέσεων προς εγκατάσταση (N = T - CL) και ο τέταρτος στον αριθμό των εγκαταστάσεων που θέλουμε να ανοίξουν.

Σύνοψη Παραμέτρων Ο Πίνακας 4.1 συνοψίζει τις βασικές παραμέτρους που χρησιμοποιούνται από την γεννήτρια προβλημάτων.

Πίνακας 4.1: Σύνοψη παραμέτρων instance generator

Κατηγορία	**Παράμετρος**	**Τιμές/Εύρος**
Δομή	Μέγεθος πλέγματος	Ορίζεται από χρήστη
	Υποψήφιες θέσεις (N)	Ορίζεται από χρήστη
	Πελάτες (CL)	Ορίζεται από χρήστη
	Εγκαταστάσεις (P)	Ορίζεται από χρήστη
Τύποι	Small (type 0)	Όγκος ≥0.0521 m^3, Χωρητικότητα 2–80
Εγκαταστάσεων	Medium (type 1)	Όγκος ≥0.1082 m^3, Χωρητικότητα 80–180
	Large (type 2)	Όγκος ≥0.1633 m^3, Χωρητικότητα 180–260
Ζήτηση	Κατανομή	Εκθετική ($\lambda = 0.023$)
	Γεωγραφικές ζώνες	8 περιοχές με διαφορετική ένταση
	Εύρος δεμάτων	1–15 ανά πελάτη
Μεταφορά	Πόδια	40.875%, Max distance: 0.5 km
	Ποδήλατο	7.125%, Max distance: 1.0 km
	MMM	7.875%, Max distance: 2.0 km
	IX	42.625%, Max distance: 3.5 km
	Ηλεκτρικό	1.5%, Max distance: 6.0 km
Κόστος	NPV Small (15 έτη)	102.2€
	NPV Medium (15 έτη)	130.8€
	NPV Large (15 έτη)	165.8€
	Διαθέσιμο κεφάλαιο	$C_{max} = 180 \cdot P$ €

Για την πραγματοποίηση της υπολογιστικής μελέτης, δημιουργήθηκαν 10 κλάσεις προβλημάτων αυξανόμενης δυσκολίας. Κάθε κλάση, περιέχει 10 προβλήματα, οπότε έχουμε συνολικά 100 προβλήματα. Στην παρούσα εργασία, η γλώσσα που χρησιμοποιήθηκε για την υλοποίηση όλων των αλγορίθμων είναι η Python στο περιβάλλον Visual Studio Code με έκδοση 1.107.0. Οι υπολογιστικές δοκιμές υλοποιήθηκαν σε σύστημα με επεξεργαστή AMD Ryzen 3 3200U (2 cores, 4 threads, έως 2.60 GHz) και 10 GB DDR4 RAM, υπό λειτουργικό σύστημα Windows 10 64-bit. Για την υπολογιστική μελέτη υλοποιήθηκαν και συγκρίθηκαν δύο εκδόσεις του αλγορίθμου

Branch and Bound. Η πρώτη αφορά την απλή έκδοση, η οποία χρησιμοποιεί στρατηγική αναζήτησης βάθους (DFS) και σειριακή επιλογή μεταβλητής διακλάδωσης. Η δεύτερη αφορά τη βελτιωμένη έκδοση, η οποία ενσωματώνει υβριδική στρατηγική εξερεύνησης καθώς και τις βελτιώσεις που παρουσιάστηκαν στην προηγούμενη ενότητα. Για κάθε μία από τις εκδόσεις, εκτελέστηκαν τα ίδια 100 προβλήματα ώστε η σύγκριση να είναι δίκαιη. Τέλος, εφαρμόστηκε περιορισμός χρόνου μιας ώρας για κάθε υποπρόβλημα. Σε περίπτωση που ο χρόνος εκτέλεσης ξεπερνούσε τη μία ώρα, η διαδικασία τερματιζόταν αυτόματα και επιστρεφόταν η καλύτερη λύση που είχε βρεθεί μέχρι εκείνη τη στιγμή.

4.2 Σύγκριση Απλής και Βελτιωμένης Έκδοσης

Σύγκριση Χρόνου Εκτέλεσης

Στον Πίνακα 4.2 καταγράφονται τα αποτελέσματα των μετρήσεων της απλής και βελτιωμένης έκδοσης του αλγορίθμου Branch and Bound. Ειδικότερα, οι τιμές που παρουσιάζονται αφορούν το μέσο γεωμετρικό χρόνο εκτέλεσης για κάθε μία από τις δέκα κλάσεις. Όπως απεικονίζεται στον πίνακα, οι χρόνοι εκτέλεσης της απλής έκδοσης κυμαίνονται από 0.063 εώς 15.594 δευτερόλεπτα για τις μικρές κλάσεις ($N \leq 8$), ενώ για τις μεσαίες και τις μεγάλες κλάσεις η εκτέλεση φτάνει τον περιορισμό χρόνου μίας ώρας. Παράλληλα, οι χρόνοι εκτέλεσης της βελτιωμένης έκδοσης κυμαίνονται από 0.004 δευτερόλεπτα εώς 1.143 για τις μικρές κλάσεις, για τις μεσαίες κλάσεις εώς 706.826 δευτερόλεπτα και για τις μεγάλες κλάσεις η εκτέλεση φτάνει επίσης τον περιορισμό χρόνου μίας ώρας. Παρατηρούμε, πως και στις 10 κλάσεις των προβλημάτων η βελτιωμένη έκδοση έχει μικρότερο χρόνο εκτέλεσης σε σχέση με την απλή έκδοση. Αποδεικνύεται πως το σύνολο των βελτιώσεων που ενσωματώθηκε, εκμεταλλεύοντας τη δομή του προβλήματος, οδηγούν σε ταχύτερη εκτέλεση. Στις μικρές και μεσαίες κλάσεις η βελτιωμένη έκδοση υπερτερεί της απλής, με χαρακτηριστικό παράδειγμα την κλάση 10_100_30_9, όπου ο χρόνος εκτέλεσης της απλή έκδοσης είναι 3564.191 δευτερόλεπτα ενώ της βελτιωμένης 243.880 δευτερόλεπτα, δηλαδή σχεδόν 15 φορές ταχύτερη από την απλή. Στις μεγαλύτερες κλάσεις, η εκτέλεση των δύο εκδόσεων φτάνει τον χρονικό περιορισμό της μιας ώρας. Αυτή η υπέρβαση του χρόνου οφείλεται στην εκθετική αύξηση

του χώρου αναζήτησης και στα χαλαρά φράγματα που προκύπτουν από την ισχυρή αλληλεξάρτηση των περιορισμών της ζήτηση, της χωρητικότητα και του προϋπολογισμού. Επίσης, οι περιορισμοί διαφορετικών κατηγοριών δημιουργούν σύνθετη συνδυαστική φύση, δυσκολεύοντας έτσι την εύρεση εφικτών λύσεων. Τέλος, η χρήση του γεωμετρικού μέσου παρέχει πιο αντιπροσωπευτική εικόνα της συνολικής απόδοσης, καθώς μειώνει την επίδραση των ακραίων τιμών. Έτσι, η τιμή 339.730 της απλής έκδοσης και η τιμή 97.625 της βελτιωμένης επιβεβαιώνουν την υπεροχή της βελτιωμένης, η οποία είναι συνολικά περίπου 3.5 φορές ταχύτερη. Είναι σημαντικό να επισημανθεί ότι η υπεροχή της βελτιωμένης δεν είναι ομοιόμορφη σε όλες τις κλάσεις. Ειδικότερα, εξαρτάται σε μεγάλο βαθμό από τη φύση των περιορισμών που έχουν τεθεί. Για παράδειγμα, στις κλάσεις 8_64_20_7 και 10_100_30_9, όπου η σχέση μεταξύ ζήτησης, χωρητικότητας και προϋπολογισμού είναι πιο "συμβατή", οι ευρετικοί μέθοδοι, οι μεθευρετικές προσεγγίσεις καθώς και το pruning αποδίδουν εξαιρετικά. Αυτό έχει ως αποτέλεσμα σημαντική μείωση του χρόνου εκτέλεσης. Από την άλλη πλευρά, στην κλάση 12_144_45_12, οι περιορισμοί είναι αυστηρότεροι και η χαλάρωση του LP παρέχει πιο αδύναμες εκτιμήσεις. Αυτό έχει ως αποτέλεσμα τη διατήρηση σημαντικής βελτίωσης, αν και λιγότερο έντονης. Στις μεγαλύτερες κλάσεις η εκθετική αύξηση του χώρου αναζήτησης καθώς και η αλληλεξάρτηση των περιορισμών περιορίζουν την αποτελεσματικότητα κάθε στρατηγικής, γεγονός που οδηγεί και τις δύο εκδόσεις στην εξάντληση του χρονικού ορίου.

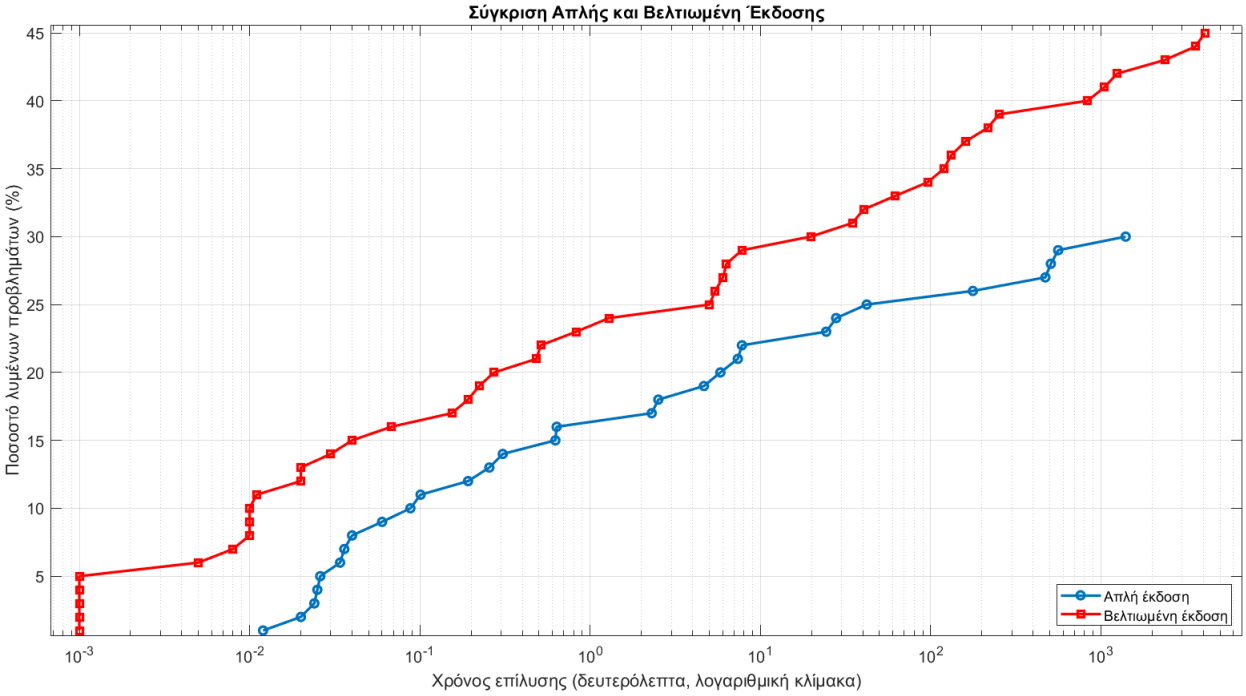
Πίνακας 4.2: Χρόνοι εκτέλεσης (σε δευτερόλεπτα) Απλής και Βελτιωμένης Έκδοσης

Κλάσεις	**Απλή Έκδοση**	**Βελτιωμένη Έκδοση**
4_16_6_3	0.063	0.004
6_36_12_5	2.687	0.165
8_64_20_7	15.594	1.143
10_100_30_9	3564.191	243.880
12_144_45_12	3600	706.826
14_196_60_15	3600	3600
16_256_80_19	3600	3600
17_289_105_21	3600	3600
18_324_115_23	3600	3600
18_324_129_25	3600	3600
Γεωμετρικός Μέσος	339.730	97.625

Στο Σχήμα 4.1 παρουσιάζεται το ποσοστό των λυμένων προβλημάτων (άξονας Y) ως συνάρτηση του χρόνου επίλυσης (άξονας X, σε λογαριθμική κλίμακα) για

τις δέκα κλάσεις προβλημάτων. Όπως φαίνεται και από το διάγραμμα η βελτιωμένη έκδοση βρίσκεται σταθερά πάνω από την απλή, γεγονός που δείχνει πώς για κάθε χρονική στιγμή η βελτιωμένη έκδοση λύνει μεγαλύτερο ποσοστό προβλημάτων. Ακόμη, παρατηρείται πως η βελτιωμένη έκδοση ξεκινά πιο αριστερά σε σχέση με την απλή. Συγκεκριμένα, για χρόνους επίλυσης με τιμές $\leq 10^{-2}$ δευτερολέπτων η βελτιωμένη έκδοση λύνει ήδη περίπου το 10% των προβλημάτων, σε αντίθεση με την απλή που δεν λύνει κανένα. Για τις τιμές του χρόνου από 10^{-2} εώς 10^{2} δευτερόλεπτα, η βελτιωμένη έκδοση λύνει περίπου το 35% των προβλημάτων ενώ η απλή το 25%. Για μεγαλύτερους χρόνους, η βελτιωμένη έκδοση συνεχίζει να υπερέχει καταλήγοντας στο 45% ενώ η απλή στο 30%, γεγονός που αναδεικνύει την καλύτερη κλιμάκωση της βελτιωμένης σε προβλήματα μεγαλύτερου μεγέθους και πολυπλοκότητας.

Σχήμα 4.1: Ποσοστό Λυμένων Προβλημάτων Απλής και Βελτιωμένης Έκδοσης



Σύγκριση Απόκλισης από τη Βέλτιστη Λύση (GAP %)

Στον Πίνακα 4.3 παρουσιάζονται οι τιμές της αντικειμενικής συνάρτησης που καταγράφηκαν από τον λύτη Gurobi, την απλή έκδοση και βελτιωμένη έκδοση του αλγορίθμου Branch and Bound, για κάθε κλάση προβλημάτων. Οι τιμές του Gurobi

αντιστοιχούν στις βέλτιστες λύσεις των προβλημάτων και αποτελούν το σημείο αναφοράς για την αξιολόγηση της ποιότητας των λύσεων που προκύπτουν από τις εκδόσεις. Όπως φαίνεται και στον πίνακα, στις μικρές κλάσεις (N ≤ 8) και οι δύο εκδόσεις βρίσκουν την βέλτιστη λύση, δηλαδή οι τιμές της αντικειμενικής συνάρτησης ταυτίζονται πλήρως με την τιμή του λύτη Gurobi. Στην κλάση 10_100_30_9, η βελτιωμένη έκδοση καταφέρνει να βρει τη βέλτιστη λύση με τιμή 5.21, σε αντίθεση με την απλή που απέχει από αυτή, με τιμή 6.743. Στις μεγαλύτερες κλάσεις και οι δύο εκδόσεις αποτυγχάνουν στην εύρεση της βέλτιστης λύσης. Παρόλα αυτά, η βελτιωμένη έκδοση βρίσκει σταθερά καλύτερες τιμές σε σύγκριση με την απλή έκδοση, με την υπεροχή της να γίνεται πιο έντονη στο μεγαλύτερο πρόβλημα 18_324_129_25, όπου η βελτιωμένη δίνει λύση 20.305 ενώ η απλή 139.01. Συμπερασματικά, η βελτιωμένη έκδοση φαίνεται να επιτυγχάνει σταθερά καλύτερες τιμές αντικειμενικής συνάρτησης επειδή συνδυάζει πιο στοχευμένη εξερεύνηση και ισχυρότερο pruning. Συγκεκριμένα, εξερευνά πιο "υποσχόμενους" κόμβους και ξεκινά από πολύ καλύτερη αρχική λύση λόγω ευρετικών και μεθευρετικών αλγορίθμων. Με αυτόν τον τρόπο, κλαδεύει τεράστιο μέρος του δέντρου που η απλή έκδοση αναγκάζεται να εξερευνήσει.

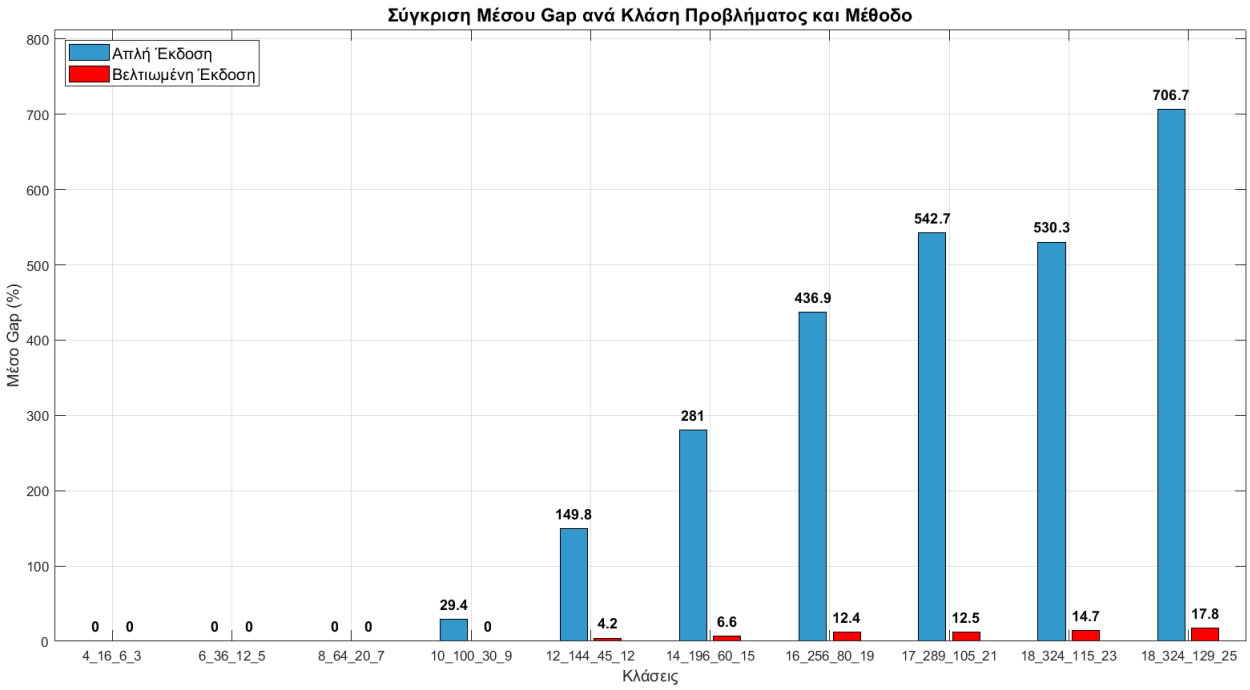
Πίνακας 4.3: Σύγκριση Τιμών Αντικειμενικής Συνάρτησης

Κλάσεις	**Λύση Gurobi**	**Λύση Απλής Έκδοσης**	**Λύση Βελτιωμένης Έκδοσης**
4_16_6_3	2.057	2.057	2.057
6_36_12_5	2.502	2.502	2.502
8_64_20_7	3.489	3.489	3.489
10_100_30_9	5.21	6.743	5.21
12_144_45_12	6.552	16.37	6.83
14_196_60_15	8.759	33.375	9.34
16_256_80_19	10.505	56.4	11.805
17_289_105_21	14.022	90.115	15.77
18_324_115_23	15.174	95.647	17.4
18_324_129_25	17.233	139.01	20.305

Στο Σχήμα 4.2 παρουσιάζεται το μέσο σχετικό σφάλμα (GAP%) των κλάσεων (άξονας Y) ως συνάρτηση των δέκα κλάσεων (άξονας X). Το GAP αποτελεί βασικό μέτρο αξιολόγησης της ποιότητας μιας λύσης σε προβλήματα βελτιστοποίησης και υπολογίζεται ως η ποσοστιαία διαφορά μεταξύ της λύσης που βρέθηκε και της βέλτιστης τιμής. Συγκεκριμένα, αποτυπώνει το ποσοστό απόκλισης της αντικειμενικής

τιμής των δύο εκδόσεων από τη βέλτιστη λύση του Gurobi. Από το σχήμα, παρατηρείται πως στις μικρές κλάσεις (N ≤ 8) και οι δύο εκδόσεις βρίσκουν τη βέλτιστη λύση, αφού εμφανίζουν μηδενικό GAP. Ακόμη, στην κλάση 10_100_30_9 η βελτιωμένη έκδοση διατηρεί μηδενικό GAP (βέλτιστη λύση) ενώ η απλή παρουσιάζει 29.4%. Στις μεσαίες και μεγάλες κλάσεις, η βελτιωμένη έκδοση προσεγγίζει με εξαιρετικά μικρότερη απόκλιση τη βέλτιστη λύση σε σχέση με την απλή έκδοση. Ειδικότερα, στην κλάση 12_144_45_12 το gap της βελτιωμένης είναι μόλις 4.2% έναντι 149.8% της απλής, ενώ στην κλάση 18_324_129_25 το μέσο GAP για την βελτιωμένη έκδοση είναι 17.8% ενώ για την απλή αυξάνεται σε 706.7%. Τα αποτελέσματα αυτά επιβεβαιώνουν την αποτελεσματικότητα της βελτιωμένης έκδοσης στην επίτευξη λύσεων υψηλής ποιότητας. Να σημειωθεί, πως για τον υπολογισμό του μέσου GAP κάθε κλάσης λήφθηκαν υπόψη μόνο τα προβλήματα για τα οποία βρέθηκε εφικτή λύση εντός του χρονικού περιορισμού.

Σχήμα 4.2: Μέσο GAP(%) ανά Κλάση Προβλήματος και Μέθοδο



Σύγκριση Εξερευνημένων Κόμβων

Στον Πίνακα 4.4 εμφανίζεται ο αριθμός των εξερευνημένων κόμβων του δέντρου Branch and Bound για το λύτη Gurobi, την απλή και τη βελτιωμένη έκδοση για κάθε

κλάση προβλήματος. Αυτός ο αριθμός, αντιστοιχεί στον αριθμό των υποπροβλημάτων που επεξεργάζεται ο αλγόριθμος και αποτελεί βασικό δείκτη της έκτασης της αναζήτησης. Όπως απεικονίζεται στον πίνακα, ο αριθμός των κόμβων διαφοροποιείται σημαντικά, ανάλογα με το μέγεθος του προβλήματος. Ο Gurobi για τις μικρές κλάσεις (N ≤ 8) εξερευνά μόνο έναν κόμβο αναδεικνύοντας την αποτελεσματικότητα του. Αντιθέτως, η απλή έκδοση εξερευνά μεγαλύτερο αριθμό κόμβων με τιμές από 23 εώς 28660, ενώ η βελτιωμένη έκδοση πετυχαίνει δραστική μείωση με τιμές από 4 εώς 335. Σε αυτές τις μικρές κλάσεις, παρόλο που και οι δύο εκδόσεις βρίσκουν την βέλτιστη λύση, η βελτιωμένη χρειάζεται έως και 85 φορές λιγότερους κόμβους από την απλή. Ακόμη, στην κλάση 10_100_30_9 όπου η βελτιωμένη έκδοση βρίσκει τη βέλτιστη λύση με 21491 κόμβους, η απλή χρειάζεται 171.669 κόμβους για να πετύχει λύση που δεν είναι βέλτιστη (6.743 έναντι 5.21, GAP 29%). Στις υπόλοιπες κλάσεις παρατηρείται μια κρίσιμη μετάβαση. Παρόλο που ο Gurobi αυξάνει τους κόμβους από 26 εώς 36293 διατηρώντας τη βέλτιστη λύση, οι δύο εκδόσεις εμφανίζουν μείωση στους κόμβους με την απλή από 101456 σε 22664 και την βελτιωμένη από 13137 σε 3234 χωρίς να καταφέρνουν να βρουν τη βέλτιστη λύση. Αυτή η μη μονοτονική συμπεριφορά στον αριθμό των εξερευνημένων κόμβων, ιδίως από την κλάση 10_100_30_9 και μετά, συνδέεται άμεσα με τη δομή του προβλήματος αλλά και το κόστος επεξεργασίας ανά κόμβο. Στις κλάσεις με N ≤ 10, οι βελτιωμένες στρατηγικές διακλάδωσης, το propagation και οι καλές αρχικές λύσεις επιτρέπουν στη βελτιωμένη έκδοση να κλαδεύει πιο επιθετικά μεγάλα τμήματα του δέντρου. Με αυτόν τον τρόπο, μειώνονται σημαντικά οι κόμβοι σε σχέση με την απλή. Ωστόσο, στις υπόλοιπες κλάσεις , η εκθετική αύξηση του χώρου αναζήτησης καθώς και η αλληλεξάρτηση των περιορισμών (ζήτηση–χωρητικότητα–προϋπολογισμός) καθιστούν τα LP φράγματα πιο χαλαρά και αυξάνουν το κόστος επίλυσης ανά κόμβο. Έτσι, παρότι η βελτιωμένη έκδοση εξερευνά λιγότερους και πιο "υποσχόμενους" κόμβους από την απλή, ο περιορισμούς του χρόνου δεν επιτρέπει την πλήρη ανάπτυξη του δέντρου. Αυτό οδηγεί σε μείωση των κόμβων αλλά και σε αδυναμία απόδειξης βελτιστότητας.

Στο Σχήμα 4.3 απεικονίζεται γραφικά ο αριθμός των εξερευνημένων κόμβων (άξονας Υ σε λογαριθμική κλίμακα $\times 10^4$) για κάθε κλάση προβλημάτων (άξονας Χ). Στις μικρές κλάσεις όπου βρίσκεται η βέλτιστη λύση, ο λύτης Gurobi εξερευνά 1

Πίνακας 4.4: Σύγκριση Εξερευνημένων Κόμβων Gurobi, Απλής και Βελτιωμένης

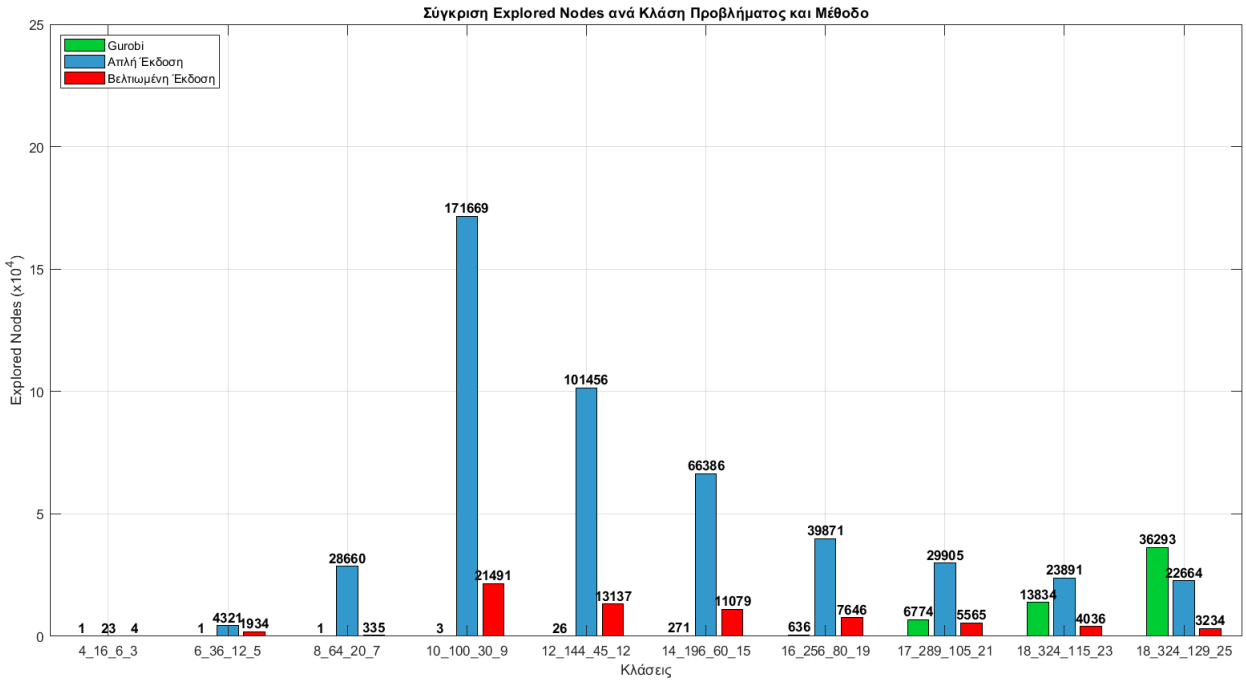
Κλάσεις	**Gurobi**	**Απλή Έκδοση**	**Βελτιωμένης Έκδοση**
4_16_6_3	1	23	4
6_36_12_5	1	4321	1934
8_64_20_7	1	28660	335
10_100_30_9	3	171669	21491
12_144_45_12	26	101456	13137
14_196_60_15	271	66386	11079
16_256_80_19	636	39871	7646
17_289_105_21	6774	29905	5565
18_324_115_23	13834	23891	4036
18_324_129_25	36293	22664	3234

κόμβο σε αντίθεση με την απλή έκδοση που απαιτεί από 23 εώς 28660 κόμβους και με τη βελτιωμένη να κυμαίνεται από 4 έως 335 κόμβους. Η διαφορά γίνεται ακόμα πιο έντονη στην κλάση 10_100_30_9, όπου η απλή έκδοση φτάνει στον μέγιστό αριθμό εξερευνημένων κόμβων με τιμή 171669, ενώ η βελτιωμένη εξερευνά μικρότερο αριθμό με τιμή 21491. Από εκεί και πέρα εμφανίζεται μια κρίσιμη διαφορά στον αριθμό των κόμβων. Συγκεκριμένα για τις μεγαλύτερες κλάσεις ενώ ο Gurobi αυξάνει τον αριθμό των κόμβων μέχρι τους 36293 εξασφαλίζοντας τη βέλτιστη λύση, οι δύο εκδόσεις εμφανίζουν μείωση των κόμβων. Όπως απεικονίζεται, η απλή έκδοση μειώνεται από 171669 σε 22664 κόμβους και η βελτιωμένη από 21491 σε 3234. Αυτή η μείωση δεν αντανακλά σε καλύτερη απόδοση των δύο εκδόσεων από τον Gurobi όπως αναφέρθηκε και παραπάνω αλλά επιβεβαιώνει την αυξημένη πολυπλοκότητα ανά κόμβο που σε συνδυασμό με τον χρονικό περιορισμό περιορίζει την εξερεύνηση του δέντρου. Ο Gurobi, χάρη στην βελτιστοποιημένη υλοποίησή του σε χαμηλό επίπεδο, επεξεργάζεται κόμβους πολύ γρηγορότερα. Τέλος, είναι εμφανής η συνολική υπεροχή της βελτιωμένης έκδοσης έναντι της απλής αφού βρίσκει καλύτερες λύσεις με λιγότερους κόμβους.

Σύγκριση Gurobi και Βελτιωμένης Έκδοσης

Στον Πίνακα 4.5 παρουσιάζονται τα αποτελέσματα του λύτη Gurobi και της βελτιωμένης έκδοσης και γίνεται σύγκριση με βάση τον γεωμετρικό χρόνο εκτέλεσης των προβλημάτων κάθε κλάσης. Να σημειωθεί πως ο Gurobi αποτελεί ακριβή μέθοδο και σε κάθε κλάση βρίσκει τη βέλτιστη λύση, ενώ η βελτιωμένη έκδοση κα-

Σχήμα 4.3: Μέσος Αριθμός Εξερευνημένων Κόμβων ανά Κλάση Προβλήματος και Μέθοδο



ταφέρνει να επιτύχει τη βέλτιστη λύση μέχρι και την κλάση 10_100_30_9, αλλά στις μεγαλύτερες κλάσεις παράγει προσεγγιστικές λύσεις εντός του χρονικού περιορισμού της μιας ώρας. Όπως φαίνεται στον πίνακα, μόνο για την κλάση 4_16_6_3 η βελτιωμένη έκδοση απαιτεί λιγότερο χρόνο για την επίλυση με τιμή 0.004 δευτερόλεπτα έναντι των 0.034 δευτερολέπτων του Gurobi. Στις υπόλοιπες μικρές κλάσεις 6_36_12_5 και 8_64_20_7, ο Gurobi υπερτερεί σημαντικά σε ταχύτητα με τιμές 0.112 και 0.243 δευτερόλεπτα αντίστοιχα, ενώ η βελτιωμένη έκδοση απαιτεί περισσότερο χρόνο 0.165 και 1.143 δευτερόλεπτα. Στην τελευταία κλάση 10_100_30_9 που η βελτιωμένη έκδοση βρίσκει τη βέλτιστη λύση, ο Gurobi επιλύει το πρόβλημα σε 1.156 δευτερόλεπτα, ενώ η βελτιωμένη χρειάζεται 243.880 δευτερόλεπτα. Η μεγάλη αυτή διαφορά δείχνει την υπολογιστική πολυπλοκότητα και το κόστος των πρόσθετων λειτουργιών που εφαρμόστηκαν στην βελτιωμένη έκδοση. Τέλος, από την κλάση 12_144_45_12 και μετά ο Gurobi αυξάνει σταδιακά τους χρόνους επίλυσης φτάνοντας μέχρι τα 663.700 δευτερόλεπτα. Αντίθετα, η βελτιωμένη έκδοση εξαντλεί το χρονικό όριο εκτέλεσης χωρίς να καταλήγει στη βέλτιστη λύση, παρότι παράγει λύσεις υψηλής ποιότητας.

Στον Πίνακα 4.6 εμφανίζεται το μέσο σχετικό σφάλμα (GAP%) των λύσεων της βελτιωμένης έκδοσης με τις αντίστοιχες βέλτιστες λύσεις του λύτη Gurobi, για κάθε

Πίνακας 4.5: Αποτελέσματα μετρήσεων λύτη Gurobi και Βελτιωμένης Έκδοσης

Κλάσεις	**Χρόνος Gurobi**	**Χρόνος Βελτιωμένης Έκδοσης**
4_16_6_3	0.034	0.004
6_36_12_5	0.112	0.165
8_64_20_7	0.243	1.143
10_100_30_9	1.156	243.880
12_144_45_12	1.842	706.826
14_196_60_15	5.501	3600
16_256_80_19	13.834	3600
17_289_105_21	74.865	3600
18_324_115_23	226.269	3600
18_324_129_25	663.700	3600
Γεωμετρικός Μέσος	4.194	97.625

κλάση προβλημάτων. Το GAP χρησιμοποιείται για να αποτυπώσει την ποσοστιαία απόκλιση της αντικειμενικής τιμής της βελτιωμένη έκδοση από την αντίστοιχη βέλτιστη του Gurobi. Όπως προκύπτει από τον πίνακα, το GAP λαμβάνει μηδενική τιμή στις πρώτες τέσσερις κατηγορίες, γεγονός που επιβεβαιώνει την εύρεση βέλτιστης λύσης από τη βελτιωμένη έκδοση εντός του καθορισμένου χρονικού ορίου. Επιπλέον, παρατηρείται ότι στην κλάση 12_144_45_12 το GAP είναι ελάχιστο με τιμή 4.2%, υποδηλώνοντας την υψηλή ποιότητα παραγόμενων λύσεων, αφού είναι αρκετά κοντά στη βέλτιστη. Στη συνέχεια, καθώς αυξάνεται το μέγεθος και η πολυπλοκότητα των προβλημάτων το GAP εμφανίζει σταδιακή αύξηση με τιμές που ξεκινούν από 6.6% στην κλάση 14_196_60_15 και καταλήγουν στη τιμή 17.8% της κλάσης 18_324_129_25. Παρόλο της αύξησης, η διατήρηση του GAP σε τιμές κάτω του 18% ακόμη και στα δυσκολότερα προβλήματα της κλάσης 18_324_129_25 αποτελεί ένδειξη υψηλής αποτελεσματικότητας των βελτιώσεων που ενσωματώθηκαν.

Συνολικά, η διαφοροποίηση ανάμεσα στον Gurobi και στην βελτιωμένη έκδοση εξηγείται από το γεγονός ότι ο Gurobi διαθέτει εξαιρετικά βελτιστοποιημένες ρουτίνες σε χαμηλό επίπεδο. Συγκεκριμένα, του επιτρέπουν να επιλύει με μεγάλη ταχύτητα μικρά και μεσαία προβλήματα. Από την άλλη, η βελτιωμένη έκδοση επιβαρύνεται από το κόστος των μεθόδων, του propagation αλλά και των πρόσθετων ελέγχων. Αυτό έχει ως συνέπεια να αυξάνουν τον χρόνο ανά κόμβο αλλά οδηγούν σε λύσεις υψηλής ποιότητας. Στις μεσαίες και μεγάλες κλάσεις με $N \geq 14$, η εκθετική αύξηση του χώρου αναζήτησης καθώς και τα χαλαρά LP φράγματα καθιστούν

αδύνατη την απόδειξη βελτιστότητας εντός του χρονικού ορίου, με αποτέλεσμα η βελτιωμένη έκδοση να παράγει προσεγγιστικές λύσεις με αρκετά χαμηλό GAP. Έτσι, δε στοχεύει να ανταγωνιστεί τον Gurobi όσον αφορά την ταχύτητα, αλλά αποσκοπεί στην επίτευξη λύσεων υψηλής ποιότητας σε δυσκολότερες κλάσεις όπου σε αυτές η πλήρης βελτιστοποίηση είναι υπολογιστικά ανέφικτη.

Πίνακας 4.6: Σύγκριση Μέσου GAP (%) μεταξύ Gurobi και Βελτιωμένης Έκδοσης

Κλάσεις	**Μέσο GAP**
4_16_6_3	0
6_36_12_5	0
8_64_20_7	0
10_100_30_9	0
12_144_45_12	4.2
14_196_60_15	6.6
16_256_80_19	12.3
17_289_105_21	12.5
18_324_115_23	14.7
18_324_129_25	17.8

Κεφάλαιο 5

Συμπεράσματα

Αυτή η διπλωματική εργασία εξετάζει το ζήτημα της χωροθέτησης θυρίδων παράδοσης, το οποίο σήμερα είναι κρίσιμο για την ικανοποίηση των πελατών και τη βιωσιμότητα του περιβάλλοντος. Στη μελετημένη βιβλιογραφία, το ζήτημα έχει προσεγγιστεί ως μια επέκταση των παραδοσιακών προβλημάτων χωροθέτησης εγκαταστάσεων. Αυτή η προσέγγιση επεκτείνεται για να συμπεριλάβει τη χωρητικότητα, την ποικιλία στις θυρίδες, τη συμπεριφορά των χρηστών και τα χρονικά πλαίσια. Το ζήτημα έχει διατυπωθεί μέσω μαθηματικών μοντέλων Μικτού Ακέραιου Γραμμικού Προγραμματισμού (MILP). Έχουν χρησιμοποιηθεί ακριβείς τεχνικές όπως οι Branch and Bound, Branch and Cut, Column Generation και η μέθοδος αποσύνθεσης Benders. Χρησιμοποιήθηκαν διάφορες ευρετικές προσεγγίσεις, όπως άπληστοι αλγόριθμοι, γενετικοί αλγόριθμοι, η μέθοδος μεταβαλλόμενης γειτονιάς (Variable Neighborhood Search), ο αλγόριθμος αναζήτησης διασποράς (Scatter Search) και το Simulated Annealing. Ταυτόχρονα, έχουν αναπτυχθεί μοντέλα που μελετούν τη συμπεριφορά των χρηστών και αντιμετωπίζουν την αβεβαιότητα, όπως η σταθερή βελτιστοποίηση (robust optimization) και η βασισμένη σε σενάρια προσέγγιση (scenario-based optimization). Σε αυτή την κατεύθυνση, η παρούσα εργασία επικέντρωσε το ενδιαφέρον στον σχεδιασμό και στην βελτιστοποίηση της προσβασιμότητας των θυρίδων και της βιωσιμότητας του περιβάλλοντος, μέσω αποδοτικών μεθόδων επίλυσης.

Αρχικά, το πρόβλημα περιγράφηκε ως p-median και υλοποιήθηκε αλγόριθμος για την μαθηματική μοντελοποίηση του μέσω MILP. Το γραμμικό ακέραιο πρόβλημα επιλύθηκε με τη χρήση του λύτη Gurobi ώστε να βρεθούν οι βέλτιστες λύσεις των προβλημάτων. Στη συνέχεια, αναπτύχθηκαν και εφαρμόστηκαν δύο εκδόσεις Branch

and Bound για την αντιμετώπιση του προβλήματος.

Η πρώτη βασίζεται στην τεχνική αναζήτησης βάθους (DFS) και στην σειριακή στρατηγική επιλογής κόμβου. Η δεύτερη στηρίζεται σε υβριδική τεχνική αναζήτησης συνδυάζοντας κατά βάθος αναζήτηση (DFS) και βέλτιστη αναζήτηση πρώτα (Best First Search). Ακόμη, ενσωματώθηκε μια σειρά από βελτιώσεις. Συγκεκριμένα, αναπτύχθηκε μια αποδοτική στρατηγική επιλογής κόμβου που ενσωματώνει τους περιορισμούς του μοντέλου, καθώς και τεχνικές διάδοσης περιορισμών. Για την παραγωγή αρχικών εφικτών λύσεων, σχεδιάστηκαν τέσσερις άπληστοι ευρετικοί αλγόριθμοι που δίνουν έμφαση σε διαφορετικά κριτήρια. Αυτοί είναι ο ευρετικός αλγόριθμος Distance Greedy Cover, ο Cost-aware Greedy, ο Feasibility-First και ο στοχαστικός GRASP Multistart. Η εφικτή λύση που προκύπτει από τους ευρετικούς αλγορίθμους χρησιμοποιείται ως αρχική λύση στους μεθευρετικούς αλγορίθμους μεταβαλλόμενης γειτονιάς (Variable Neighborhood Search), Simulated Annealing και επαναληπτικής τοπικής αναζήτησης (Iterated Local Search).Η καλύτερη λύση που προκύπτει από τους μεθευρετικούς χρησιμοποιείται ως άνω όριο στον αλγόριθμο.

Έπειτα ακολούθησε η εκτέλεση κάθε αλγορίθμου σε ένα σύνολο εκατό προβλημάτων ώστε να καταγραφεί η συμπεριφορά τους. Αρχικά, η μαθηματική μοντελοποίηση του προβλήματος επιλύθηκε με τη χρήση του λύτη Gurobi ώστε οι βέλτιστες λύσεις των προβλημάτων να χρησιμοποιηθούν ως σημείο σύγκρισης. Ο γεωμετρικός μέσος χρόνος εκτέλεσης για αυτήν την μέθοδο ήταν 4.194 δευτερόλεπτα. Ακολούθως, εκτελέστηκε η πρώτη έκδοση του αλγορίθμου Branch and Bound, η οποία κατάφερε να βρει βέλτιστη λύση στα 30 από τα 100 συνολικά προβλήματα εντός του χρονικού ορίου μιας ώρας. Ο γεωμετρικός μέσος του χρόνου εκτέλεσης ήταν 339.730, τιμή πολύ μεγαλύτερη σε σύγκριση με του Gurobi. Στη συνέχεια, εκτελέστηκε η δεύτερη έκδοση του αλγορίθμους Branch and Bound, η οποία βρήκε τη βέλτιστη λύση στα 45 από τα 100 προβλήματα. Ο γεωμετρικός μέσος του χρόνου εκτέλεσης αυτής της έκδοσης ανήλθε στα 97.625 δευτερόλεπτα σημειώνοντας μείωση κατά σχεδόν 4 φορές σε σχέση με την πρώτη έκδοση. Όσον αφορά την ποιότητα των λύσεων, η πρώτη έκδοση εμφανίζει μέσο σχετικό σφάλμα (GAP%) από 29.4% εώς 706.7%, ενώ η δεύτερη σημαντικά μικρότερο με τιμές που κυμαίνονται από 4.2% εώς 17.8%. Όσον αφορά τους κόμβους που εξερευνώνται, η πρώτη έκδοση εξερευνά από 23 εώς 171669 κόμβους σε αντίθεση με την δεύτερη που εξερευνά από 4 εώς 21491.

Η παρούσα μελέτη επιβεβαιώνει την αναγκαιότητα της βέλτιστης χωροθέτησης θυρίδων παράδοσης, λαμβάνοντας υπόψη πολλαπλούς και ρεαλιστικούς περιορισμούς που προωθούν βιώσιμες λύσεις. Η εκτέλεση των αλγορίθμων που πραγματοποιήθηκε ενισχύει την δυνατότητα ανάλυσης και λήψης αποφάσεων, εξασφαλίζοντας ταυτόχρονα πρακτικά εφαρμόσιμες λύσεις. Για το πρόβλημα μας η δεύτερη έκδοση του αλγορίθμου Branch and Bound που ενσωματώνει υβριδική τεχνική αναζήτησης και βελτιώσεις οδήγησε σε καλύτερα αποτελέσματα. Είναι σημαντικό να σημειωθεί πως παρότι η βελτιωμένη έκδοση δεν βρίσκει τη βέλτιστη λύση στο 55% των συνολικών προβλημάτων λόγω χρονικού ορίου, οι λύσεις που παράγει πλησιάζουν ικανοποιητικά τις βέλτιστες. Μια αρκετά ενδιαφέρουσα κατεύθυνση για μελλοντική έρευνα αποτελεί η ανάπτυξη δυναμικών ή στοχαστικών μοντέλων χωροθέτησης. Στην πράξη, με αυτά μπορούν να ενσωματωθούν αβεβαιότητες στη ζήτηση, στις ροές δεμάτων καθώς και στη διαθεσιμότητα των θυρίδων. Πιο συγκεκριμένα, η ενσωμάτωση χρονικά μεταβαλλόμενων παραμέτρων θα επέτρεπε την άμεση προσαρμογή των αποφάσεων σε πραγματικές συνθήκες λειτουργίας. Αντίθετα, οι στοχαστικές προσεγγίσεις θα μπορούσαν να αποτυπώσουν με καλύτερο τρόπο τη μεταβλητότητα της ζήτησης και των χωρητικοτήτων. Επίσης να σημειωθεί πως η διερεύνηση υβριδικών αλγορίθμων που συνδυάζουν ακριβείς και μαθευρετικές τεχνικές θα μπορούσε να βελτιώσει περισσότερο την απόδοση σε μεγάλης κλίμακας προβλήματα. Τέλος, αξίζει να εξεταστεί η Χρήση Γεωγραφικών Συστημάτων Πληροφοριών (GIS) για οπτικοποίηση των αποτελεσμάτων και τέλος η εφαρμογή της προσέγγισης σε πραγματικά δεδομένα από εταιρείες θυρίδων παράδοσης (π.χ InPost, BoxNow).

Βιβλιογραφία

[1] J. H. R. van Duin, B. W. Wiegmans, B. van Arem, and Y. van Amstel, "From home delivery to parcel lockers: a case study in amsterdam," *Transportation Research Procedia*, vol. 46, pp. 37–44, 2020. City Logistics 2019.

[2] A. Eliyan, A. Elomri, and L. Kerbache, "The last-mile delivery challenge: evaluating the efficiency of smart parcel stations," *Supply Chain Forum: An International Journal*, vol. 22, no. 4, pp. 360–369, 2021.

[3] Q. Zhang and E. Demir, "Parcel locker solutions for last mile delivery: a systematic literature review and future research directions," *Frontiers in Future Transportation*, vol. 6, p. 1654621, 2025.

[4] T. Campisi and I. Di Ruocco, "Accessibility and spatial location of parcel lockers in last-mile delivery: A literature review," *The Open Transportation Journal*, vol. 19, p. e26671212391037, August 2025.

[5] D. R. Pinchasik, I. B. Hovi, and B. Dong, "Replacing home deliveries by deliveries to parcel lockers: cost, traffic, emissions, and societal cost effects of locker network expansions in greater oslo," *International Journal of Logistics: Research and Applications*, Nov. 2023. Published online: 22 Nov 2023.

[6] I. B. Hovi and E. Bø, "Unlocking the potential: How can parcel lockers drive efficiency and environmental friendliness in e-commerce?," *Sustainable Futures*, vol. 7, p. 100189, 2024.

[7] B. Sawik, A. Serrano-Hernandez, A. Muro, and J. Faulin, "Multi-criteria simulation-optimization analysis of usage of automated parcel lockers: A practical approach," *Mathematics*, vol. 10, no. 23, p. 4423, 2022.

[8] J. Lemke, S. Iwan, and J. Korczak, "Usability of the parcel lockers from the customer perspective – the research in polish cities," *Transportation Research Procedia*, vol. 16, pp. 272–287, 2016.

[9] M. Peppel and S. Spinler, "The impact of optimal parcel locker locations on costs and the environment," *International Journal of Physical Distribution & Logistics Management*, vol. 52, no. 4, pp. 324–350, 2022.

[10] Y. H. Lin, Y. Wang, D. He, and L. H. Lee, "Last-mile delivery: Optimal locker location under multinomial logit choice model," *Transportation Research Part E: Logistics and Transportation Review*, vol. 142, p. 102059, 2020.

[11] Y. Lin, Y. Wang, L. H. Lee, and E. P. Chew, "Profit-maximizing parcel locker location problem under threshold luce model," *Transportation Research Part E: Logistics and Transportation Review*, vol. 157, p. 102541, 2022.

[12] E. Rodríguez-Esparza, E. Kampitakis, A. D. Masegosa, E. Onieva, and E. I. Vlahogianni, "Optimizing parcel locker locations under capacity and pick-up time constraints," May 2024. Preprint (not peer-reviewed), submitted to Transportation Research Part C: Emerging Technologies. SSRN working paper.

[13] M. Ding, Y. Song, and Y. Hu, "Supply–demand matching of smart parcel lockers in a residential area: Insights from tianjin," *Buildings*, vol. 13, no. 8, p. 2088, 2023.

[14] M. Kahr, "Determining locations and layouts for parcel lockers to support supply chain viability at the last mile," *Omega*, vol. 113, p. 102721, 2022.

[15] E. Rosca, F. Rusca, M. A. Rosca, and A. Rusca, "Performance analysis of automated parcel lockers in urban delivery: Combined agent-based–monte carlo simulation approach," *Logistics*, vol. 8, no. 2, p. 61, 2024.

[16] D. Sailer, R. Klein, and C. Steinhardt, "Dynamic demand management for parcel lockers," Sept. 2024. arXiv:2409.05061v2.

[17] Y. Deutsch and B. Golany, "A parcel locker network as a solution to the logistics last mile problem," *International Journal of Production Research*, vol. 56, no. 1-2, pp. 251–261, 2018.

[18] J. Krarup and P. M. Pruzan, "The simple plant location problem: Survey and synthesis," *European Journal of Operational Research*, vol. 12, pp. 36–81, Jan. 1983.

[19] M. Schnieder, C. Hinde, and A. West, "Combining parcel lockers with staffed collection and delivery points: An optimization case study using real parcel delivery data (london, uk)," *Journal of Open Innovation: Technology, Market, and Complexity*, vol. 7, no. 3, p. 183, 2021.

[20] T. Raviv and M. Gansterer, "Location, capacity, and layout problems of automated parcel lockers," in *Reference Module in Social Sciences*, Encyclopedia in Operations Management, pp. 1–20, Elsevier, 2025.

[21] L. Zhou, C. Li, C. Hu, and J. Du, "Parcel locker location problem with selectable volume sizes and heterogeneous customers in the last mile delivery," *Transportation Letters*, vol. 16, no. 9, pp. 1140–1154, 2024.

[22] M. El Amrani, Y. Benadada, and B. Gendron, "Generalization of capacitated p-median location problem: modeling and resolution," *WSEAS Transactions on Business and Economics*, vol. 14, pp. 430–444, 2017.

[23] Y. Wang, Y. Zhang, M. Bi, J. Lai, and Y. Chen, "A robust optimization method for location selection of parcel lockers under uncertain demands," *Mathematics*, vol. 10, p. 4289, 2022.

[24] M. Rabe, J. Gonzalez-Feliu, J. Chicaiza-Vaca, and R. D. Tordecilla, "Simulation-optimization approach for multi-period facility location problems with forecasted and random demands in a last-mile logistics application," *Algorithms*, vol. 14, no. 2, p. 41, 2021.

[25] F. M. Ottaviani, G. Zenezini, A. De Marco, and A. Carlin, "Locating automated parcel lockers (apl) with known customers' demand: A mixed approach proposal," *European Journal of Transport and Infrastructure Research*, vol. 23, no. 2, pp. 24–45, 2023.

[26] S. Mancini, M. Gansterer, and C. Triki, "Locker box location planning under uncertainty in demand and capacity availability," *Omega*, vol. 120, p. 102910, 2023.

[27] Z.-H. Che, T.-A. Chiang, and Y.-J. Luo, "Multiobjective optimization for planning the service areas of smart parcel locker facilities in logistics last mile delivery," *Mathematics*, vol. 10, no. 3, p. 422, 2022.

[28] R. Luo, S. Ji, and J. Yuanyuan, "An active-learning pareto evolutionary algorithm for parcel locker network design considering accessibility of customers," *Computers & Operations Research*, vol. 141, p. 105677, 2022.

[29] S. L. Hakimi, "Optimum locations of switching centers and the absolute centers and medians of a graph," *Operations Research*, vol. 12, no. 3, pp. 450–459, 1964.

[30] S. Boyd and J. Mattingley, "Branch and bound methods," tech. rep., Stanford University, Mar. 2007. Notes for EE364b, Winter 2006–2007.

[31] S. S. R. Shariff, N. H. Moin, and M. Omar, "An alternative heuristic for capacitated p-median problem (cpmp)," in *Proceedings of the 2013 IEEE Business Engineering and Industrial Applications Colloquium (BEIAC)*, pp. 916–921, IEEE, 2013.

[32] K. Fleszar and K. S. Hindi, "An effective vns for the capacitated p-median problem," *European Journal of Operational Research*, vol. 191, no. 3, pp. 612–622, 2008.

[33] M. Yaghini, M. Karimi, and M. Rahbar, "A hybrid metaheuristic approach for the capacitated p-median problem," *Applied Soft Computing*, vol. 13, no. 8, pp. 3922–3930, 2013.

[34] H. R. Lourenço, O. C. Martin, and T. Stützle, "Iterated local search," in *Handbook of Metaheuristics* (F. Glover and G. A. Kochenberger, eds.), vol. 57 of *International Series in Operations Research & Management Science*, pp. 321–353, Springer, 2003.

[35] S. Tadić, M. Krstić, v. Stević, and M. Veljović, "Locating collection and delivery points using the p-median location problem," *Logistics*, vol. 7, no. 1, p. 10, 2023.

[36] G. Koloch, "Postal optimization by three metaheuristics – a case study," *Acta Logistica*, vol. 12, no. 2, pp. 253–260, 2025.

[37] B. Ma, C.-C. Teo, and Y. D. Wong, "Location analysis of parcel locker network: Effects of spatial characteristics on operational performance," *Transportation Research Part E: Logistics and Transportation Review*, vol. 192, p. 103776, 2024.

[38] Z. Li and B. Li, "The data-driven optimization of parcel locker locations in a transit co-modal system with ride-pooling last-mile delivery," *Applied Sciences*, vol. 15, no. 9, p. 5217, 2025.

[39] P. Zhang and L. Wu, "An optimization model for parcel lockers layout," *Quest Journals: Journal of Software Engineering and Simulation*, vol. 10, no. 5, pp. 16–21, 2024.

[40] K. Gutenschwager, M. Rabe, and J. Chicaiza-Vaca, "Comparing direct deliveries and automated parcel locker systems with respect to overall co_2 emissions for the last mile," *Algorithms*, vol. 17, no. 1, p. 4, 2024.

[41] K. Hofer, S. Flucher, M. Fellendorf, M. Schadler, and N. Hafner, "Estimation of changes in customer's mobility behaviour by the use of parcel lockers," *Transportation Research Procedia*, vol. 47, pp. 425–432, 2020.

[42] V. Bonomi, R. Mansini, and R. Zanotti, "Last mile delivery with parcel lockers: Evaluating the environmental impact of eco-conscious consumer behavior," *IFAC PapersOnLine*, vol. 55, no. 5, pp. 72–77, 2022.

[43] G. Santos and A. Caranzo, "Potential distributional impacts of road pricing: A case study," *Case Studies on Transport Policy*, vol. 10, pp. 1–31, 2022.

[44] International Transport Forum, "The future of public transport funding," itf research report, ITF, OECD, Paris, 2024.

[45] A. Bilal and D. R. Känzig, "The macroeconomic impact of climate change: Global vs. local temperature," NBER Working Paper 32450, National Bureau of Economic Research, Cambridge, MA, May 2024.

[46] E. Rosca, F. C. Oprea, A. Ilie, S. Burciu, and F. Rusca, "Automated parcel locker configuration using discrete event simulation," *Systems*, 2025.

[47] I. B. Hovi, D. R. Pinchasik, B. Dong, H. Strømstad, and Ø. L. Brunstad, "Parcel lockers as delivery solution: Usage patterns, experiences and effects of network expansions," Tech. Rep. 1959/2023, Institute of Transport Economics (TØI), Norwegian Centre for Transport Research, Oslo, Norway, June 2023.

[48] D. R. Morrison, S. H. Jacobson, J. J. Sauppe, and E. C. Sewell, "Branch-and-bound algorithms: A survey of recent advances in searching, branching, and pruning," *Discrete Optimization*, vol. 19, pp. 79–102, 2016.